

Studienskript

A man in a dark suit is shown from the side, holding a white tablet. The background is a grayscale world map with a grid of binary code (0s and 1s) overlaid. There are bright light flares on the map. The entire image is framed by a thick orange border that has a tab-like shape at the top left and a tail at the bottom left.

REQUIREMENTS ENGINEERING

IREN01

iu

INTERNATIONALE
HOCHSCHULE

REQUIREMENTS ENGINEERING

IMPRESSUM

Herausgeber:
IU Internationale Hochschule GmbH
IU International University of Applied Sciences
Juri-Gagarin-Ring 152
D-99084 Erfurt

Postanschrift:
Albert-Proeller-Straße 15-19
D-86675 Buchdorf
media@iu.org
www.iu.de

IREN01
Versionsnr.: 003-2023-0817
N.N.

© 2023 IU Internationale Hochschule GmbH
Dieses Lernskript ist urheberrechtlich geschützt. Alle Rechte vorbehalten.
Dieses Lernskript darf in jeglicher Form ohne vorherige schriftliche Genehmigung der IU Internationale Hochschule GmbH (im Folgenden „IU“) nicht reproduziert und/oder unter Verwendung elektronischer Systeme verarbeitet, vervielfältigt oder verbreitet werden.
Die Autor:innen/Herausgeber:innen haben sich nach bestem Wissen und Gewissen bemüht, die Urheber:innen und Quellen der verwendeten Abbildungen zu bestimmen. Sollte es dennoch zu irrtümlichen Angaben gekommen sein, bitten wir um eine dementsprechende Nachricht.

INHALTSVERZEICHNIS

REQUIREMENTS ENGINEERING

Einleitung

Wegweiser durch das Studienskript	6
Basisliteratur	7
Weiterführende Literatur	8
Übergeordnete Lernziele	10

Lektion 1

Grundlagen und Begriffe des Requirements Engineering	11
1.1 Requirements Engineering im Softwareprozess	12
1.2 Kernaktivitäten im Requirements Engineering	13
1.3 Was ist eine Anforderung?	15

Lektion 2

Ermittlung von Anforderungen	21
2.1 Bestimmung des Systemkontextes	22
2.2 Bestimmung der Quellen von Anforderungen	24
2.3 Auswählen der geeigneten Ermittlungstechniken	28
2.4 Anforderungen unter Einsatz der Techniken ermitteln	31

Lektion 3

Ausgewählte Ermittlungstechniken	35
3.1 Kreativitätstechniken	39
3.2 Befragungstechniken	44
3.3 Beobachtungstechniken	47
3.4 Prototyping	48

Lektion 4

Dokumentation von Anforderungen	55
4.1 Aktivitäten zur Dokumentation von Anforderungen	56
4.2 Typische Elemente der Anforderungsdokumentation	58
4.3 Dokumentationsformen	62

Lektion 5	
Modellierung von Prozessen	69
5.1 Grundlagen und Begriffe	70
5.2 Modellierung mit der Business Process Model and Notation	76
5.3 Modellierung mit Ereignisgesteuerten Prozessketten	85
Lektion 6	
Modellierung von Systemen	95
6.1 Grundlagen Unified Modeling Language	96
6.2 UML-Use-Case-Diagramm	99
6.3 UML-Aktivitätsdiagramm	102
6.4 UML-Klassendiagramm	106
6.5 UML-Zustandsdiagramm	112
Lektion 7	
Prüfen und Abstimmen von Anforderungen	117
7.1 Aktivitäten zum Prüfen und Abstimmen von Anforderungen	118
7.2 Prüfkriterien	120
7.3 Prüfprinzipien	123
7.4 Prüftechniken	126
7.5 Abstimmen von Anforderungen	129
Lektion 8	
Management von Anforderungen und Techniken zur Priorisierung	133
8.1 Verwalten von Anforderungen	134
8.2 Techniken zur Priorisierung von Anforderungen	140
Verzeichnisse	
Literaturverzeichnis	150
Abbildungsverzeichnis	152

EINLEITUNG

HERZLICH WILLKOMMEN

WEGWEISER DURCH DAS STUDIENSKRIPT

Dieses Studienskript bildet die Grundlage Ihres Kurses. Ergänzend zum Studienskript stehen Ihnen weitere Medien aus unserer Online-Bibliothek sowie Videos zur Verfügung, mit deren Hilfe Sie sich Ihren individuellen Lern-Mix zusammenstellen können. Auf diese Weise können Sie sich den Stoff in Ihrem eigenen Tempo aneignen und dabei auf lernspezifische Anforderungen Rücksicht nehmen.

Die Inhalte sind nach didaktischen Kriterien in Lektionen aufgeteilt, wobei jede Lektion aus mehreren Lernzyklen besteht. Jeder Lernzyklus enthält jeweils nur einen neuen inhaltlichen Schwerpunkt. So können Sie neuen Lernstoff schnell und effektiv zu Ihrem bereits vorhandenen Wissen hinzufügen.

In der IU Learn App befinden sich am Ende eines jeden Lernzyklus die Interactive Quizzes. Mithilfe dieser Fragen können Sie eigenständig und ohne jeden Druck überprüfen, ob Sie die neuen Inhalte schon verinnerlicht haben.

Sobald Sie eine Lektion komplett bearbeitet haben, können Sie Ihr Wissen auf der Lernplattform unter Beweis stellen. Über automatisch auswertbare Fragen erhalten Sie ein direktes Feedback zu Ihren Lernfortschritten. Die Wissenskontrolle gilt als bestanden, wenn Sie mindestens 80 % der Fragen richtig beantwortet haben. Sollte das einmal nicht auf Anhieb klappen, können Sie die Tests beliebig oft wiederholen.

Wenn Sie die Wissenskontrolle für sämtliche Lektionen gemeistert haben, führen Sie bitte die abschließende Evaluierung des Kurses durch.

Die IU Internationale Hochschule ist bestrebt, in ihren Skripten eine gendersensible und inklusive Sprache zu verwenden. Wir möchten jedoch hervorheben, dass auch in den Skripten, in denen das generische Maskulinum verwendet wird, immer Frauen und Männer, Inter- und Trans-Personen gemeint sind sowie auch jene, die sich keinem Geschlecht zuordnen wollen oder können.

BASILITERATUR

Pohl, K./Rupp, C. (2021): *Basiswissen Requirements Engineering – Aus- und Weiterbildung zum "Certified Professional for Requirements Engineering"*. 5. Auflage, dpunkt.verlag, Heidelberg. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=cat05114a&AN=ihb.52945&site=eds-live&scope=site>

Rupp, C./die SOPHISTen (2021): *Requirements-Engineering und -Management: Aus der Praxis von klassisch bis agil*. 7. Auflage, Carl Hanser, München. – 6. Auflage 2014 vorhanden: <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=cat05114a&AN=ihb.23765&site=eds-live&scope=site>

Rupp, C./die SOPHISTen/Queins, S. (2012): *UML 2 glasklar: Praxiswissen für die UML-Modellierung*. 4. Auflage, Carl Hanser, München. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=cat05114a&AN=ihb.23576&site=eds-live&scope=site>

WEITERFÜHRENDE LITERATUR

Miller, L. A./Thomas, J. C. Jr. (1977): *Behavioral issues in the use of interactive systems*. In: International Journal of Man-Machine Studies, 9. Jg., Heft 5, S. 509–536. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=edselp&AN=S0020737377800023&site=eds-live&scope=site>

LEKTION 1

Lauenroth, K. (2011): *Eine kleine praktische Philosophie über das Requirements Engineering*.

LEKTION 2

Rupp, C./die SOPHISTen (2021): *Requirements-Engineering und -Management: Aus der Praxis von klassisch bis agil*. 7. Auflage, Carl Hanser, München, S. 93–118, 129–164. – 6. Auflage 2014 vorhanden: <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=cat05114a&AN=ihb.23765&site=eds-live&scope=site>

LEKTION 3

Nöllke, M. (2020): *Kreativitätstechniken*. 8. Auflage, Haufe Lexware, München. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=edswis&AN=edswis.HAUF9783648140239127&site=eds-live&scope=site>

Safayeni, F. et al. (2008): *Requirement engineering in new product development*. In: Communications of the ACM, 51. Jg., Heft 3, S. 77–82. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=bsu&AN=31317844&site=eds-live&scope=site>

LEKTION 4

Ludewig, J./Lichter, H. (2013): *Software Engineering: Grundlagen, Menschen, Prozesse, Techniken*. 3. Auflage, dpunkt.verlag, Heidelberg, Kapitel 12. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=nlebk&AN=908638&site=eds-live&scope=site>

LEKTION 5

Domingo, D./Martins, F. (2017): *Using BPMN to model Internet of Things behavior within business process*. In: International Journal of Information Systems and Project Management, 5. Jg., Heft 4, S. 39–51. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=edsdoj&AN=edsdoj.99b84ee602074fd59041a7bb5f0e1184&site=eds-live&scope=site>

Weiland, A./Meuche, Th. (2013): *BWL in Fallstudien: Ein einführendes Lehrbuch*. Schäffer-Poeschel, Stuttgart, Kapitel 4. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=nlebk&AN=1047842&site=eds-live&scope=site>

LEKTION 6

Object Management Group (OMG): *Unified Modeling Language UML*. (Im Internet verfügbar).

LEKTION 7

Ludewig, J./Lichter, H. (2013): *Software Engineering: Grundlagen, Menschen, Prozesse, Techniken*. 3. Auflage, dpunkt.verlag, Heidelberg, Kapitel 13. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=nlebk&AN=908638&site=eds-live&scope=site>

LEKTION 8

Preußig, J. (2018): *Agiles Projektmanagement, Scrum, User Stories, Timeboxing & Co*. 2. Auflage, Haufe-Lexware, Freiburg. <http://search.ebscohost.com.pxz.iubh.de:8080/login.aspx?direct=true&db=edshbz&AN=edshbz.DE.605.HBZ01.022649989&site=eds-live&scope=site>

ÜBERGEORDNETE

LERNZIELE

Die frühen Phasen der Softwareentwicklung sind maßgeblich davon gekennzeichnet, dass fachliche und technische Anforderungen (engl.: requirements) an das IT-System zu ermitteln sind. Die Anforderungsermittlung muss äußerst umsichtig betrieben werden, weil alle folgenden Aktivitäten im Softwareentwicklungsprozess auf der Grundlage der dokumentierten Anforderungen geplant und durchgeführt werden.

Ziel des Kurses **Requirements Engineering** ist es, Vorgehensweisen, Methoden und Modelle zu vermitteln, die eine strukturierte und methodische Ermittlung und Dokumentation von Anforderungen an betriebliche Informationssysteme ermöglichen.

LEKTION 1

GRUNDLAGEN UND BEGRIFFE DES REQUIREMENTS ENGINEERING

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- was Requirements Engineering bedeutet und was dessen Ziele sind.
- welche Kernaktivitäten beim Requirements Engineering durchgeführt werden.
- was als Anforderung bezeichnet wird und welche Arten von Anforderungen unterschieden werden können.

1. GRUNDLAGEN UND BEGRIFFE DES REQUIREMENTS ENGINEERING

Einführung

Aufgabe des Requirements Engineering ist es, herauszufinden und festzulegen, was ein zu erstellendes System alles können soll, welche Funktionen im Detail von ihm gefordert werden und für den Wissenstransfer zum Entwicklungsteam zu sorgen. Dazu wird in dieser Lektion zunächst der Begriff Requirements Engineering vorgestellt, bevor die für das Requirements Engineering typischen Aktivitäten Ermittlung, Dokumentation sowie Prüfen und Abstimmen eingeführt werden. Darüber hinaus wird erläutert, was eine Anforderung eigentlich ist und welche Arten von Anforderungen unterschieden werden.

1.1 Requirements Engineering im Softwareprozess

Bevor in einem Softwareprojekt mit der Konstruktion der Software begonnen werden kann, muss zunächst das Ziel bestimmt werden, das mit der Neuerstellung oder der Veränderung von IT-Systemen erreicht werden soll. Eine grobe Zielbestimmung erfolgt dabei bereits in der Phase „Planung“ im Softwarelebenszyklus. Ausgehend von diesem Ziel, müssen nun Stück für Stück alle Funktionen und Eigenschaften der Systeme ermittelt werden, welche nach Abschluss der Entwicklung unterstützt werden sollen. In der Regel werden in einem Projekt immer mehrere IT-Systeme gemeinsam verändert, daher bezeichnet der Begriff „System“ im Folgenden vereinfachend die Menge aller zu verändernden Systeme.

Beispiel

In der Planungsphase wird beschlossen, ein System einzuführen, mit dem die Beantragung und Abrechnung von Dienstreisen durchgängig digitalisiert werden kann, um vollständig auf Papierformulare zu verzichten. Ausgehend von diesem allgemeinen Ziel des Systems müssen nun die konkreten Funktionen ermittelt, dokumentiert und abgestimmt werden, die das Softwareentwicklungsteam dann umsetzen soll.

Im Software Engineering werden die geforderten Funktionen und Eigenschaften „Anforderungen“ (engl.: requirements) genannt. Die Aktivitäten zur Ermittlung, Dokumentation, Abstimmung und Verwaltung werden unter dem Begriff „Requirements Engineering“ (abgekürzt: RE) zusammengefasst. Angelehnt an Pohl und Rupp (2015, S. 12) kann man Requirements Engineering folgendermaßen definieren:

Requirements Engineering (RE) ist ein kooperativer, iterativer, inkrementeller Prozess, dessen Ziel ist zu gewährleisten, dass:

- alle relevanten Anforderungen bekannt und in erforderlichem Detaillierungsgrad verstanden sind.
- alle Anforderungen konform zu den Dokumentationsvorschriften dokumentiert bzw. konform zu den Spezifikationsvorschriften spezifiziert sind.
- die involvierten Stakeholder ausreichende Übereinstimmung über die bekannten Anforderungen erzielen.

Folgende Merkmale zum Requirements Engineering lassen sich aus dieser Definition ableiten:

In Aktivitäten des RE sind immer mehrere Personen oder Personengruppen involviert, die miteinander kooperieren müssen. Konkret handelt es sich dabei um Personen oder Personengruppen, deren Arbeits- und Einflussbereich vom zu erstellenden System beeinflusst wird oder mit dem System in Kontakt kommt, aber auch alle Personen, deren Arbeits- und Einflussbereich am Erstellungsprozess des Systems miteinbezogen wird. Die Menge aller betroffenen bzw. beteiligten Personen wird „Stakeholder“ genannt.

Requirements Engineering ist keine einzelne, abgeschlossene Aktivität zu Beginn eines Softwareprojekts. RE erfolgt in mehreren Zyklen (sogenannten Iterationen). Durch die mehrfache Durchführung von Aktivitäten des RE werden Anforderungen an das System Stück für Stück ermittelt und verfeinert. Da Software Engineering ein stark erkenntnisgetriebener Prozess ist, erfolgt RE in der Regel projektbegleitend. So wird beispielsweise bei der Softwareentwicklung nach Scrum (Schwaber/Sutherland 2017) das Requirements Engineering durch die Rolle Product Owner verantwortet, die über die gesamte Laufzeit des Projekts tätig ist.

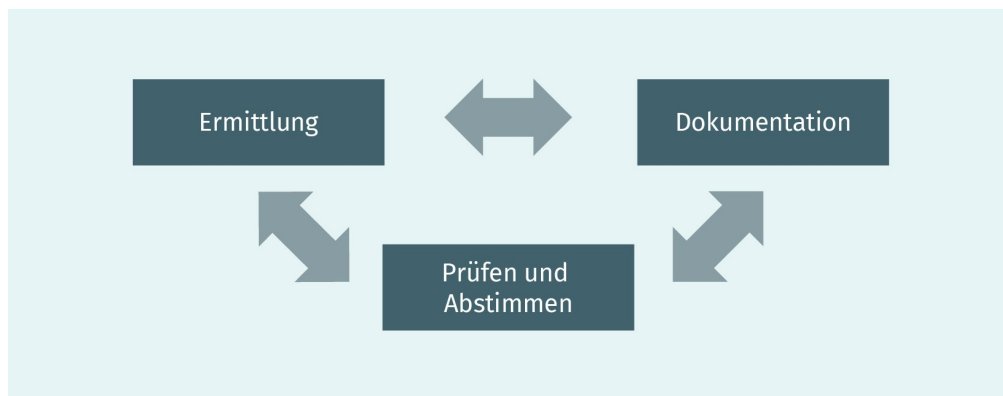
1.2 Kernaktivitäten im Requirements Engineering

Aus der oben genannten Definition von RE lassen sich die in Abbildung 1 dargestellten drei Kernaktivitäten des Requirements Engineering ableiten:

- Ermittlung von Anforderungen,
- Dokumentation von Anforderungen und
- Prüfen und Abstimmen von Anforderungen.

Zu diesen Aktivitäten lässt sich keine feste Reihenfolge beschreiben, denn der verantwortliche Requirements Engineer entscheidet immer in Abhängigkeit der aktuellen Projektsituation, welche Art von Aktivität als nächstes erfolgt.

Abbildung 1: Kernaktivitäten im Requirements Engineering



Quelle: erstellt im Auftrag der IU, 2023.

Ermittlung von Anforderungen

Ziel der Aktivität „Ermittlung“ von Anforderungen ist die Identifikation der Anforderungen an das System, die zur Erreichung des Ziels benötigt werden. Dafür müssen die Anforderungen erkannt und in dem für die aktuelle Projektsituation erforderlichen Detaillierungsgrad verstanden werden.

Beispiel

Beim Start des Projekts ist zunächst wichtig, alle relevanten grundsätzlichen Funktionsbereiche zu ermitteln, ohne dabei bereits auf Details einzugehen (z. B. „Anlegen einer Dienstreise“, „Bearbeiten einer Dienstreise“, „Anlegen einer Reisekostenabrechnung“, „Erstattung von Spesen“, „Genehmigen der Spesenerstattung“). Je weiter fortgeschritten ein Projekt ist, desto detaillierter müssen die einzelnen Anforderungen beschrieben werden. So muss zum Funktionsbereich „Bearbeiten einer Dienstreise“ unter anderem genau festgelegt werden, welche Eingabefelder zur Verfügung stehen, welche Felder Pflichtfelder sind, welche Bedingungen eingehalten werden müssen und welche Personen überhaupt Änderungen vornehmen dürfen.

Dokumentation von Anforderungen

Ziel der Aktivität „Dokumentation“ ist es, dafür zu sorgen, dass der aktuelle Erkenntnisstand für alle Stakeholder gesichert wird und sich jeder Beteiligte zu jeder Zeit einen Überblick verschaffen kann. Anforderungsdokumente sind in der organisationsübergreifenden Softwareentwicklung Bestandteil von rechtsgültigen Verträgen, auf deren Basis geprüft wird, ob die vereinbarte Leistung durch den Softwarehersteller auch tatsächlich erbracht wurde.

Um komplizierte Strukturen und Zusammenhänge geeignet zu dokumentieren, werden im Software Engineering ergänzend zur Beschreibung von Anforderungen in Textform häufig grafische Modelle eingesetzt (sogenannte Softwaremodelle). Die konkrete Dokumentationsform ist jedoch immer abhängig von der Art der Anforderungen, die dokumentiert wer-

den soll, vom Zweck und vom Personenkreis, für den die Dokumentation bestimmt ist, und von Vorschriften oder Vereinbarungen zum Dokumentationsformat, die im Rahmen des Projekts gelten.

Prüfen und Abstimmen von Anforderungen

Da alle weiteren Aktivitäten im Software Engineering direkt von den ermittelten und dokumentierten Anforderungen abhängen, müssen die Anforderungen vor der Freigabe zur Umsetzung geprüft und mit den relevanten Stakeholdern abgestimmt werden. Ziel der Prüfung ist, die Qualität der Menge der dokumentierten Anforderung hinsichtlich der Kriterien Inhalt, Dokumentation und Abgestimmtheit sicherzustellen. Mit der Prüfung soll erreicht werden, dass die Anforderungen eine hohe Dokumentationsqualität haben und beispielsweise Missverständnisse durch Mehrdeutigkeiten vermieden und sich widersprechende oder konkurrierende Anforderungen identifiziert werden.

Werden bei einer Prüfung Konflikte oder Widersprüche identifiziert, müssen diese unter Einbeziehung der Stakeholder aufgelöst werden. Insbesondere in Projekten zur Erstellung von industriellen Informationssystemen sind viele verschiedene Stakeholder einzubeziehen. Im Verlauf des Requirements Engineering werden daher in der Regel konkurrierende oder sich gegenseitig widersprechende Anforderungen ermittelt und dokumentiert. Daher nimmt das Konfliktmanagement im Abstimmungsprozess eine wichtige Rolle ein.

Ergänzend zu den oben vorgestellten Kernaktivitäten wird in der Literatur (beispielsweise bei Pohl/Rupp 2015) auch die Verwaltung von Anforderungen als Kernaktivität genannt.

1.3 Was ist eine Anforderung?

In diesem Lernzyklus wird der Begriff Anforderung genauer betrachtet und herausgestellt, welche verschiedenen Arten von Anforderungen es gibt. Mit dem Begriff Anforderungen werden die geforderten Funktionen und Eigenschaften zu IT-Systemen bezeichnet, die nötig sind, um ein bestimmtes Ziel zu erreichen. Für ein professionelles Requirements Engineering ist es allerdings wichtig, Anforderungen an Systeme von fachlichen Problemen zu unterscheiden. Um diesen Unterschied zu verdeutlichen, führen Sie bitte das folgende Gedankenexperiment durch:



GEDANKENEXPERIMENT

Stellen Sie sich vor, Sie sind mit einer Freundin oder einem Freund zu einer bestimmten Uhrzeit in Ihrem Lieblingsrestaurant verabredet. Sie müssen vorher allerdings noch schnell einen Brief zum Postbriefkasten bringen. Um sicherzugehen, dass Sie pünktlich zu ihrer Verabredung kommen, möchten Sie nach dem Einwurf des Briefs gerne die aktuelle Uhrzeit wissen. Welches System bzw. welche Lösung fällt Ihnen spontan ein, um das Problem zu lösen? (Anmerkung: Mit Problem ist hier die Bestimmung der Uhrzeit gemeint.)

Schritt 1)

Notieren Sie sich Ihr/e System/Lösung zur Bestimmung der Uhrzeit hier:

_____.

Schritt 2)

Überlegen Sie kurz, was Ihrer Meinung nach die wichtigsten geforderten Funktionen und Eigenschaften an Ihr System sind und notieren Sie sich diese in Form von Stichpunkten hier:

- _____
- _____
- _____
- _____
- _____

Schritt 3)

Am Ende dieser Lektion haben wir mögliche Systeme zur Lösung des Problems aufgezählt. Schauen Sie sich diese Liste kurz an und vergleichen Sie zu jeder der dort genannten Lösungen, inwieweit diese die gleichen oder abweichenden Funktionen und Eigenschaften zu der von Ihnen genannten Lösung haben.

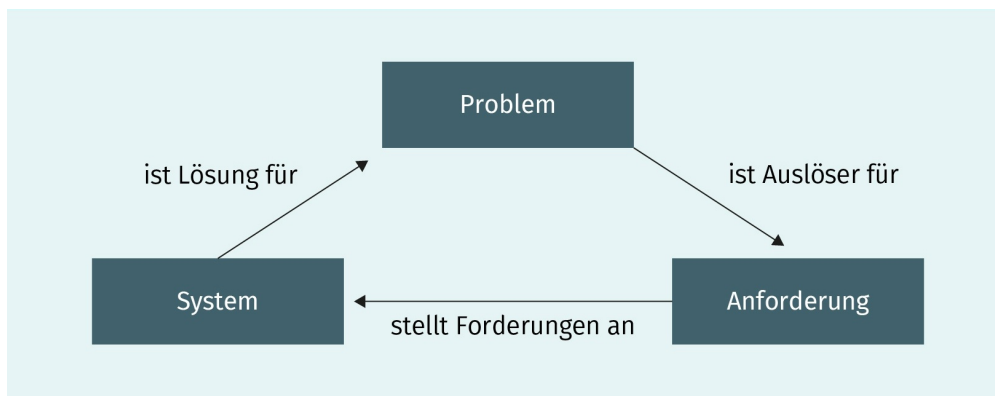
Erkenntnis)

In Schritt 1 haben Sie sich auf ein konkretes System zur Lösung eines Problems festgelegt und daraufhin in Schritt 2 Funktionen und Eigenschaften gezielt auf das von Ihnen ausgesuchte System beschrieben. Wenn im Rahmen von Aktivitäten des Requirements Engineering nun Personen zu Anforderungen befragt werden, werden häufig nur genau die Anforderungen genannt, die sich bereits auf das konkrete System beziehen, welches sich die Personen als mögliche Lösung vorstellen. Dabei besteht die Gefahr, dass mögliche alternative Lösungen von Anfang an ausgeblendet und nicht erkannt werden.

Zusammenhang zwischen Problem, Anforderung und System

Im Requirements Engineering ist es daher wichtig, dass von allen Beteiligten der in Abbildung 2 gezeigte Zusammenhang zwischen dem eigentlichen Problem, den kommunizierten Anforderungen und dem System (der Lösung) verstanden wird. Eine ausführliche Herleitung und Diskussion dazu kann bei Lauenroth (2011) nachgelesen werden.

Abbildung 2: Zusammenhang zwischen Problem, Anforderung und System



Quelle: erstellt im Auftrag der IU, 2023 in Anlehnung an Lauenroth 2011, S. 3.

Die Anforderungen beschreiben notwendige Funktionen und Eigenschaften eines Systems zur Lösung eines Problems. Das Problem beschreibt, was in der realen Welt verändert oder erreicht werden soll, also ein bestimmtes Ziel oder einen bestimmten Zweck, der mit dem System erreicht werden soll. Im Gedankenexperiment ist beispielsweise die Bestimmung der Uhrzeit das Problem. Das System stellt eine konkrete Lösung dar, mit dem das Problem gelöst bzw. das Ziel oder der Zweck erreicht werden kann.

Das Problem bzw. die fachliche Fragestellung ist in der Regel erst einmal völlig unabhängig von Anforderungen und Lösungen. Es gibt immer mehrere Ansätze, mit denen das Problem gelöst werden kann. Die Anforderungen bilden dabei die Beziehungen zwischen dem Problem und der Lösung, indem sie konkrete Eigenschaften der Lösung beschreiben, um das Problem zu lösen. Das bedeutet auch, dass Anforderungen niemals oder nur sehr schwer unabhängig von der konkreten Lösungsidee formuliert werden können. Daher bestimmt eine (ggf. vorzeitige) Festlegung der Lösung die Formulierung der Anforderungen. So haben Sie in Schritt 2 des Gedankenexperiments Eigenschaften zu der von Ihnen gewählten Lösungsidee beschrieben. Hätten Sie sich für eine andere Lösung entschieden, würde sich die Menge der ermittelten Anforderungen an Ihr System wahrscheinlich unterscheiden.

Insbesondere in der Kommunikation mit Anwendern werden die Ebenen Problem, Anforderung und Lösung bei der Anforderungsanalyse vermischt. Aus diesem Grund ist es unbedingt notwendig, bei Kommunikationssituationen im Requirements Engineering regelmäßig zu reflektieren und zu bestimmen, auf welcher Ebene gerade kommuniziert wird. Weiterhin muss zwingend darauf geachtet werden, dass zu jeder formulierten Anforderung auch das zugehörige fachliche Problem formuliert wird. Anderenfalls besteht die Gefahr, dass möglicherweise besser geeignete Lösungen gar nicht erst identifiziert werden können.

Arten von Anforderungen

Grundsätzlich lassen sich im Requirements Engineering verschiedene Arten von Anforderungen unterscheiden: funktionale Anforderungen, Qualitätsanforderungen und Randbedingungen.

Funktionale Anforderungen

Funktionale Anforderungen sind Anforderungen, die vom System bereitzustellende Funktionen definieren. Beispiele für funktionale Anwendungen sind das Berechnen einer Laufzeit, das Anzeigen aller verfügbarer Waren im Bestand oder das Versenden einer Bestellung per E-Mail. Dabei handelt es sich um (fachliche) Funktionen, die durch ein System unterstützt werden sollen.

Qualitätsanforderungen

Qualitätsanforderungen hingegen legen qualitative Eigenschaften fest, die das System unterstützen müssen. Mit ihnen wird keine neue Funktion beschrieben, sondern es werden bereits beschriebene funktionale Anforderungen um qualitative und quantitative Eigenschaften ergänzt. Beispiele hierfür sind die Unterstützung von 1.000 Bestellvorgängen pro Sekunde, die verschlüsselte Speicherung von personenbezogenen Daten oder eine Reaktionsgeschwindigkeit des Systems von weniger als einer Sekunde.

Randbedingungen

Mit den Randbedingungen werden organisatorische oder technische Vorgaben beschrieben, die durch das zu erstellende System erfüllt werden müssen. Typische Randbedingungen sind Gesetze oder regulatorische Vorschriften von Behörden, firmeninterne Vorgaben und Richtlinien wie ein Corporate Style Guide oder die Vorgabe eines technischen Rahmens wie die Programmiersprache oder die Zielplattform wie Windows, Linux, Android oder iOS.

Werden Qualitätsanforderungen oder Randbedingungen zu spät oder gar nicht erkannt, stellt das in der Regel ein großes Risiko dar. Im Unterschied zu vielen funktionalen Anforderungen ist die späte oder nachträgliche Umsetzung dieser Anforderungen sehr aufwendig und kann zu einer vollständigen Neukonzeption und Neuentwicklung des Systems führen.



ALTERNATIVE LÖSUNGEN ZUM GEDANKENEXPERIMENT

- mechanische Uhr
- elektrische Uhr
- Uhr im Mobiltelefon
- telefonische Zeitansage
- Internetseite
- Zeitstempel im Ticketentwerter für Bustickets
- eine Person ansprechen und nach der Uhrzeit fragen



ZUSAMMENFASSUNG

Bevor mit der Konstruktion oder der Veränderung von Teilen des Systems begonnen werden kann, müssen Funktionen und Eigenschaften des Systems ermittelt werden, welche das System nach Abschluss der Entwicklung unterstützen soll. Die Aktivitäten zur Ermittlung, Dokumentation, Abstimmung und Verwaltung von Anforderungen werden unter dem Begriff „Requirements Engineering“ (abgekürzt RE) zusammengefasst. Im RE sind alle relevanten Stakeholder des Systems involviert. Durch mehrfache Durchführung von Aktivitäten des RE (Iterationen) werden Anforderungen an das System ermittelt und verfeinert.

Mit dem Begriff Anforderungen werden die geforderten Funktionen und Eigenschaften zu IT-Systemen bezeichnet, die nötig sind, um ein bestimmtes Ziel zu erreichen. Für ein professionelles Requirements Engineering ist es allerdings wichtig, Anforderungen an Systeme von fachlichen Problemen zu unterscheiden. Im Requirements Engineering werden drei verschiedene Arten von Anforderungen unterschieden. Funktionale Anforderungen sind Anforderungen, die vom System bereitzustellende Funktionen definieren. Qualitätsanforderungen hingegen legen qualitative Eigenschaften fest, die das System unterstützen müssen. Mit den Randbedingungen werden organisatorische oder technische Vorgaben beschrieben, die durch das zu erstellende System erfüllt werden müssen.

LEKTION 2

ERMITTLUNG VON ANFORDERUNGEN

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- was ein Systemkontext ist, was die Kontextgrenzen sind und wie man dies bestimmt.
- welche Quellen es zur Ermittlung von Anforderungen gibt.
- welche Kategorien es zur Einordnung von Techniken für die Ermittlung von Anforderungen gibt.
- was bei der Wahl geeigneter Techniken zur Anforderungsermittlung zu berücksichtigen ist.
- wie Anforderungen unter Einsatz der Techniken ermittelt werden können.

2. ERMITTLUNG VON ANFORDERUNGEN

Einführung

Die Ermittlung von Anforderungen ist die Kernaktivität im Requirements Engineering, in der die funktionalen Anforderungen, die Qualitätsanforderungen und die Randbedingungen für ein System identifiziert und bewertet werden. Das System, für das die Anforderungen erhoben werden, muss sich später in ein bestehendes Umfeld eingliedern. Daher ist zunächst die Umgebung des Systems zu analysieren, um die nötigen Quellen für Anforderungen an das System zu definieren. Aus den Quellen werden Anforderungen durch den Einsatz von Ermittlungstechniken erhoben.

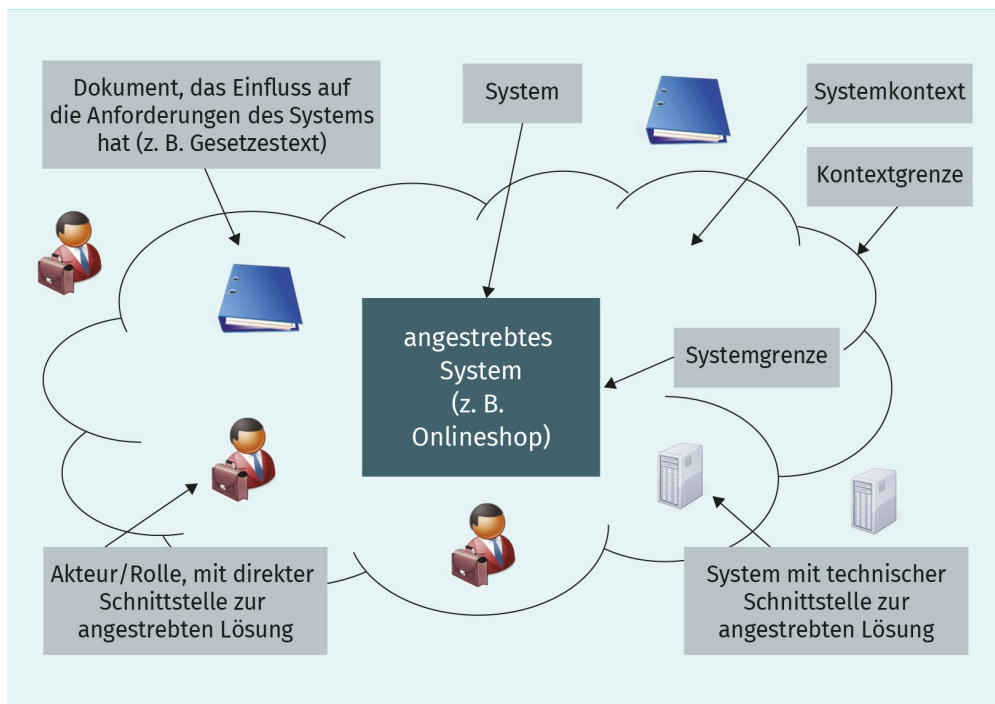
Zur systematischen Ermittlung von Anforderungen muss der Requirements Engineer die folgenden vier Schritte durchführen:

1. **Systemkontext bestimmen:** Es wird analysiert, welche Stakeholder und welche anderen Systeme direkte Abhängigkeiten zu dem erstellenden System haben, um eine Idee über die konkreten Quellen für Anforderungen zu bekommen.
2. **Quellen für Anforderungen ermitteln:** Typische Quellen für Anforderungen sind Stakeholder, Dokumente (z. B. Gesetze, Richtlinien) und andere Systeme (z. B. abzulösende Altsysteme oder Konkurrenzsysteme).
3. **Geeignete Ermittlungstechniken auswählen:** Je nach Anforderungsquelle, Projektsituation und Art der Anforderungen muss eine geeignete Ermittlungstechnik oder eine Kombination aus verschiedenen Ermittlungstechniken ausgewählt werden (z. B. Befragungstechnik, Kreativitätstechnik, Beobachtungstechnik, Erstellen von GUI-Prototypen).
4. **Anforderungen unter Einsatz der Techniken ermitteln:** Aus den bestimmten Quellen mit den gewählten Ermittlungstechniken werden Anforderungen in dem für die aktuelle Situation erforderlichen Detailgrad ermittelt.

2.1 Bestimmung des Systemkontextes

Abbildung 3 zeigt den Zusammenhang zwischen einem System und seinem Systemkontext.

Abbildung 3: Zusammenhang von System, Systemkontext und irrelevanter Umgebung



Quelle: erstellt im Auftrag der IU, 2023.

Der Systemkontext ist der relevante Teil der Systemumgebung, der zur Definition und zum Verständnis von Anforderungen analysiert werden muss. Im Systemkontext befinden sich Stakeholder und Umsysteme, die direkt mit dem zu entwickelnden System interagieren.

Stakeholder verfolgen bestimmte Ziele (z. B. Waren im Onlineshop bestellen) mit der Nutzung des Systems. Über technische Schnittstellen zu Umsystemen können die Funktionen anderer Systeme eingebunden werden, wie z. B. das Prüfen von Adressdaten. Dokumente können auch Teil des Systemkontextes sein und Randbedingungen für die Entwicklung des Systems liefern.

Die Systemgrenze trennt das System von seiner relevanten Umgebung und ist nicht von Beginn an stabil. Durch die Hinzunahme oder den Ausschluss von Anforderungen verändern sich der funktionale Umfang des Systems und damit auch die Systemgrenze. Die Prüfung von Adressdaten beispielsweise kann entweder im System oder durch ein Umsystem umgesetzt werden. Wird die Prüfung vom System umgesetzt, liegt die Funktionalität innerhalb der Systemgrenze, wird die Prüfung jedoch über eine Schnittstelle von einem Umsystem bezogen, liegt die Funktionalität außerhalb der Systemgrenze.

Neben der Systemgrenze gibt es noch die Kontextgrenze, die den Systemkontext von der für das System irrelevanten Umgebung separiert. Die Analyse, was zum Systemkontext gehört und was nicht, liefert Aufschluss darüber, welche Quellen für die Anforderungen relevant sind und welche nicht. Alles, was sich in der irrelevanten Umgebung befindet,

verfolgt keine Ziele und formuliert keine Randbedingungen für das angestrebte System. Jedoch ist auch die Kontextgrenze nicht von Beginn an stabil, denn häufig wird erst im Verlauf des Projekts klar, ob beispielsweise Gesetze relevant sind oder nicht.

2.2 Bestimmung der Quellen von Anforderungen

Wie zuvor beschrieben, lassen sich Anforderungsquellen durch die Analyse des Systemkontextes bestimmen. Im Systemkontext finden sich Dokumente, bereits existierende Systeme und Stakeholder.

Stakeholder sind die wichtigste Quelle für Anforderungen. Beispiele für mögliche Stakeholder können Auftraggeber, Benutzer, Mitarbeiter der Rechtsabteilung oder auch Architekten und der IT-Betrieb sein. Schon allein die Menge der unterschiedlichen Stakeholder gibt einen Hinweis darauf, dass deren Interessen und der Grad der möglichen Einflussnahme auf das System unterschiedlich sein können. Der Auftraggeber hat eine starke Machtposition, bestimmt dieser doch als Geldgeber, welche Anforderungen realisiert werden. Nicht selten entsteht daraus ein Interessenkonflikt. Vernachlässigt der Auftraggeber die Qualität zugunsten geringerer Entwicklungskosten, trifft dies in erster Linie die Anwender. Diese haben in der Regel keine Budgetverantwortung, legen jedoch besonders hohen Wert auf die Bedienbarkeit und den Funktionsumfang. Der Projektleiter hat vor allem die Zeit im Fokus, sodass insbesondere Qualität und Funktionalität niedriger priorisiert werden könnten, wenn die Zeit knapp wird.

Es ist also im Requirements Engineering darauf zu achten, Stakeholder nicht nur zu identifizieren, sondern auch Motive und den Grad der Einflussnahme auf das jeweilige Projekt zu bestimmen. Dies geschieht im Rahmen des Stakeholdermanagements.

Die Art der Dokumente, die relevante Anforderungen beinhalten können, ist höchst unterschiedlich. Neben unternehmensinternen Dokumenten wie Leitlinien, Unternehmens- oder IT-Strategien, Nachhaltigkeitsberichten und Sicherheitskonzepten, können öffentliche Dokumente wie Standards und Best-Practices-Quellen für Anforderungen darstellen. Nicht zuletzt sind in Gesetzen, Verordnungen, Erlassen und Richtlinien wichtige Anforderungen für ein System auffindbar. Darüber hinaus lassen sich in der Dokumentation von Altsystemen Anforderungen für ein System finden, das das alte System ablösen soll. Ein typisches Risiko ist das Übersehen von unbekannten, aber wichtigen gesetzlichen Vorgaben, beispielsweise zum Datenschutz. Diese haben aber oft weitreichenden Einfluss auf die Gestaltung des Systems. Werden diese zu spät erkannt, führt das zu großen Verzögerungen und hohen Überarbeitungsaufwänden.

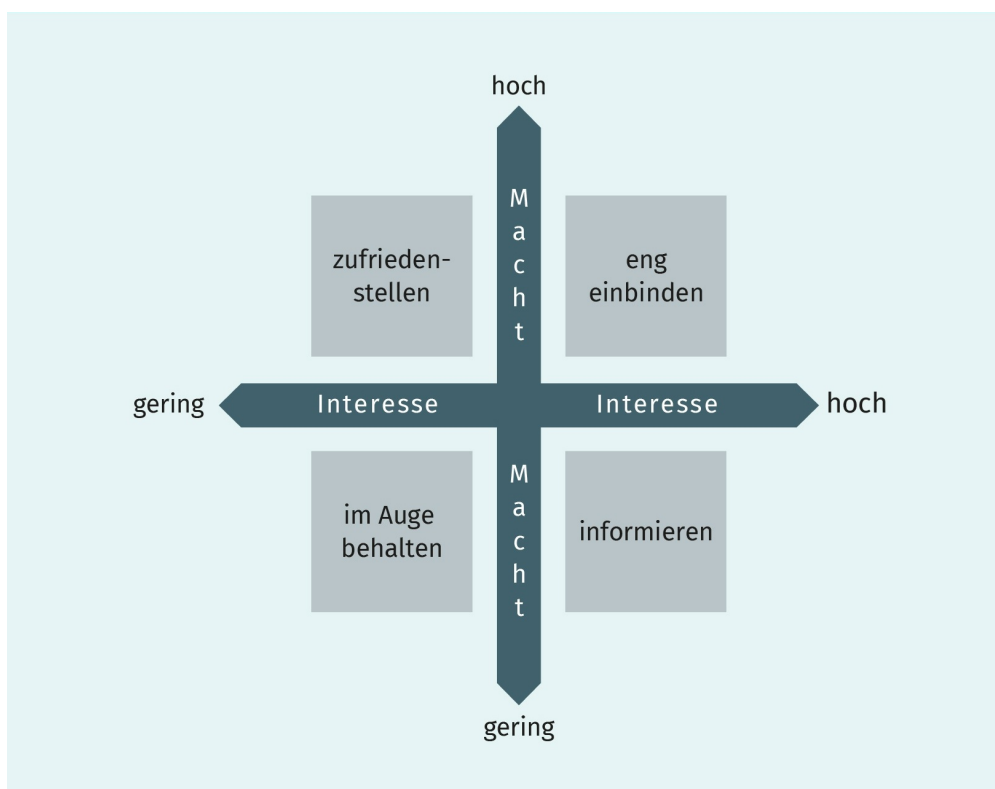
Bereits existierende Systeme sind eine weitere Anforderungsquelle. Wird ein System abgelöst, liefert das Altsystem wichtige funktionale und qualitative Anforderungen sowie Randbedingungen für das neue System. Darüber hinaus kann das System auch Quelle für Anforderungen sein, die zur Verbesserung des neuen Systems gegenüber dem alten System dienen. Gleichzeitig kann auf Basis des Altsystems durch Nutzungsprotokolle analysiert

werden, welche Funktionalität selten genutzt wurde und daher ein Kandidat für die Eliminierung ist. Häufig gibt auch die Produktanalyse von Mitbewerbern Aufschluss über funktionale und qualitative Anforderungen für das eigene System. Zudem führen die meisten Projekte keine neuen Systeme ein, sondern entwickeln bestehende Software weiter. Die Qualität und die Geschwindigkeit der Änderungen ist dabei maßgeblich von der Qualität der bestehenden Dokumentation abhängig.

Stakeholdermanagement

Aktives Stakeholdermanagement ist elementarer Bestandteil des Requirements Engineering. Jeder Stakeholder hat einen individuellen Hintergrund, Kenntnisstand und ein individuelles Problemverständnis. Dies führt dazu, dass auch jeder Stakeholder eine eigene Sichtweise und eigene Prioritäten hat. Zunächst ist es die Aufgabe des Requirements Engineer, die relevanten Stakeholder zu identifizieren und deren Interessen und Einflüsse auf das Projekt im Detail zu analysieren. Oft geschieht das in Zusammenarbeit mit der Projektleitung. Anhand dieser Erkenntnisse werden die Stakeholder in eine Stakeholder-Priorisierungsmatrix eingetragen (siehe Abbildung 4).

Abbildung 4: Stakeholder-Priorisierungsmatrix



Quelle: erstellt im Auftrag der IU, 2023.

Stakeholder mit geringem Interesse und geringer Macht sind nicht gänzlich zu vernachlässigen, erfordern jedoch den geringsten Aufwand, da diese nur passiv involviert werden müssen. Stakeholder, die Interesse an dem Projekt zeigen, müssen hingegen kontinuierlich

lich über den aktuellen Projektstand informiert werden (beispielsweise durch die Veröffentlichung von Anforderungen in einem Wiki). Die Bedürfnisse der Stakeholder, die zwar geringes Interesse am Projekt besitzen, jedoch großen Einfluss ausüben können, sind unbedingt zu befriedigen, sodass sie ihre Macht nicht gegen die Interessen des Projekts einsetzen. In die letzte Kategorie der Matrix werden die Stakeholder mit großem Einfluss und hohem Interesse an dem Projekt eingeordnet. Diese Key-Stakeholder sind eng in das Projekt zu involvieren, da von ihnen großer Mehrwert zu erwarten ist.

Ausgehend von dieser Priorisierung muss der Grad der Stakeholderinvolvierung im Detail geplant werden. Die benötigte Unterstützung, ggf. eine Projekttrolle mit zugehörigen Aufgaben und die vom Stakeholder benötigten Informationen sind festzulegen. Alle verfügbaren Informationen werden tabellarisch festgehalten (siehe Tabelle 1). Für jeden Stakeholder sind zusätzlich auch noch der Name, die Rolle im Unternehmen, der Grad der Macht/des Interesses sowie die Ziele einzutragen. Als Ergebnis dieser Phase entsteht eine detaillierte Dokumentation aller Informationen zu den relevanten Stakeholdern.

Tabelle 1: Stakeholder-Tabelle

Name	Rolle	Macht	Interesse	Unterstützung (ggf. Projektrolle)	Benötigte Informationen
Meier	Sachbearbeiter	gering	hoch	Quelle für GUI-Anforderungen	Verbesserungsvorschläge
Müller	Auftraggeber	hoch	hoch	Geld	Freigabe der Anforderungen

Quelle: erstellt im Auftrag der IU, 2023.

Stakeholdermanagement ist eine zentrale Aufgabe des Requirements Engineer und hat gravierende Auswirkungen auf den Erfolg eines Projekts. Vergessene oder fehlende Stakeholder etwa können dazu führen, dass Anforderungen spät oder gar nicht erkannt werden. Anforderungen, die zu spät im Projekt erkannt werden, verursachen hohe Kosten. Zentrale Aufgabe im Stakeholdermanagement ist zudem nicht nur die Erkennung aller relevanten Stakeholder, sondern auch die Herstellung und Aufrechterhaltung der Kooperationsbereitschaft dieser. Ursachen für unzureichende Kooperationsbereitschaft können beispielsweise ein Motivationsmangel, Unzufriedenheit mit dem Altsystem oder auch die Angst vor Veränderung sein. Als Best Practices im Umgang mit Stakeholdern haben sich die folgenden Punkte etabliert:

- Regelmäßige Statusbesprechungen: Stakeholder müssen über den aktuellen Stand auf dem Laufenden gehalten werden, damit sich diese einbezogen fühlen.
- Aus Projektbetroffenen durch Motivation Projektbeteiligte machen: Wenn Stakeholder sich durch das Projekt negativ beeinflusst fühlen, muss der Requirements Engineer das Projekt erläutern und die Stakeholder motivieren, aktiv mitzuwirken.
- Aufmerksamkeit: Anliegen von Stakeholdern müssen stets ernst genommen und durch entsprechende Reaktionen wertgeschätzt werden.

- Positiv eingestellte Stakeholder halten, negativ eingestellte Stakeholder überzeugen: Positiv eingestellte Stakeholder sind durch aktive Einbeziehung und regelmäßige Informationsweitergabe zu bestärken, negativ eingestellte Stakeholder sind durch Erläuterung und Motivation zu überzeugen.
- Festlegen von Aufgaben, Verantwortungsbereichen und Weisungsbefugnissen zur Vermeidung von Missverständnissen: Eine klare Zuteilung von Aufgaben und Verantwortungen fördern die Wertschätzung und vermeiden Gerangel um Kompetenz.

Beispiel

Der angehende Requirements Engineer Herr Klein ist 35 Jahre alt und verheiratet. Seine Ausbildung hat er als Versicherungskaufmann bei der Pfefferminzia Versicherungen AG abgeschlossen und arbeitet seitdem in selbigem Unternehmen. Wegen seiner ausgeprägten Affinität zur IT wurde er vor drei Monaten zum IT-Verbindungsmann der Leistungsabteilung der Pfefferminzia AG befördert. Im Rahmen dieser Stelle ist es seine Aufgabe, die Kommunikation der Leistungsabteilung zur IT zu koordinieren. Da die Pfefferminzia erkannt hat, dass das Requirements Engineering eine erfolgskritische Phase im Softwareprozess ist, macht Herr Klein derzeit eine Ausbildung zum Requirements Engineer.

Herr Klein versucht zunächst die Projektvision, also das Ziel des Projekts, schriftlich auf ungefähr einer halben DIN-A4-Seite als Fließtext festzuhalten. Das Formulieren der Projektvision dient auch der Überprüfung des eigenen Projektverständnisses. Herr Klein stellt fest, dass er Schwierigkeiten hat, die Vision präzise zu formulieren. Dies ist ein Indikator für mangelndes Verständnis des Projekts, also nimmt er umgehend Kontakt mit den Projektverantwortlichen auf, um seine Wissenslücken zu schließen und die Projektvision vollständig zu erarbeiten.

Mit der klar formulierten Projektvision beginnt Herr Klein nun zu planen, welche konkreten Ermittlungsziele erreicht werden sollen. Die dokumentierten Ziele stimmt er mit den Projektverantwortlichen ab und bestimmt zu jedem Ziel, welche Anforderungsquellen zur Zielerreichung beitragen.

Kritisch bei der Ermittlung von Anforderungen ist die Verfügbarkeit von Stakeholdern, daher hat Herr Klein zu jedem identifizierten Stakeholder (Primärstakeholder) auch eine Person festgelegt, die als Ausweichmöglichkeit dienen kann, sofern der Primärstakeholder keine Zeit zur Verfügung stellt. Umgehend beginnt Herr Klein, Termine mit den Primärstakeholdern zu vereinbaren. Dazu bietet er den Stakeholdern mithilfe eines Tools zur Terminfindung (beispielsweise Doodle) alternative Termine an. Die Stakeholder wählen selbstständig, an welchem Termin sie die Ermittlung durchführen wollen.

Herr Klein merkt, dass die Stakeholder Herr Müller und Frau Meier keinen Termin ausgewählt haben. Für Herrn Müller hat Herr Klein Herrn Schmidt als Alternative identifiziert und lädt diesen zur Terminumfrage ein. Für Frau Meier ist es Herrn Klein nicht gelungen, einen Ersatzstakeholder zu identifizieren, denn sie nimmt als Abteilungsleiterin der Leistungsabteilung eine zentrale Rolle für die Anforderungsermittlung ein. Also sucht Herr Klein zunächst den persönlichen Kontakt mit Frau Meier und merkt zügig, dass diese seine Kontaktaufnahme und die Terminvereinbarung abblockt. Frau Meier ist nicht kooperationsbereit, und die Aufgabe von Herrn Klein ist es nun herauszufinden, warum Frau Meier

nicht kooperationsbereit ist. Nach einigen Gesprächen mit Kollegen stellt Herr Klein fest, dass Frau Meier Sorge hat, durch die neue Software weniger Einfluss auf das Alltagsgeschäft zu haben, da sie vermutet, dass mit der Software viele Entscheidungen, die sie aktuell trifft, automatisch geschehen werden. Durch Zufall trifft Herr Klein Frau Meier in der Cafeteria der Pfefferminzia und es gelingt ihm, ihr in einem unverfänglichen Gespräch die Projektvision zu erläutern. Dabei macht Herr Klein deutlich, dass Frau Meier die Möglichkeit hat, das System aktiv mitzugestalten. Er nimmt ihr so die Furcht und weckt ihre Neugier, sodass sie daraufhin einen Termin zur Ermittlung ihrer Anforderungen vereinbaren.

In dem Termin erläutert Herr Klein die konkreten Ziele der Befragung und zeigt ihr, dass das System neben einer Erneuerung der technologischen Basis auf den aktuellen Stand auch eine Funktionalität bietet, in der definiert werden kann, welche Entscheidungen regelbasiert und welche Entscheidungen durch einen Vorgesetzten getroffen werden müssen. Außerdem macht er ihr deutlich, dass die aktive Mitarbeit an der Gestaltung dieser Regelkomponente in erster Linie eine Erleichterung ihrer Arbeit bedeuten wird, da sie nicht mehr alles entscheiden muss, sondern nur noch die in ihren Augen relevanten Entscheidungen trifft. Dies überzeugt Frau Meier, dass das System auch ihr zugute kommt und entscheidet sich, aktiv an der Gestaltung teilzunehmen.

Der zuvor beschriebene Sachverhalt stellt exemplarisch dar, wie Herr Klein die Ursachen für mangelnde Kooperationsbereitschaft identifiziert und analysiert hat, um daraufhin mit einer geeigneten Maßnahme die Kooperationsbereitschaft wieder herzustellen. Hätte Herr Klein auf die Teilnahme von Frau Meier verzichtet, wäre er Gefahr gelaufen, dass wichtige Anforderungen (nämlich welche Entscheidungen regelbasiert getroffen werden sollen) gar nicht oder inkorrekt ermittelt worden wären.

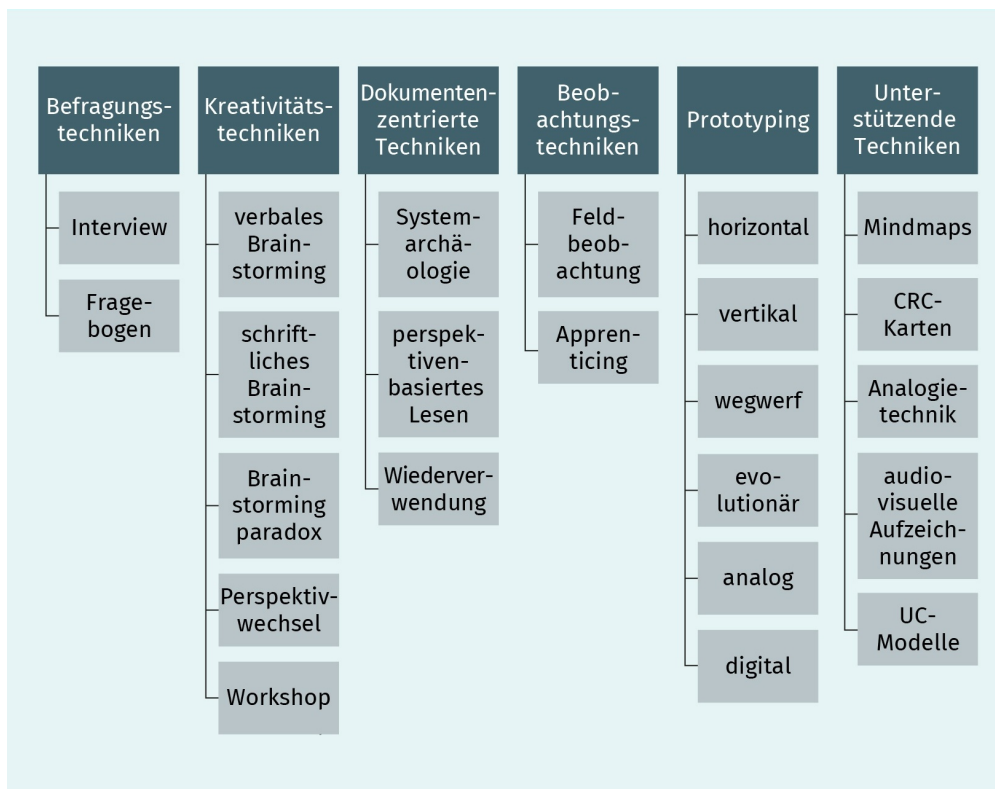
2.3 Auswählen der geeigneten Ermittlungstechniken

Für die Ermittlung von Anforderungen gibt es verschiedene Techniken, welche grundsätzlich immer von der spezifischen Situation und der Art der zu ermittelnden Anforderungen abhängig sind. Im Folgenden werden die jeweiligen Technikkategorien und die Einflussfaktoren zur Wahl einer geeigneten Kategorie diskutiert.

Überblick über typische Ermittlungstechniken

Abbildung 5 zeigt eine Auswahl verschiedener Techniken zur Anforderungserhebung, die im Requirements Engineering angewendet werden können. Sie sind in Kategorien eingeteilt und werden in der Regel nicht isoliert, sondern in Kombination eingesetzt.

Abbildung 5: Überblick über Techniken der Anforderungsermittlung



Quelle: erstellt im Auftrag der IU, 2023.

Befragungstechniken setzen voraus, dass die befragten Stakeholder auf der einen Seite fähig sind, Anforderungen explizit zu äußern, und auf der anderen Seite über die notwendige Zeit zur Befragung und den Willen, diese durchzuführen, verfügen. Mit Befragungstechniken lassen sich sowohl innovative als auch explizit geforderte Anforderungen sowie grundlegende Anforderungen ermitteln. Der Detailgrad der Anforderungen kann sehr hoch sein. Beispiele sind Interviews und Fragebögen.

Kreativitätstechniken eignen sich besonders zur Gewinnung innovativer Anforderungen und einer gemeinsamen Systemvision durch kreative und kooperative Zusammenarbeit. Der Aufwand für die Techniken ist vergleichsweise hoch, und mittels ihr werden weniger detaillierte Anforderungen ermittelt. Beispiele sind Brainstorming und Perspektivwechsel.

Mit dokumentenzentrierten Techniken lassen sich insbesondere grundlegende Anforderungen ermitteln, da bei Anwendung dieser Techniken bereits vorhandene Systeme oder Dokumentationen von existierenden Systemen genutzt werden, um auf deren Basis Anforderungen zu gewinnen. Diese Anforderungen sind also bereits in vorherigen Systemen umgesetzt worden und werden wiederverwendet. Beispiele für konkrete Techniken sind die Systemarchäologie und perspektivenbasiertes Lesen.

Die Beobachtungstechniken eignen sich besonders für die Gewinnung grundlegender Funktionalität, da der Requirements Engineer Arbeitsabläufe beobachtet und dokumentiert, um daraus Systemfunktionalität abzuleiten. Beispiele für Beobachtungstechniken sind die Feldbeobachtung und das Apprenticing.

Mit Prototyping können innovative Anforderungen gewonnen werden, indem initiale Systemversionen (sogenannte Prototypen) erstellt werden und so ein tieferes Problemverständnis und Wissen über potenzielle Lösungen erlangt wird. Prototypen können unterschiedlich beschaffen sein (z. B. analoge Handskizzen und digitale Prototypen), die Beschaffenheit ist zudem abhängig vom Projektfortschritt. Je weiter ein Projekt fortgeschritten ist, desto eher werden digitale Prototypen erzeugt.

Einflussfaktoren auf die Wahl der Ermittlungstechniken

Um möglichst vollständige Anforderungen zu erhalten, werden die Techniken in der Praxis häufig kombiniert (z. B. eine Kreativitätstechnik, um Fragen für ein Interview zu erarbeiten). Es müssen verschiedene Faktoren, die Einfluss auf das Projekt und die Anforderungen haben, untersucht werden, um die geeigneten Techniken auswählen zu können.

Menschliche Einflussfaktoren sind soziale, gruppendynamische und kognitive Fähigkeiten der Stakeholder. Darüber hinaus ist zu beachten, ob die Anforderungen explizit geäußert oder als selbstverständlich vorausgesetzt werden. Unbewusst vorliegendes Wissen wird beispielsweise besser durch eine Kreativitätstechnik als durch eine dokumentenzentrierte Technik ermittelt. Grundlegende, als selbstverständlich vorausgesetzte Anforderungen lassen sich besser durch Beobachtungstechniken als durch Befragungstechniken erheben.

Organisatorische Einflüsse haben Auswirkungen auf die Wahl der geeigneten Ermittlungstechniken. Beispiele für organisatorische Einflüsse können die Vertragsart oder die Art des Entwicklungsprojekts sein. In einem knapp budgetierten Festpreisprojekt zur Migration eines Informationssystems werden weniger Kreativitätstechniken notwendig sein als Techniken, mit denen Basisfunktionalität und explizit geforderte Anforderungen ermittelt werden können, also beispielsweise Befragungstechniken. Auch die räumliche und zeitliche Verfügbarkeit von Stakeholdern hat wesentlichen Einfluss auf die Wahl der Ermittlungstechnik. Sind Stakeholder zeitlich nur begrenzt verfügbar, ist eine ausführliche Befragung weniger geeignet.

Fachlich/inhaltliche Einflüsse sind auch ein Faktor, den es bei der Wahl geeigneter Ermittlungstechniken zu berücksichtigen gilt. Die Komplexität eines Projekts kann beispielsweise erfordern, dass eine gut strukturierte dokumentenzentrierte Technik zum Einsatz kommt.

2.4 Anforderungen unter Einsatz der Techniken ermitteln

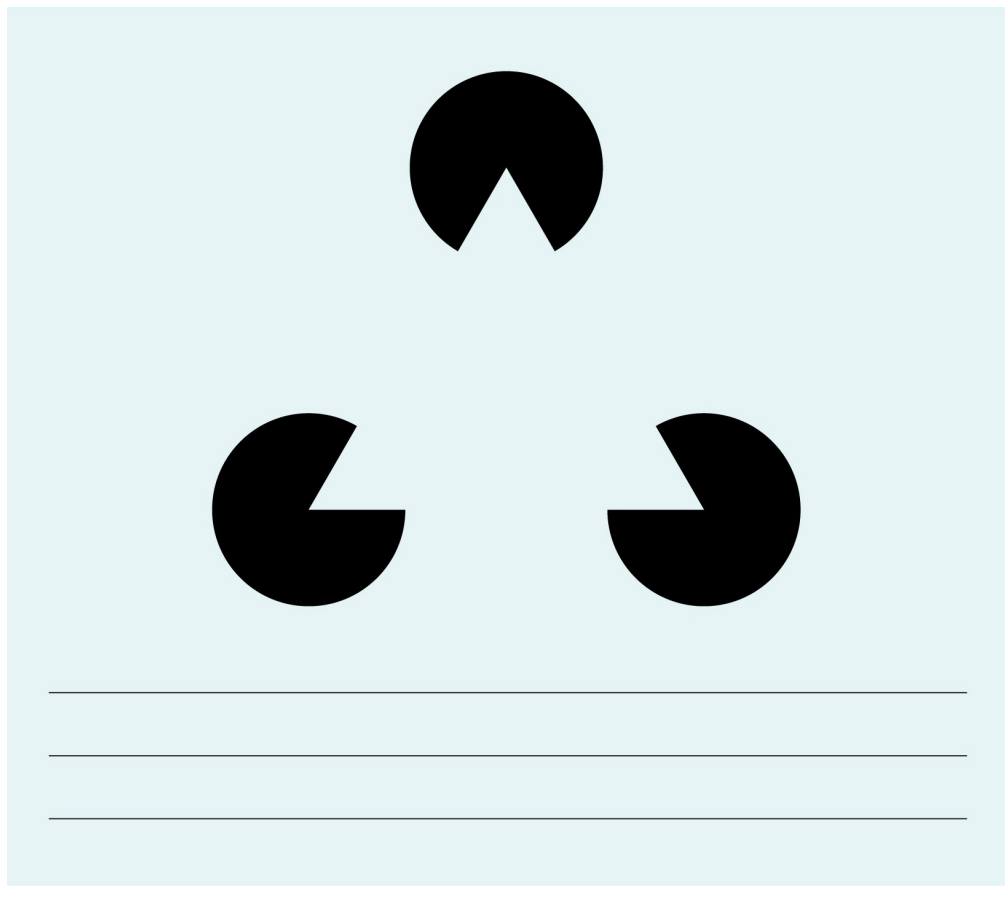
Fragen wir uns nun, was Herr Klein, der als angehender Requirements Engineer dafür verantwortlich ist, die Anforderungen möglichst vollständig zu erheben, durch eine Kombination geeigneter Techniken realisieren kann. Nachdem die Projektvision formuliert, dokumentiert und abgestimmt wurde, hat Herr Klein die konkreten Ziele der Ermittlung festgelegt. Zu jedem Ziel hat er durch eine Analyse des Systemkontextes die relevanten Anforderungsquellen identifiziert und dokumentiert. Direkt im Anschluss beginnt er, mit Stakeholdern Termine zur Ermittlung zu vereinbaren und wählt eine geeignete Kombination aus Ermittlungstechniken (z. B. Interview und Systemarchäologie), die es ermöglicht, die konkreten Ziele der Ermittlung je Anforderungsquelle zu erreichen. Die Projektvision und die konkreten Ziele einzelner Ermittlungen können den jeweiligen Stakeholdern vorab mitgeteilt werden, müssen jedoch in der Ermittlung selbst unbedingt erläutert werden, um die Möglichkeit zu haben, eventuelle Verständnisfragen vorab zu klären.

Während der Durchführung der Ermittlung muss Herr Klein je nach Ermittlungstechnik ggf. steuernd eingreifen und durch Moderation sicherstellen, dass der Fokus der Anforderungserhebung nicht verloren geht. Gleichzeitig muss er ständig die eigene Wahrnehmung und Aussagen der Stakeholder hinterfragen, um zu überprüfen, ob er die Anforderung richtig verstanden hat.

Menschliche Wahrnehmung

Die menschliche Wahrnehmung spielt eine gewichtige Rolle in der Anforderungsermittlung. Die Verarbeitung von Informationen ist von der individuellen Erfahrung und kulturellen Prägung abhängig. Dies bedeutet, dass bei verschiedenen Menschen die gleichen Informationen unterschiedlich wahrgenommen und interpretiert werden. Durch die folgenden zwei Beispiele wird dieser Sachverhalt illustriert. Betrachten Sie Abbildung 6 und beschreiben Sie bitte, was genau schwarz und was hellgrün gedruckt wurde:

Abbildung 6: Experiment zur menschlichen Wahrnehmung



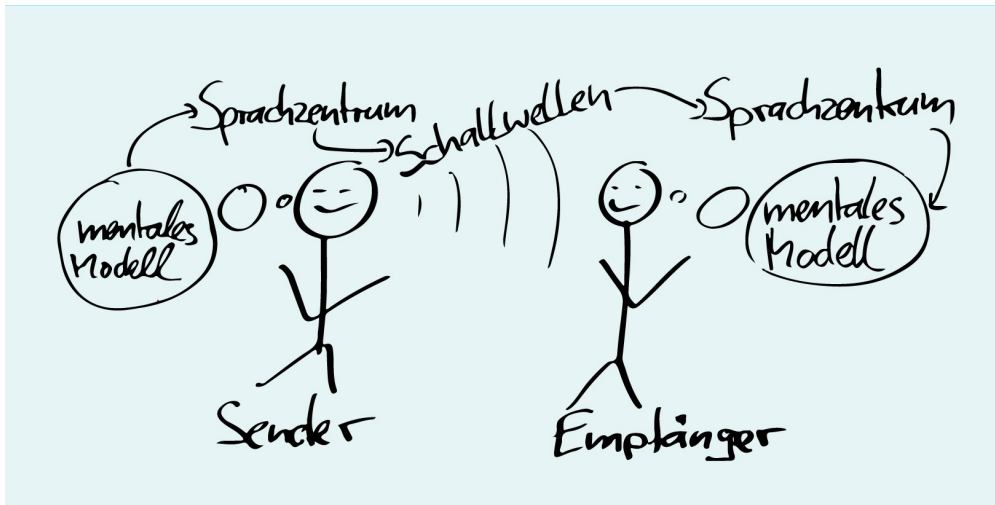
Quelle: erstellt im Auftrag der IU, 2023.

Viele sehen ein hellgrünes Dreieck und nicht, wie gedruckt, drei Kreise, in denen je ein Stück fehlt. Ein weiteres Beispiel liefert der folgende Text. Lesen Sie diesen laut vor:

Sie sind in der Lage dieses Text laut vorzulesen, so dass er Sinn macht, obwohl Sie nicht verstehen, was gedruckt ist. Dieses Beispiel verdeutlicht, was die menschliche Wahrnehmung und Sprechergewohnheiten auf Basis Ihrer Erfahrung zu Sätzen im Text sind.

Betrachten wir nicht nur die Informationsaufnahme, sondern auch die Informationswiedergabe, zeigt sich das gleiche Bild. Durch das Gedächtnis hat jeder Mensch ein ganz persönliches inneres Modell von der realen Welt. Dieses Modell ist beeinflusst von Wissen, Erfahrung und kultureller Prägung des Einzelnen. Mentale Bilder werden in der Realität häufig stark vereinfacht vom Sender wiedergegeben und vom Empfänger auf sein individuelles mentales Modell abgebildet. Der Grund für Missverständnisse liegt also in der Interpretation des gesprochenen Wortes auf Basis der Abbildung auf das eigene mentale Modell des Empfängers. Je weniger Informationen der Sender dabei kommuniziert, also je stärker er sein mentales Bild vereinfacht, desto größer ist die Wahrscheinlichkeit von Missverständnissen. Abbildung 7 zeigt den Zusammenhang zwischen der Kommunikation von Sender und Empfänger.

Abbildung 7: Natürlich-sprachliche Kommunikation zwischen Sender und Empfänger



Quelle: erstellt im Auftrag der IU, 2023.

Im Requirements Engineering ist beispielsweise eine besonders gründliche Begriffsbildung erforderlich. Oft verwendete Begriffe, die für jeden Beteiligten persönlich völlig klar sind, stellen sich oft erst nach langer Diskussion als unterschiedlich heraus. Für weitere Informationen zur menschlichen Wahrnehmung empfiehlt sich das Studium von Rupp 2014, S. 146–174.



ZUSAMMENFASSUNG

Um Anforderungen systematisch zu ermitteln, muss der Requirements Engineer zunächst die vorläufige Systemgrenze und die Grenze zwischen der relevanten und irrelevanten Umgebung analysieren. Beide Grenzen bewegen sich in einem Korridor. Mit zunehmendem Wissen und zunehmender Stabilität der Anforderungen wird der Korridor schmaler, bis die Grenzen schließlich fix sind. Relevante Stakeholder, Dokumente und andere Systeme werden mit dem Wissen, was Teil des Systems ist, als Quellen für Anforderungen identifiziert. In der Regel müssen mehrere Quellen untersucht werden, um eine hinreichend vollständige Menge von Anforderungen zu ermitteln.

Für die Ermittlung der relevanten Anforderungen stehen eine Reihe von Techniken zur Verfügung, die sich in die Kategorien Befragungstechniken, Kreativitätstechniken, dokumentenzentrierte Techniken, Beobachtungstechniken und Prototyping klassifizieren lassen. Die Auswahl einer geeigneten Kombination von Ermittlungstechniken unterliegt dem Einfluss von menschlichen, organisatorischen und fachlichen sowie inhaltli-

chen Randbedingungen. Für die Ermittlung der Anforderungen ist es wichtig, dass alle Quellen identifiziert werden und deren Verfügbarkeit rechtzeitig geplant wird.

Außerdem ist die menschliche Wahrnehmung zu beachten. Sowohl in der Rolle des Senders als auch in der des Empfängers muss sich der Requirements Engineer darüber bewusst sein, dass das mentale Modell jedes Individuums Einfluss auf die Interpretation des Gesagten, Erlebten oder Gelesenen hat und durch kontinuierliches Feedback Anforderungen an die Stakeholder zurückspiegelt, mit dem Ziel, zu prüfen, ob das Verstandene das Geäußerte trifft.

LEKTION 3

AUSGEWÄHLTE ERMITTLUNGSTECHNIKEN

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- welche konkreten Techniken zu welcher Technikkategorie gehören.
- wie die vorgestellten Techniken eingesetzt werden.
- welche Einsatzszenarien sich für welche Technik eignen.
- welche Stärken und Schwächen die vorgestellten Techniken haben.
- welcher Aufwand mit dem Einsatz jeder Technik verbunden ist.

3. AUSGEWÄHLTE ERMITTLUNGSTECHNIKEN

Einführung

Menschliche, organisatorische und fachliche bzw. inhaltliche Faktoren haben Einfluss auf die Wahl der geeigneten Ermittlungstechnik. Tabelle 2 gibt einen Überblick darüber, welche konkreten Einflussfaktoren der drei Kategorien welchen Einfluss auf die Nutzung einer Technik haben. Der Requirements Engineer kann diese Tabelle als Anhaltspunkt nutzen, um konkrete Techniken auszuwählen, muss jedoch in jedem Fall die spezifische Projektsituation berücksichtigen und die Auswahl der Techniken auf den Einsatz im jeweiligen Projekt abstimmen. In den folgenden Lernzyklen werden ausgewählte Techniken der einzelnen Kategorien im Detail erläutert.

Tabelle 2: Eignung konkreter Techniken zum praktischen Einsatz (1)

	verbales Brain- storming	schriftli- ches Brain- storming mit User Stories	Work- shop	Wechsel der Perspektive
Menschliche Einflussfaktoren				
geringe Motivation der Stakeholder (aktiv mitzuwirken)	–	–	–	–
schlechte kommunikative Fähigkeiten	–	0	–	–
geringes Abstraktionsvermögen	–	–	0	–
viele verschiedene Meinungen	+	+	++	+
Machtgefälle zwischen beteiligten Parteien	–	+	–	–
problematische Gruppendynamik	–	+	–	+
Organisatorische Einflussfaktoren				
Entwicklung für den komplexen Markt	++	++	++	+
fixiertes, knappes Projektbudget	++	++	+	+
hohe örtliche Verteilung der Stakeholder	–	0 ¹	–	–

schlechte zeitliche Verfügbarkeit der Stakeholder	+	+	-	-
hohe Anzahl der Stakeholder	+	+	-	+

Fachliche/inhaltliche Einflussfaktoren

hohe Kritikalität des Sachverhalts	0	+ ²	+	+
großer Systemumfang	0	+	0	0
keine Erfahrung im Fachgebiet	0	0	0	0
grobe Anforderungen gesucht	++	++	+	+
detaillierte Anforderungen gesucht	+	0 ³	+	+
nicht funktionale Anforderungen	0	-	0	0
Komplexität des Sachverhalts	0	0	0	0

Legende:

- nicht empfohlen

0 kein Einfluss → ist anwendbar

+ empfohlen

++ sehr empfohlen

Anmerkungen:

¹ mithilfe digitaler Kommunikationstechnologie

² um einen ersten Eindruck über die Anforderungen zu bekommen, unbedingt mit weiteren Techniken kombinieren

³ mehrere Iterationen durchführen

Quelle: erstellt im Auftrag der IU, 2023.

Tabelle 3: Eignung konkreter Techniken zum praktischen Einsatz (2)

	Feldbeobachtung	Apprenticing	Fragebogen	Interview
Menschliche Einflussfaktoren				
geringe Motivation der Stakeholder (aktiv mitzuwirken)	+	-	0	+
schlechte kommunikative Fähigkeiten	++	++	-	+
geringes Abstraktionsvermögen	++	++	0	+
viele verschiedene Meinungen	++	++	+	0
Machtgefälle zwischen beteiligten Parteien	0	0	0	0

problematische Gruppendynamik	0	0	0	0
-------------------------------	---	---	---	---

Organisatorische Einflussfaktoren

Entwicklung für den komplexen Markt	–	–	++	0
fixiertes, knappes Projektbudget	+	–	–	+
hohe örtliche Verteilung der Stakeholder	0	0	++	0
schlechte zeitliche Verfügbarkeit der Stakeholder	+	–	+	++
hohe Anzahl der Stakeholder	0	–	++	0

Fachliche/inhaltliche Einflussfaktoren

hohe Kritikalität des Sachverhalts	++	–	+	+
großer Systemumfang	+	–	–	+
keine Erfahrung im Fachgebiet	–	+	–	–
grobe Anforderungen gesucht	+	0	+	++
detaillierte Anforderungen gesucht	+	++	–	+
nicht funktionale Anforderungen	0	+	–	+
Komplexität des Sachverhalts	–	–	–	+

Legende:

– nicht empfohlen

0 kein Einfluss → ist anwendbar

+ empfohlen

++ sehr empfohlen

Anmerkungen:

¹ mithilfe digitaler Kommunikationstechnologie

² um einen ersten Eindruck über die Anforderungen zu bekommen, unbedingt mit weiteren Techniken kombinieren

³ mehrere Iterationen durchführen

Quelle: erstellt im Auftrag der IU, 2023.

3.1 Kreativitätstechniken

Kreativitätstechniken eignen sich zur Entwicklung innovativer Anforderungen oder einer ersten Vision des zu entwickelnden Systems. Jedoch sind Kreativitätstechniken weniger geeignet um detaillierte Anforderungen an das Systemverhalten zu ermitteln. Im Folgenden wird mit dem Sechs-Hut-Denken eine konkrete Variante des Perspektivwechsels erläutert. Außerdem werden zwei Arten des Brainstormings – das klassische verbale und das schriftliche Brainstorming – sowie das Prinzip von Workshops erklärt.

Perspektivwechsel

Der Perspektivwechsel ist eine Kreativitätstechnik, die dazu dient, ein Problem aus verschiedenen Blickwinkeln zu betrachten. Die beteiligten Personen denken und argumentieren dabei aus der Perspektive einer ganz bestimmten Rolle, beispielsweise des Nutzers, des Testers oder eines Kunden. Auf diese Weise können bei der Ermittlung von Anforderungen Perspektiven von Stakeholdern berücksichtigt werden, deren direkte Einbeziehung nicht möglich ist. Eine Variante ist die Methode des Sechs-Hut-Denkens von Edward de Bono (2016, S. 1–16), wobei jeder Hut einen Blickwinkel auf ein Problem darstellt. Den Teilnehmern bei der Anwendung dieser Methode werden symbolisch unterschiedlich farbige Hüte aufgesetzt, die die Perspektive, aus der der Stakeholder das Problem betrachten wird, identifizieren:

- Der weiße Hut steht für Objektivität und Neutralität. Der Stakeholder achtet vor allem auf Zahlen und Fakten.
- Der rote Hut steht für eine subjektive Meinung und persönliche Empfindungen. Der Stakeholder äußert seine Gefühle, Ängste und Hoffnungen.
- Der schwarze Hut lässt den Stakeholder objektiv, aber negativ argumentieren.
- Der gelbe Hut lässt einen Stakeholder auch objektiv, jedoch eher positiv argumentieren.
- Der grüne Hut stellt die Kreativität in den Vordergrund. Der Stakeholder ist darauf bedacht, neue Ideen einzubringen.
- Der blaue Hut steht für Kontrolle und Organisation des gesamten Denkprozesses. Die Person mit dem blauen Hut moderiert und koordiniert den Prozess der Ideenfindung. Auch diese Perspektive wird von unterschiedlichen Stakeholdern eingenommen und nicht nur vom Requirements Engineer, der typischerweise moderiert.

Stakeholder, die besonders überzeugt von ihrer Sichtweise sind, werden durch einen Perspektivwechsel animiert, neue Sichten anzunehmen. Daher eignet sich die Methode auch besonders um eingeengte Sichtweisen und Formulierungen zu lösen. Der Perspektivwechsel kann auch eingesetzt werden, um existierende Lösungsansätze von verschiedenen Seiten zu betrachten. Jedoch ist die Anforderungssammlung mit hohem Detaillierungsgrad sehr aufwendig, und für introvertierte und konservative Stakeholder kann die Methode abgehoben wirken. Der Requirements Engineer muss bei der Einführung sehr behutsam vorgehen, um nicht Gefahr zu laufen direkt in eine „esoterische Ecke“ gesteckt zu werden, denn dadurch kann die Kooperationsbereitschaft der Stakeholder vermindert werden.

Tabelle 4: Steckbrief „Perspektivwechsel“

Einsatzszenarien	• wenn ein Gesamtüberblick über das Projekt gewonnen werden soll • wenn festgefahrene Meinungen aufgebrochen werden sollen
Stärken	• umfassende Betrachtung aus den Perspektiven
Schwächen	• für begrenzte Teilnehmerzahl anwendbar • problematisch bei konservativen und introvertierten Stakeholdern

Quelle: erstellt im Auftrag der IU, 2023.

Verbales Brainstorming

Das verbale Brainstorming wird mit einer Gruppe von fünf bis zehn Personen durchgeführt. Diese Gruppe sammelt in einer vorgegebenen Zeit und zu einem vorgegebenen Thema Ideen. Die Stakeholder äußern ihre Ideen und nutzen die Ideen der anderen Stakeholder als Inspiration für weitere Ideen. Daher ist Brainstorming besonders erfolgreich, wenn die teilnehmenden Stakeholder möglichst heterogen ausgewählt wurden und durch Einflüsse aus anderen Domänen angeregt werden, diese in die eigene Ideenfindung einfließen zu lassen. Für mehr Informationen zu sogenannten multifunktionalen Teams siehe z. B. Marczak/Kwan/Damian (2009) und Nambisan/Wilemon (2000).

Der Requirements Engineer notiert alle Ideen, ohne diese zu bewerten, und achtet darauf, dass alle Stakeholder die Ideen der anderen zulassen, egal wie absurd diese auch sein mögen. Während der Sammlung darf keinerlei Bewertung oder Kritik geäußert werden, da dies den kreativen Prozess hemmt und Stakeholder unter Umständen demotiviert werden. Nach der Sammlung werden die Ideen analysiert und aggregiert, doppelte Ideen werden eliminiert, und es besteht die Möglichkeit, die Ideen zu klassifizieren. Durch Einsatz moderner Informations- und Telekommunikationssoftware ist es auch möglich, Brainstorming mit räumlich verteilten Stakeholdern durchzuführen. Der Vorteil von Brainstorming ist, dass in relativ kurzer Zeit viele Ideen gesammelt und innovative Anforderungen ermittelt werden können. Demgegenüber steht der Nachteil, dass bei schwieriger Gruppendynamik nur schlechte oder wenige Ergebnisse zu erwarten sind. Gleiches gilt auch, wenn unterschiedlich dominante Stakeholder in der Gruppe sind. Darüber hinaus ist es für den Requirements Engineer herausfordernd, alle Informationen zugleich zu notieren.

Tabelle 5: Steckbrief „Verbales Brainstorming“

Einsatzszenarien	• wenn innovative Anforderungen gewonnen werden sollen • wenn Stakeholder heterogen zusammengesetzt sind • wenn eine Vision entstehen soll
Stärken	• Ermittlung innovativer Anforderungen möglich • Stakeholder müssen nicht vor Ort sein
Schwächen	• für maximal 8–10 Teilnehmer anwendbar • problematisch bei gruppendynamischen Effekten, dominierenden Stakeholdern oder Hierarchiekonflikten

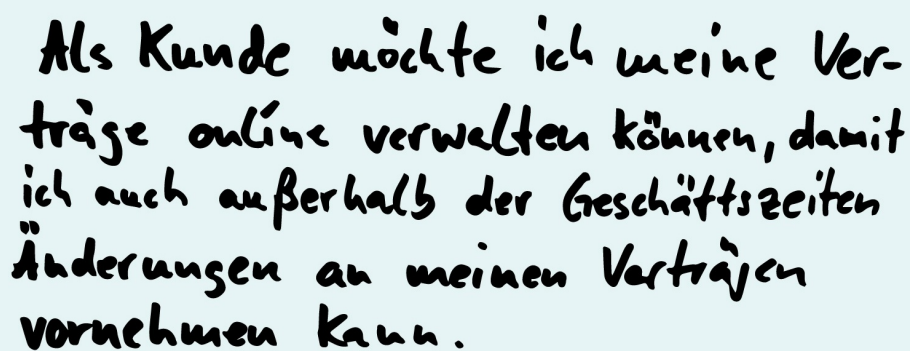
Quelle: erstellt im Auftrag der IU, 2023.

Wenn die Gruppendynamik, dominierende Stakeholder oder Hierarchiekonflikte in der Stakeholdergruppe die Ergebnisqualität beeinträchtigen, kann die Variante des schriftlichen Brainstormings angewendet werden. Stakeholder sammeln in Ruhe ihre Ideen und notieren diese. Die Grenze zum verbalen Brainstorming kann verschwimmen, wenn zur Sammlung elektronische Kollaborationswerkzeuge eingesetzt werden. Die Stakeholder notieren ihre Ideen z. B. auf einem Tablet, jede Idee wird an einen Server gesendet und unmittelbar für alle Stakeholder angezeigt (z. B. über einen Beamer). Werden die Ideen digital gesammelt, bedeutet dies eine Entlastung für den Requirements Engineer, der die Ideen nicht notieren muss. Dadurch besteht keine Gefahr, bei gleichzeitiger Äußerung Ideen zu vergessen und durch Nachfragen den Kreativitätsprozess zu hemmen. Die Ideensammlung kann jedoch auch anonym durchgeführt werden, indem Stakeholder ihre Ideen nur sammeln und notieren, jedoch nicht laut äußern. Die Anonymität hilft, negative Gruppendynamik und Hierarchiekonflikte zu verhindern. Nachteil kann sein, dass die eigene Kreativität nicht durch geäußerte Ideen der anderen Stakeholder angeregt wird. Es gibt verschiedene Varianten des schriftlichen Brainstormings. An dieser Stelle wird nur das Konzept der User Stories als Kreativitätsmethode eingeführt, eine weitere Variante, die 6-3-5-Methode (Backerra/Malorny/Schwarz 2019, S. 79–85), ist in der angegebenen weiterführenden Literatur zu finden.

Schriftliches Brainstorming mit User Stories

User Stories sind funktionale Anforderungen, die aus Sicht eines Benutzers formuliert werden. Eine User Story wird immer nach einem bestimmten Schema formuliert: Als <<Benutzerrolle>>, möchte ich <<ein bestimmtes Ziel erreichen (funktionale Anforderung)>>, um <<Begründung, warum das Ziel erreicht werden soll>>. So könnte eine User Story also beispielsweise lauten: „Als Kunde möchte ich meine Verträge online verwalten können, damit ich auch außerhalb der Geschäftszeiten Änderungen an meinen Verträgen vornehmen kann.“

Abbildung 8: Beispiel für eine User Story



Als Kunde möchte ich meine Verträge online verwalten können, damit ich auch außerhalb der Geschäftszeiten Änderungen an meinen Verträgen vornehmen kann.

Quelle: erstellt im Auftrag der IU, 2023.

In der Praxis hat es sich bewährt, User Stories mit einem dicken Stift auf eine Karteikarte zu schreiben (vgl. Abbildung 8). Durch den begrenzten Platz werden Stakeholder angeregt, präzise zu formulieren, und im Anschluss lassen sich Karten leicht sortieren und gruppieren sowie doppelte Ideen eliminieren. Um User Stories als Kreativitätsmethode einzusetzen, muss der Stakeholder zunächst das Konzept der User Stories und deren Schema kurz erläutern, dann erhalten die Stakeholder Karteikarten und werden aufgefordert, Ideen für funktionale Anforderungen aus einer bestimmten Perspektive zu sammeln. Erfahrungsgemäß sollte man die Stakeholder bei der ersten Ideensammlung aus beliebigen Perspektiven und ohne Begründung sammeln lassen, um den Kreativitätsprozess nicht zu behindern.

Die Stakeholder notieren ihre funktionalen Anforderungen auf den Karteikarten und heften diese nach Beendigung der Sammlungsphase an eine Pinnwand. Wenn die Sammlung nicht anonym geschieht, empfiehlt es sich, die Stakeholder ihre jeweiligen Anforderungen vorstellen zu lassen und an dieser Stelle auch eine fachliche Begründung für die Anforderung zu verlangen. Wird eine plausible Begründung geliefert, notiert der Stakeholder diese auf der Karteikarte, fällt die Formulierung einer Begründung schwer oder wird keine Begründung geliefert, muss diese noch einmal überarbeitet werden. Gegebenenfalls ist die Anforderung dann nicht so wichtig und kann zurückgestellt oder eliminiert werden. Falls die Sammlung anonym durchgeführt wird, muss nach der Sammlungsphase Zeit zum Notieren der Begründung gegeben werden. Dies hilft den Stakeholdern auch, die formulierten Anforderungen nochmals auf den Prüfstand zu stellen.

Nachdem die Anforderungen gesammelt wurden, müssen doppelte Anforderungen aussortiert werden. Darüber hinaus besteht die Möglichkeit, die Anforderungen nach Themen (z. B. Benutzerverwaltung) zu gruppieren. Dieses Vorgehen wird so lange wiederholt, bis die Stakeholder keine neuen Ideen mehr haben. Um mehr Ideen zu produzieren, kann die Methode Perspektivenwechsel mit der User-Story-Technik kombiniert werden. Das ist insbesondere dann sinnvoll, wenn wichtige Stakeholdergruppen nicht direkt verfügbar sind.

Tabelle 6: Steckbrief „Schriftliches Brainstorming mit User Stories“

Einsatzszenarien	• wenn innovative Anforderungen gewonnen werden sollen • wenn Stakeholder heterogen zusammengesetzt sind • wenn negative gruppensdynamische Effekte zu erwarten sind
Stärken	• Ermittlung innovativer Anforderungen möglich • Ideen können nicht aus Versehen in der Diskussion untergehen. • Der Requirements Engineer muss die Ideen nicht protokollieren. • kann negative Gruppendynamik und Hierarchiekonflikte verhindern
Schwächen	• für begrenzte Teilnehmerzahl anwendbar • Die Spontaneität und Inspiration der eigenen Kreativität durch Ideen anderer geht verloren, wenn nur eine Iteration durchgeführt wird.

Quelle: erstellt im Auftrag der IU, 2023.

Workshop

In einem Workshop kommen verschiedene Interessenvertreter von Stakeholdergruppen zusammen, um gemeinsam an der Ermittlung von Anforderungen zu arbeiten. Diese Vertreter müssen mit dem nötigen Fachwissen und der nötigen Entscheidungskompetenz ausgestattet sein, damit als Resultat des Workshops abgestimmte Anforderungen gewonnen werden. Dabei sind die räumliche Nähe und der direkte Austausch untereinander ein ideales Mittel, um in Konflikt zueinander stehende Anforderungen zu identifizieren und durch Auflösung der Konflikte bereits abgestimmte Anforderungen zu erhalten. Die Abstimmung mit allen Stakeholdern würde alternativ erst in einem gesonderten, nachgelagerten Schritt geschehen. Häufig gewinnt in solchen Workshops derjenige Interessenvertreter, dessen Gruppe die größte Macht besitzt, z. B. die Gruppe mit Budgetverantwortung.

Zur Durchführung eines erfolgreichen Workshops müssen feste Regeln definiert werden, und er muss von einem erfahrenen Requirements Engineer geleitet werden, der auch in komplizierten gruppendynamischen Prozessen zielführend moderiert. Häufig werden im Rahmen von Workshops Anforderungen auch nach Relevanz geordnet, inhaltlich konsolidiert und konkretisiert.

Ein Workshop besteht aus verschiedenen Phasen:

- In der Vorbereitung definiert der Requirements Engineer zunächst die Ziele des Workshops. Auf Basis der Ziele werden die Arbeitsergebnisse festgelegt und ein Vorgehen zur Zielerreichung definiert (z. B. eine Kombination der zuvor beschriebenen Techniken). Daraufhin wird der Ort, die Zeit und die Dauer des Workshops definiert und ein entsprechender Raum mit notwendigem Material (z. B. Beamer, Whiteboard, Flipchart) gebucht. Der Requirements Engineer wählt die beteiligten Stakeholder aus, lädt diese ein und stimmt die Workshopziele mit ihnen ab. Daraufhin vergibt der Requirements Engineer die Rolle des Protokollanten.
- Die Durchführung beginnt mit einer Eröffnung durch den Requirements Engineer, indem die Stakeholder begrüßt und Ziele und Arbeitsergebnisse sowie das Vorgehen erläutert werden. Dazu werden die anzuwendenden Techniken (z. B. schriftliches Brainstorming und Perspektivwechsel) vorgestellt und Regeln für den Workshop festgelegt (z. B. keine Kritik bei Ideenfindung; ausreden lassen). Im Arbeitsteil moderiert der Requirements Engineer, der Protokollant dokumentiert die Arbeitsergebnisse. Die Durchführung wird mit einer Sammlung offener Fragen und einer Retrospektive des Workshops abgeschlossen.
- Die Nachbereitung dient der Aufarbeitung der Arbeitsergebnisse durch den Requirements Engineer und die anschließende Genehmigung durch alle beteiligten Stakeholder.

Tabelle 7: Steckbrief „Workshop“

Einsatzszenarien	• zur Klärung offener Fragen • Ermittlung abgestimmter und geordneter Anforderungen
Stärken	• Konflikte werden direkt offensichtlich. • abgestimmte Anforderungen

- Gruppendynamische Effekte können auftreten.
- räumliche Zusammenarbeit notwendig

Quelle: erstellt im Auftrag der IU, 2023.

3.2 Befragungstechniken

Der erfolgreiche Einsatz von Befragungstechniken setzt voraus, dass die befragten Stakeholder fähig sind, Anforderungen explizit zu äußern, über die notwendige Zeit verfügen und den Willen zur Befragung haben. Mit Befragungstechniken lassen sich sowohl innovative als auch explizit gewünschte sowie grundlegende Anforderungen ermitteln. Zudem lassen sich sehr detaillierte Anforderungen ermitteln.

Interview

Das Interview ist eine vom Requirements Engineer aktiv gesteuerte Befragungstechnik, die es ermöglicht, sehr detaillierte Anforderungen zu ermitteln. Ein Interview wird in drei Phasen unterteilt:

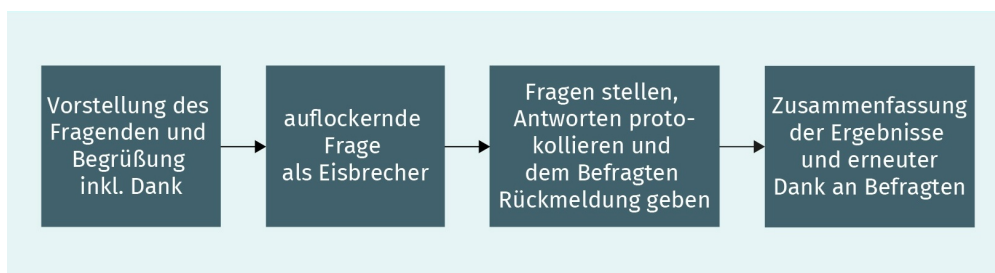
1. Phase: Vorbereitung

Um ein Interview durchzuführen, muss der Requirements Engineer die Fachsprache (sogenannte Terminologie) der Befragten kennen und diese nutzen. Darüber hinaus ist die explizite Definition des Interviewziels, die Auswahl und Einladung der Teilnehmer sowie die Festlegung des Intervieworts und die konkrete Gestaltung der Interviewfragen Bestandteil der vorbereitenden Aktivitäten. Die Interviewfragen können offen oder geschlossen formuliert werden. Bei offenen Fragen muss der Befragte selbstständig und frei antworten. Ein Beispiel für eine offene Frage ist: „Welche Funktionalität würden Sie sich für Ihre Arbeit mit dem zukünftigen System wünschen?“ Sie lassen dem Befragten also den Freiraum, viele Informationen zu geben, welche aber auch aufwendiger auszuwerten sind. Bei geschlossenen Fragen haben sich direkte Fragen (z. B. „Muss das Antragsystem Drag-and-drop unterstützen?“) und Feststellungen (z. B. „Das Antragsystem sollte Drag-and-drop unterstützen!“) besonders bewährt. Zur Beantwortung der geschlossenen Fragen gibt es verschiedene vorgegebene Antwortmöglichkeiten. Ja-/Nein-Antworten fragen die Zustimmung oder Ablehnung zu einer direkten Frage ab. Ratingskalen (z. B. stimme zu, neutral, stimme nicht zu) fragen den Grad der Übereinstimmung zu einer Feststellung ab. Dabei kommt es auf die formulierte Feststellung an, ob die Skala eine ungerade oder eine gerade Anzahl an Antwortmöglichkeiten bietet. Eine neutrale Antwortmöglichkeit ist immer bei einer ungeraden Anzahl an Antwortmöglichkeiten gegeben und kann dazu führen, dass Stakeholder, die sich nicht entscheiden wollen, immer die Mitte wählen. Mit einer geraden Anzahl an Antwortmöglichkeiten muss der Stakeholder also mindestens eine Tendenz der Zustimmung oder Ablehnung äußern.

2. Phase: Durchführung

Die Durchführung beginnt mit einer Vorstellung der eigenen Person, falls sich Requirements Engineer und Befragter nicht kennen, und einem kurzen Dank, dass der Befragte sich Zeit genommen hat. Es kann auch eine Einstiegsfrage gestellt werden, die nicht unbedingt Teil der Interviewfragen sein muss. Sie dient der Schaffung einer angenehmen, lockeren und positiven Atmosphäre. Daraufhin erläutert der Requirements Engineer das Ziel des Interviews und beginnt mit der Befragung. Die Antworten auf die Fragen müssen protokolliert werden, daher kann entweder ein Protokollant beisitzen oder das Gespräch aufgenommen werden. In beiden Fällen ist dies vorher mit dem Befragten zu besprechen. Der Requirements Engineer gibt dem Befragten Rückmeldung auf seine Antworten, um zu prüfen, ob er die Antwort richtig verstanden hat. Es können aber auch vertiefende, weiterführende Fragen gestellt werden. Darüber hinaus sollte der Requirements Engineer darauf achten, regelmäßig kleine Pausen einzulegen (z. B. nach jeder Antwort), die dem Befragten die Chance geben, das Gesagte zu reflektieren. Die Durchführung wird mit einer Zusammenfassung der Interviewergebnisse, einem Dank und dem Hinweis darauf, dass der Befragte eine Zusammenfassung der Ergebnisse erhält, abgeschlossen. Abbildung 9 gibt einen Überblick über die Schritte der Durchführung.

Abbildung 9: Schritte bei der Durchführung eines Interviews



Quelle: erstellt im Auftrag der IU, 2023 in Anlehnung an Rupp 2014, S. 110.

3. Phase: Nachbereitung

Zur Nachbereitung der Interviews wird das Protokoll aufbereitet und zur Bestätigung der Ergebnisse an den jeweiligen Befragten gesendet.

Die zuvor beschriebene Form der Interviewvorbereitung charakterisiert ein standardisiertes Interview. Im Gegensatz zu einem explorativen Interview wird hier auf ein konkretes Ziel hingearbeitet, die Fragen sind entsprechend vorbereitet. Explorative Interviews folgen keiner strukturierten Befragung, sondern lassen dem Befragten die Chance, sein Meinungsbild abzugeben. Standardisierte und explorative Interviews können beide als Einzel- oder Gruppeninterview durchgeführt werden. Der Vorteil bei einem Interview liegt in der Möglichkeit, aktiv zu steuern und durch gezieltes Nachfragen unbewusste Anforderungen bewusst zu machen. Demgegenüber steht der Nachteil, dass Interviews sehr aufwendig und daher nur für eine sehr begrenzte Zahl an Teilnehmern anwendbar sind.

Tabelle 8: Steckbrief „Interview“

Einsatzszenarien	• wenn die Anzahl der zu befragenden Stakeholder begrenzt ist • wenn sehr detaillierte und unbewusste Anforderungen ermittelt werden müssen
Stärken	• Ermittlung detaillierter Anforderungen möglich • Möglichkeit der expliziten Steuerung durch den Requirements Engineer
Schwächen	• für begrenzte Teilnehmerzahl anwendbar • Stakeholder müssen vor Ort sein.

Quelle: erstellt im Auftrag der IU, 2023.

Fragebogen

Fragebögen eignen sich, um eine große Anzahl von Stakeholdern zu befragen. Zunächst müssen die zu befragenden Stakeholder festgelegt werden, dabei ist eine Vollbefragung der Stichprobe grundsätzlich vorzuziehen. Ist keine Vollbefragung möglich, muss der Requirements Engineer eine repräsentative Menge Stakeholder auswählen. Weil der Requirements Engineer anders als bei einem Interview nicht die Möglichkeit hat, die Befragung aktiv zu steuern, müssen die Fragen besonders sorgfältig formuliert werden. Die Gewinnung der Fragen kann beispielsweise durch eine Kreativitätstechnik oder einen Workshop unterstützt werden. Unter Umständen ist eine Priorisierung der Fragen notwendig. Wenn der Befragte beispielsweise die Lust verliert oder zu viele Fragen beantworten muss, ist die Gefahr groß, dass die Befragung abgebrochen wird. Neben der Priorisierung der Fragen muss auch festgelegt werden, welchen Detaillierungsgrad die Fragen besitzen sollen. Es ist daher zu bestimmen, welche Fragen offen und welche geschlossen gestellt werden sollen. Digitale Fragebögen können entweder in Textverarbeitungsprogrammen oder in eigens dafür entwickelten Tools (z. B. SurveyMonkey) erstellt und durchgeführt werden. Der Vorteil bei der Nutzung eines Tools ist die Möglichkeit, die Daten direkt im Tool auswerten zu können.

Tabelle 9: Steckbrief „Fragebogen“

Einsatzszenarien	• wenn viele Stakeholder befragt werden müssen • wenn Stakeholder räumlich nicht greifbar sind
Stärken	• Befragung vieler Stakeholder möglich • einfache Auswertung digitaler Fragebögen mit geschlossenen Antworten • Stakeholder müssen nicht vor Ort sein.
Schwächen	• Implizites Wissen ist schlecht erfragbar. • Der Requirements Engineer hat keinen Einfluss auf die Befragung und kann keine Verständnisfragen klären oder selbst nachfragen.

Quelle: erstellt im Auftrag der IU, 2023.

3.3 Beobachtungstechniken

Beobachtungstechniken eignen sich besonders, wenn die Fachexperten keine Zeit haben oder nicht in der Lage sind, Anforderungen explizit zu äußern, z. B. weil diese als selbstverständlich vorausgesetzt werden. Der Requirements Engineer beobachtet Arbeitsabläufe, dokumentiert diese unter Berücksichtigung von Fehlern, Risiken und offenen Fragen und leitet daraus potenzielle Anforderungen an das System ab. Er muss die beobachteten Abläufe kritisch hinterfragen, um Soll-Situationen zu produzieren und Schwachstellen in Arbeitsabläufen und im ggf. schon bestehenden System offenzulegen. Im Folgenden werden die zwei Techniken Feldbeobachtung und Apprenticing vorgestellt.

Feldbeobachtung

In einer Feldbeobachtung ist der Requirements Engineer vor Ort und beobachtet unmittelbar die stattfindenden Geschäftsprozesse. Er erfasst die Aktivitäten und deren zeitliche Reihenfolge, um daraus die Arbeitsabläufe zu ermitteln. Dabei muss der Beobachter nicht nur passiv beobachten, sondern kann aktiv nachfragen und sich Arbeitsabläufe erläutern lassen. Der Beobachter muss, bevor er eine Feldbeobachtung durchführt, das Arbeitsumfeld der Stakeholder grundlegend verstehen und sich ein Bild davon machen, was die Stakeholder eigentlich tun.

Die große Gefahr bei einer Feldbeobachtung ist, dass der Requirements Engineer als Aufpasser wahrgenommen wird, der die Arbeit der Stakeholder überwacht. Daher muss der Requirements Engineer sehr feinfühlig sein. Sollten die Stakeholder sich beobachtet oder kontrolliert vorkommen, kann das Einfluss auf die Art des Arbeitens haben und damit die Prozesse und letztendlich die Anforderungen verfälschen.

Eine weitere Herausforderung ist das kritische Hinterfragen aller Abläufe, um Optimierungspotenzial zu identifizieren. Festgefahrene und verbesserungswürdige Prozesse sollten nicht eins zu eins in das neue System übernommen werden. Der Requirements Engineer kann die Feldbeobachtung durch Audio- und Videoaufzeichnungen unterstützen, muss in diesem Fall jedoch vorab das Einverständnis der Beobachteten einholen. Mit einer Feldbeobachtung lassen sich sprachlich schwer vermittelbare Abläufe identifizieren und unbewusste, als selbstverständlich vorausgesetzte Anforderungen ermitteln.

Tabelle 10: Steckbrief „Feldbeobachtung“

Einsatzszenarien	• wenn Abläufe schwer vermittelbar sind • wenn unbewusste, als selbstverständlich vorausgesetzte Anforderungen ermittelt werden sollen
Stärken	• Ermittlung unbewusster Anforderungen • Ermittlung schwer in Worte zu fassender Anforderungen • ist auch umsetzbar, wenn Stakeholder zeitlich schlecht verfügbar sind • Feststellung von Abweichungen in Prozessen möglich

Schwächen

- Abläufe müssen beobachtbar sein.
- Der Requirements Engineer kann als Aufpasser wahrgenommen werden, was das Verhalten der Leute und damit die Anforderungen beeinflusst.

Quelle: erstellt im Auftrag der IU, 2023.

Apprenticing

Das Apprenticing (Apprenticing heißt auf Deutsch sinngemäß „in die Lehre gehen“) ist eine weitere Beobachtungstechnik. Sie unterscheidet sich von der Feldbeobachtung durch die Tatsache, dass der Requirements Engineer nicht nur Abläufe beobachtet, sondern diese auch ausführt. Der Requirements Engineer geht bei Stakeholdern in die Lehre und erlernt deren Tätigkeit. Dies bietet ihm als Lehrling die Möglichkeit, unklare bzw. unverständliche Aktionen und Abläufe direkt zu hinterfragen. Darüber hinaus können auch hier unbewusste Abläufe für den Requirements Engineer explizit und durch Hinterfragen für den Stakeholder selbstverständliche Anforderungen herausgestellt werden. Dadurch fühlt sich der Stakeholder nicht mehr kontrolliert, sondern in einer erhabenen Position, da er als Lehrer sein Wissen weitergeben darf.

Das Erlernen verschiedenster Tätigkeiten ist sehr aufwendig, was als ein Nachteil dieser Technik zu sehen ist. Die Technik kann auch nicht in jedem Betätigungsfeld angewendet werden. Sicherheitskritische Tätigkeiten, wie beispielsweise die Überwachung eines Kraftwerks oder einer Produktionsstraße, eignen sich weniger, da der Lernende erheblichen Schaden anrichten kann, wenn Fehler unterlaufen.

Tabelle 11: Steckbrief „Apprenticing“

Einsatzszenarien	• wenn Abläufe schwer vermittelbar sind • wenn unbewusste, als selbstverständlich vorausgesetzte Anforderungen ermittelt werden sollen
Stärken	• Ermittlung unbewusster Anforderungen • Ermittlung schwer in Worte zu fassender Anforderungen • „Lernender“ statt „Beobachteter“ zu sein fördert die Kooperationsbereitschaft der Stakeholder. • Unwissen zu zeigen kann in schwierigen Situationen den Stakeholdern die Angst nehmen, dies auch zu tun.
Schwächen	• nicht in sicherheitskritischem Arbeitsumfeld anwendbar • Menge der Stakeholder ist nicht klar begrenzt.

Quelle: erstellt im Auftrag der IU, 2023.

3.4 Prototyping

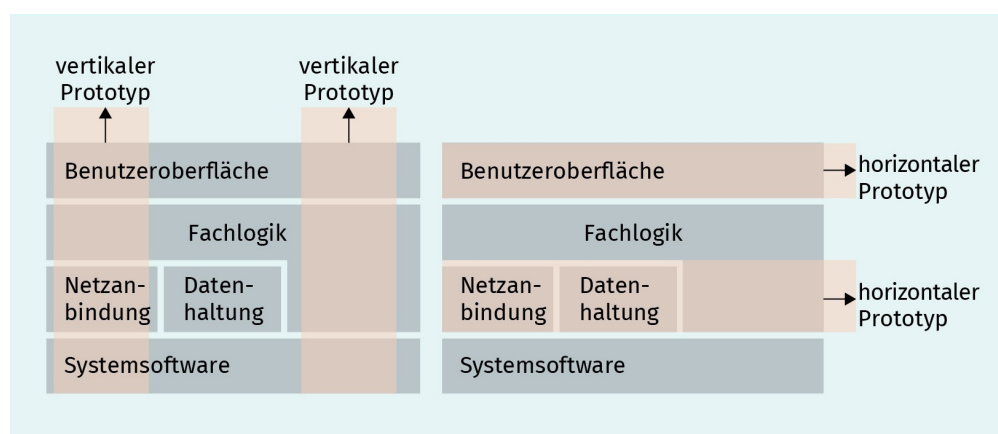
Ein Prototyp ist eine initiale Version eines Softwaresystems, mit der Konzepte demonstriert oder Entwürfe erprobt werden können, um generell mehr über das Problem und dessen möglichen Lösungen zu erfahren. Diese Definition verdeutlicht, dass Prototypen zur

Gewinnung von Anforderungen insofern genutzt werden können, als dass durch die Erstellung und Kommunikation ein tieferes Problemverständnis erlangt wird. Basierend auf diesem Verständnis können dann Anforderungen ermittelt werden.

Grundsätzlich können Prototypen hinsichtlich verschiedener Kriterien unterschieden werden:

- Die Beschaffenheit gibt an, ob es sich um analoge oder digitale Prototypen handelt.
- Die Beständigkeit bzw. der Verwendungszweck unterscheiden zwischen Prototypen, die nach der Erstellung und dem vorgesehenen Gebrauch nicht weiter entwickelt werden, sogenannte Wegwerfprototypen, und solchen, die zum späteren Produkt weiterverarbeitet werden, sich also zum Endprodukt weiterentwickeln, sogenannte evolutionär und inkrementelle Prototypen.
- Das dritte Kriterium nach dem man Prototypen unterscheiden kann, ist der Grad der umgesetzten Funktionalität. Horizontale Prototypen setzen eine Schicht der Softwarearchitektur um (z. B. die Datenhaltungsschicht). Die relevanten Bestandteile dieser Schicht werden implementiert, die anderen Schichten werden nicht implementiert. Der vertikale Prototyp wird auch Durchstichprototyp genannt, weil dieser eine Funktionalität oder ein Feature umsetzt – jedoch über alle Schichten der Softwarearchitektur hinweg. Abbildung 10 veranschaulicht, wie horizontale und vertikale Prototypen umgesetzt werden.

Abbildung 10: Vertikale und horizontale Prototypen



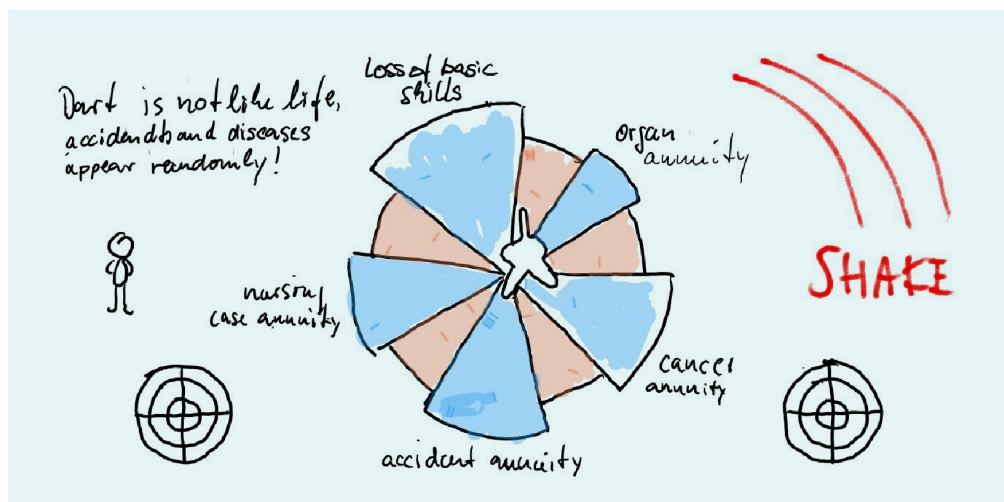
Quelle: erstellt im Auftrag der IU, 2023.

Prinzipiell können die Kriterien für Prototypen kombiniert werden, einige schließen sich jedoch aus. Analoge Prototypen können nicht vertikal „implementiert“ werden und sich auch nicht evolutionär und inkrementell zum Endprodukt weiterentwickeln. Im Folgenden werden die gängigen Arten horizontaler Prototypen eines Graphical User Interface (GUI) sowie der vertikale Prototyp vorgestellt.

Horizontale GUI-Prototypen

Horizontale GUI-Prototypen können als Hilfsmittel zur Abbildung von Dialogflüssen und zum Erwartungsmanagement genutzt werden. Je nach Projektphase werden entweder analoge Handskizzen oder digital implementierte Prototypen eingesetzt. Zu Beginn eines Projekts wird eine Handskizze verwendet (vgl. Abbildung 11), um schnell erste Konzepte zur Umsetzung einer grafischen Oberfläche zu visualisieren. Vorteil hierbei ist die intuitive Durchführbarkeit der Technik sowie die Gewinnung zusätzlicher funktionaler Anforderungen. Darüber hinaus suggeriert die Handskizze kein fertiges System, und die Stakeholder trauen sich, Änderungswünsche zu äußern.

Abbildung 11: Horizontaler Oberflächenprototyp als Handskizze



Quelle: Trac.assembla 2014.

Tabelle 12: Steckbrief „Handskizze“

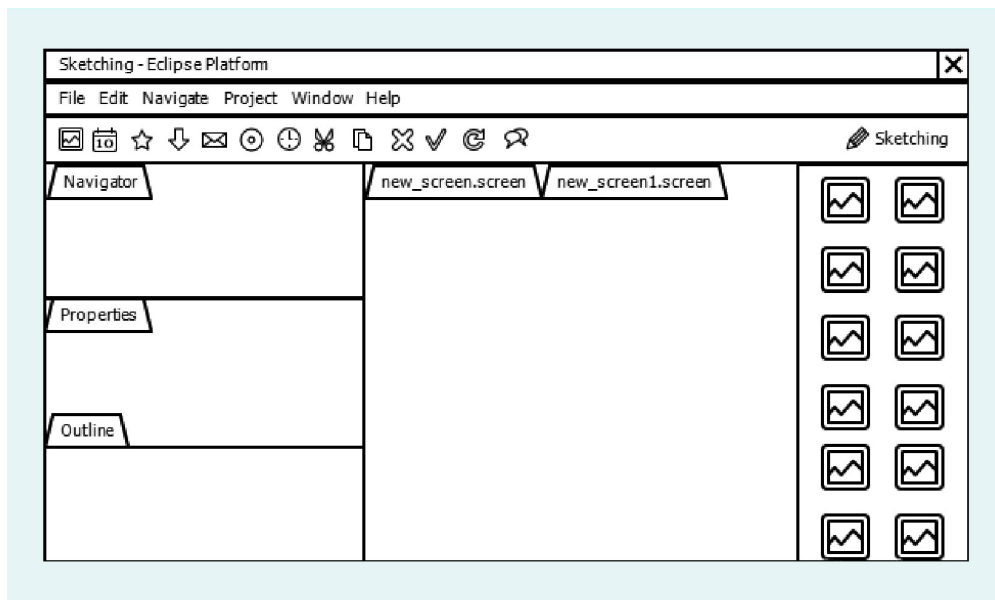
Einsatzszenarien	Handskizzen zur Erstellung erster Konzepte und zur initialen Abstimmung, also in frühen Projektphasen
Stärken	- schnell erstellt, benötigt kein Werkzeug - schnell änderbar, auch direkt in Workshop - technologieunabhängig
Schwächen	- nur Konzepte, keine Details - ungeeignet für große Prototypen - physisch begrenzt

Quelle: erstellt im Auftrag der IU, 2023.

Wireframes (vgl. Abbildung 12) sind digital angefertigte Oberflächenskizzen, die einheitlich gestaltet und ggf. klickbar, jedoch noch offensichtlich unfertig sind. Vorteil ist hier, dass der Navigationsfluss schon gezeigt werden kann und diese genauer sind als Handskizzen (z. B. kann der Styleguide der Organisation berücksichtigt werden). Hingegen ist

die Erstellung von Wireframes aufwendiger. Es können sowohl Vorlagen für Visio, als auch Desktop-Tools (z. B. WireframeSketcher) oder webbasierte Sketch-Werkzeuge (z. B. Mockingbird) genutzt werden.

Abbildung 12: Horizontaler Oberflächenprototyp als Wireframe



Quelle: Eclipse Tips – Prakash G. R. 2021.

Tabelle 13: Steckbrief „Wireframes“

Einsatzszenarien	• Wireframes zur Abstimmung funktionaler und struktureller Details der GUI
Stärken	• relativ einfach erstellt, benötigen aber Werkzeug • suggerieren dem Anwender noch kein fertiges System • Stakeholder „trauen“ sich, Änderungen zu wünschen. • Komplette GUI-Masken mit allen Elementen sind darstellbar. • Gestaltungsregeln sind einhaltbar (z. B. CI oder Bildschirmauflösung).
Schwächen	• Visueller Eindruck der GUI ist noch „verzerrt“ im Vergleich zu echten GUI.

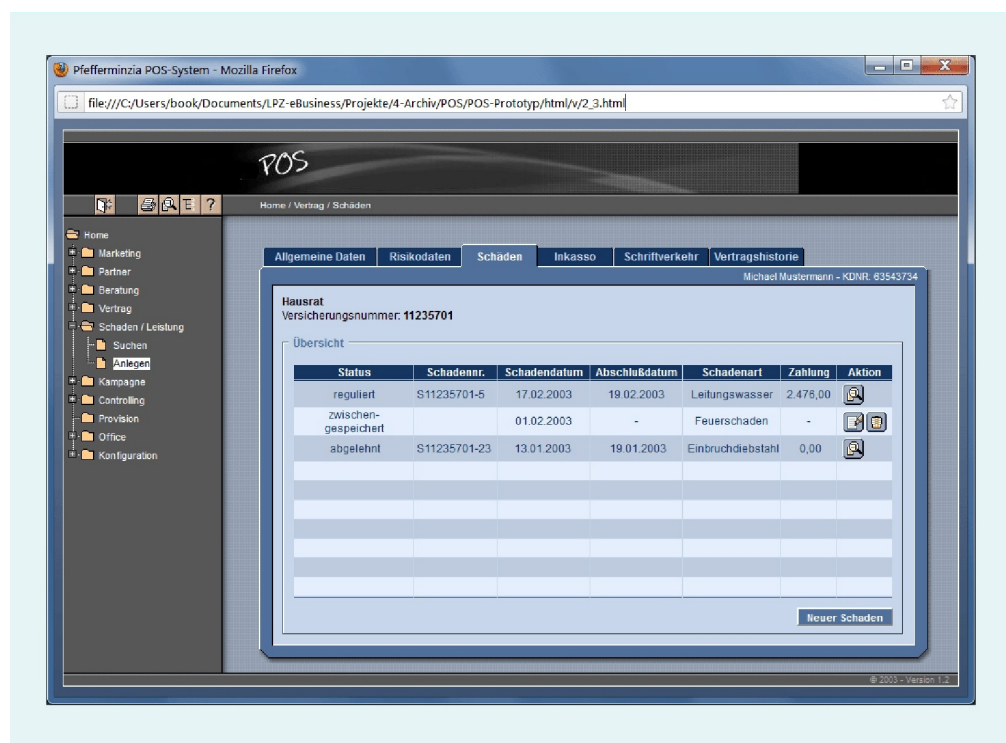
Quelle: erstellt im Auftrag der IU, 2023.

Ein Mock-up (vgl. Abbildung 13) ist ein in der Zieltechnologie des Systems implementierter Prototyp in Originalgröße und Farbe, jedoch ohne Funktionalität. Die Funktionalität kann durch das Hinterlegen eines unveränderlichen Datensatzes simuliert werden. Mock-ups werden eingesetzt, um das Erscheinungsbild des geplanten Systems zu visualisieren und abzustimmen. Diese Prototypen werden schnell und ohne Berücksichtigung besonderer Qualitätsanforderungen („quick and dirty“) entwickelt, wenn sie so schnell wie möglich verfügbar sein müssen. Dieser Prototyp darf unter keinen Umständen Teil des Endprodukts werden.

Ein Prototyp, der zum Produkt weiterentwickelt wird, wird evolutionär und inkrementeller Prototyp genannt. Da der Prototyp Teil des Endprodukts wird, sind schon bei dessen Entwicklung die Qualitätsanforderungen zu berücksichtigen und ist der Prototyp explizit unter den Qualitätsgesichtspunkten zu entwickeln. Dies führt jedoch zu einem Zielkonflikt zwischen früher Verfügbarkeit und der Erweiterbarkeit des Prototypen.

Bei Mock-ups ist es schwierig, dem Kunden zu vermitteln, dass neben der bereits implementierten GUI auch noch die Logik und die Datenhaltung sowie ggf. die Integration mit Umsystemen umzusetzen ist. Kunden haben meist kein Verständnis für die Aufwände der Softwareentwicklung, weil diese für Fachexperten meist nicht offensichtlich sind.

Abbildung 13: Horizontaler Oberflächenprototyp als Mock-up



Quelle: erstellt im Auftrag der IU, 2023.

Tabelle 14: Steckbrief „Mock-up“

Einsatzszenarien	Mock-ups sind zur Feinabstimmung von Anforderungen und Design und zur Abstimmung zum Einsatz auf mehreren Plattformen einsetzbar.
Stärken	- annähernd 1:1-Darstellung der echten Systemschnittstelle - Nutzer erkennt seine Anwendung wieder. - sehr detaillierte Abstimmungen möglich - Probieren der GUI an verschiedenen Endgeräten und Plattformen möglich - gut für Stakeholder mit geringem Vorstellungsvermögen - testbar mit echten Werten

Schwächen	• sehr aufwendig zu erstellen und anzupassen • suggeriert ggf. schon fertiges System
-----------	---

Quelle: erstellt im Auftrag der IU, 2023.

Vertikale Prototypen

Ein vertikaler Prototyp implementiert ausgewählte Funktionen des Zielsystems vollständig durch alle Systemschichten hindurch. Diese Technik ist dort geeignet, wo Funktionalitäts- und Implementierungsoptionen geklärt werden sollen. Dies ist in der Regel der Fall, wenn neue Technologien getestet oder Pilotsysteme gebaut werden. Ein vertikaler Prototyp ermöglicht es, einzelne Anwendungsfälle oder eine Zieltechnologie auszuprobieren oder die generelle Umsetzbarkeit zu testen.

Tabelle 15: Steckbrief „Vertikaler Prototyp“

Einsatzszenarien	• um Anwendungsfälle oder Zieltechnologien auszuprobieren • um Machbarkeitsstudien durchzuführen
Stärken	• ermöglicht es, Implementierungsoptionen zu testen
Schwächen	• ist sehr aufwendig

Quelle: erstellt im Auftrag der IU, 2023.



ZUSAMMENFASSUNG

Kreativitätstechniken eignen sich zur Entwicklung innovativer Anforderungen oder einer ersten Vision des zu entwickelnden Systems. Der Perspektivwechsel ist eine Kreativitätstechnik, die dazu dient, ein Problem aus verschiedenen Blickwinkeln zu betrachten. Das verbale Brainstorming wird mit einer Gruppe von fünf bis zehn Personen durchgeführt und dient der ersten Sammlung von Ideen. Die Variante des schriftlichen Brainstormings mit User Stories ermöglicht es, in Ruhe Ideen zu sammeln und diese zu notieren. In einem Workshop kommen verschiedene Interessenvertreter von Stakeholdergruppen zusammen, um gemeinsam an der Ermittlung von Anforderungen zu arbeiten.

Die Befragungstechniken setzen voraus, dass die befragten Stakeholder auf der einen Seite fähig sind, Anforderungen explizit zu äußern, und auf der anderen Seite über die notwendige Zeit verfügen und den Willen zur Befragung haben. Mit Befragungstechniken lassen sich sowohl innovative als auch explizit geforderte und grundlegende Anforderungen ermitteln. Das Interview ist eine vom Requirements Engineer aktiv gesteuerte Befragungstechnik, die es ermöglicht, sehr detaillierte Anforderungen zu ermitteln. Fragebögen eignen sich, um eine große Anzahl von Stakeholdern zu befragen.

Die Beobachtungstechniken eignen sich besonders für die Gewinnung grundlegender Funktionalität, da der Requirements Engineer Arbeitsabläufe beobachtet und dokumentiert, um daraus Systemfunktionalität abzuleiten. In einer Feldbeobachtung ist der Requirements Engineer vor Ort und beobachtet unmittelbar die stattfindenden Geschäftsprozesse. Das Apprenticing ist eine weitere Beobachtungstechnik, die sich von der Feldbeobachtung durch die Tatsache unterscheidet, dass der Requirements Engineer nicht nur Abläufe beobachtet, sondern diese auch ausführt.

Mit Prototyping können innovative Anforderungen gewonnen werden, indem initiale Systemversionen (sogenannte Prototypen) erstellt werden und so ein tieferes Problemverständnis und Wissen über potenzielle Lösungen erlangt wird. Prototypen können je nach Projektfortschritt unterschiedlich beschaffen sein (z. B. analoge Handskizzen und digitale Prototypen). Je weiter ein Projekt fortgeschritten ist, desto eher werden digitale Prototypen erzeugt. Horizontale GUI-Prototypen können als Hilfsmittel zur Abbildung von Dialogflüssen und zum Erwartungsmanagement genutzt werden. Ein vertikaler Prototyp implementiert ausgewählte Teile des Zielsystems vollständig durch alle Schichten hindurch. Diese Technik ist dort geeignet, wo Funktionalitäts- und Implementierungsoptionen noch offen sind.

LEKTION 4

DOKUMENTATION VON ANFORDERUNGEN

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- welche konkreten Aktivitäten zur Dokumentation von Anforderungen durchgeführt werden müssen.
- was es für typische Elemente in Anforderungsdokumenten gibt.
- welche typischen Dokumentationsformen es gibt und was sie für Vor- und Nachteile haben.

4. DOKUMENTATION VON ANFORDERUNGEN

Einführung

Das Ergebnis des Requirements Engineering ist eine Menge dokumentierter Anforderungen, auf deren Grundlage alle weiteren Aktivitäten im Softwareprozess erfolgen. Um eine zielgruppengerechte Dokumentationsform sicherzustellen, muss der Requirements Engineer aus verschiedenen Dokumentationsformen die geeigneten auswählen und die Anforderungen unter Berücksichtigung typischer Elemente von Anforderungsdokumentationen dokumentieren.

4.1 Aktivitäten zur Dokumentation von Anforderungen

Ziel der Kernaktivität „Dokumentation von Anforderungen“ ist die Sicherung des aktuellen Erkenntnisstands. Es wird sichergestellt, dass das gewonnene Wissen nicht nur in den Köpfen der beteiligten Personen vorhanden ist, sondern von allen Projektbeteiligten eingesehen werden kann. Einer der wichtigsten Zwecke der Dokumentation von Anforderungen ist damit die Unterstützung der Kommunikation innerhalb des Softwareprojekts. Aus folgenden Gründen ist Kommunikationsunterstützung in Form von dokumentierten Anforderungen wichtig:

- Anforderungen haben eine zentrale Bedeutung in Softwareprojekten, von ihnen sind alle folgenden Aktivitäten direkt oder indirekt abhängig.
- Anforderungen haben rechtliche Relevanz, insbesondere bei der Vergabe von Dienstleistungen an externe Dienstleister. Auf ihrer Basis werden Verträge geschlossen und Verbindlichkeiten eingegangen. Im Streitfall können die dokumentierten Anforderungen zur Klärung herangezogen werden.
- Die Menge der Anforderungen ist sehr komplex; IT-Systeme haben häufig tausende von Anforderungen, die teilweise voneinander abhängig sind.
- Die Verfügbarkeit der Anforderungen muss über die Projekt- bzw. Systemlaufzeit gewährleistet werden. Da die Menge der aktiv am Projekt bzw. am System arbeitenden Personen sich über den Lebenszyklus eines Softwaresystems ändert, müssen sich neue Projektmitglieder schnell und verbindlich in das Thema einarbeiten können.

Für die systematische Dokumentation von Anforderungen lassen sich vier Schritte identifizieren:

1. Bestimmung von Zweck und Zielgruppe,
2. Auswahl der Detailebene und Dokumentationsform,

3. Dokumentation der Anforderungen,
4. Prüfung, ob Dokumentation noch zu Zweck und Zielgruppe passt.

Bestimmung von Zweck und Zielgruppe

Da mit der Dokumentation gezielt die Kommunikation im Softwareprojekt unterstützt werden soll, müssen vor der Dokumentation der Zweck und die Zielgruppe bestimmt werden, für welche die Dokumentation erstellt wird. In Abhängigkeit der aktuellen Projektsituation können die relevante Zielgruppe und der konkrete Zweck der Dokumentation variieren. Typische Ziele für Anforderungsdokumentationen sind:

- Kommunikationsunterstützung der beteiligten Stakeholder, beispielsweise wenn Anforderungen geklärt und priorisiert oder die ermittelten Anforderungen den Architekten und Entwicklern zur Umsetzung übergeben werden müssen;
- Wissensspeicher und Referenz für Beschlüsse und Definitionen, damit auch noch nach der Inbetriebnahme und für Aktivitäten zur Wartung und Weiterentwicklung die Ergebnisse der ursprünglichen Anforderungsermittlung nachgelesen werden können;
- Verbindlichkeit von Aussagen und Klärung im Streitfall, damit es bei Missverständnissen oder Unklarheiten ein verbindliches Dokument gibt, das von allen Seiten als gemeinsame Vereinbarung anerkannt wird.

Mit der Bestimmung von Zweck und Zielgruppe der Dokumentation erlangt der Requirements Engineer Klarheit über die Verwendung der Dokumentation. So kann er gewährleisten, dass die Dokumentation am Zweck und an den Kenntnissen der Zielgruppe ausgerichtet wird. Die Dokumentation der Anforderungen kann daher vom Inhalt, vom Umfang und in der Dokumentationsform den tatsächlichen Erfordernissen angepasst werden. So wird unter Umständen für die Abstimmung mit dem Fachbereich eine andere Darstellung verwendet als für die Abstimmung mit dem Topmanagement, die sich wiederum von der Dokumentation für den Wissenstransfer zum Entwicklungsteam unterscheiden kann.

Beispiele für typische Kommunikationssituationen sind unter anderem:

- Diskussionen mit Fachbereich über funktionale Anforderungen,
- Entscheidungsvorlage für Management zum Freigeben des Budgets oder die
- Schätzung von Dauer und Umfang der Umsetzung durch Softwarearchitekten und Entwickler.

Typische Risiken im praktischen Umgang mit dokumentierten Anforderungen sind beispielsweise Stakeholder, welche die eingesetzte Dokumentationsform nicht verstehen, dieses jedoch nicht zugeben. Wenn beispielsweise Fachabteilungen mit sehr detaillierten Softwaremodellen konfrontiert werden, um Anforderungen an das System abzustimmen, besteht die Gefahr von Missverständnissen. Ein weiteres Risiko ist die Gefahr, dass wichtige und ggf. noch nicht abgestimmte Anforderungen in großen Dokumenten mit mehreren hundert Seiten verborgen sind und im Rahmen der Prüfung einfach übersehen werden. Wenn zudem Anforderungen nur als Sammlung von User Stories dokumentiert werden, fehlt die Übersicht und deren Anhängigkeiten zueinander.

Auswahl der Detailebene und Dokumentationsform

Nachdem Zweck und Zielgruppe der Dokumentation bestimmt wurden, wird als nächstes die Detailebene und die Darstellungsart ausgewählt. Da die dokumentierten Anforderungen je nach Einsatz von verschiedenen Personengruppen gelesen und verstanden werden, ist es wichtig, das Vorwissen und die Interessen der jeweiligen Stakeholder gezielt zu berücksichtigen. Hinsichtlich der Detailebene der Dokumentation ist zu entscheiden, ob beispielsweise ein Überblick ausreicht (z. B. für die Kommunikation mit dem Topmanagement), oder eine detaillierte Darstellung der Anforderungen benötigt wird (beispielsweise für das Schätzen von Aufwand und Dauer der Umsetzung). Nach der Bestimmung der erforderlichen Detailebene werden die benötigten Dokumentationsformen festgelegt. Typische und weit verbreitete Dokumentationsformen von Anforderungen sind Softwaremodelle, Prototypen, Skizzen, Tabellen und Text.

Dokumentation der Anforderungen

Auf Basis der vorhergehenden Aktivitäten können die Anforderungen nun in einer für den Zweck und die Zielgruppe geeigneten Form dokumentiert werden.

Prüfung, ob Dokumentation noch zu Zweck und Zielgruppe passt

Da sich die konkrete Dokumentation je nach Projekt über mehrere Wochen und Monate hinziehen kann, ist es wichtig, nach Abschluss noch einmal kritisch zu prüfen, ob das Ergebnis noch für den ursprünglichen Zweck und das ursprüngliche Ziel angemessen ist. Häufig kommt es in der Praxis vor, dass sich das Dokumentationsteam in die Arbeit so vertieft hat, dass der ursprüngliche Zweck aus den Augen verloren wurde. Darüber hinaus passiert es häufig, dass sich während der Dokumentation, als Folge von aktuellen Entwicklungen im Projekt, Änderungen zu Zielgruppe oder Zweck ergeben haben. In diesem Fall kann es nötig sein, dass die Dokumentation auf die geänderten Bedingungen noch einmal angepasst werden muss.

4.2 Typische Elemente der Anforderungsdokumentation

Grundsätzlich gibt es keine einheitliche Struktur für die Anforderungsdokumentation, die in allen Projekten genutzt werden kann. Sowohl die Art der Dokumentation, beispielsweise Backlogs, Tickets oder Word-Dokumente, als auch die konkrete Struktur und deren Inhalt sind abhängig von der Art des Systems, der Organisation des Softwareprojekts und den unternehmensspezifischen Vorgaben. Häufig gibt es verbindliche Vorgaben oder Empfehlungen für die Gliederung von Anforderungsdokumentation bzw. ein Template für Anforderungen. Mit der Vorgabe der Struktur soll erreicht werden, dass Anforderungen projektübergreifend vergleichbar beschrieben werden. Das erleichtert zum einen die Einarbeitung in die Dokumentation sowie das Zurechtfinden in den recht umfangreichen

Dokumentationen und gibt dem Ersteller der Dokumentation eine Hilfestellung beim Verfassen. Darüber hinaus dient eine vorgegebene Struktur auch als Checkliste, ob alle erforderlichen Inhalte in der Dokumentation berücksichtigt wurden.

Fast vorgegebene Dokumententemplates entwickeln sich häufig in der Praxis zu Write-only-Dokumenten: Sie sind zwar als Konvention veröffentlicht, werden jedoch in den Projekten nicht eingesetzt. Das liegt oft daran, dass es sehr unterschiedliche Projekte mit ganz spezifischen Anforderungen an die Dokumentenstruktur gibt. Allerdings wird beim Erstellen von der Dokumentenstruktur mehr Wert auf die Vollständigkeit als auf eine einfache und pragmatische Handhabung im Projekt gelegt. Daher sollten Vorgaben zu Dokumentenstrukturen kontinuierlich beobachtet und auf den praktischen Bedarf hin angepasst werden.

Obwohl es keine allgemeingültige Gliederung für Anforderungsdokumente gibt, lassen sich jedoch vier typische und im Folgenden erläuterte Elemente für Anforderungsdokumentationen identifizieren:

- Produkt- bzw. Projektvision,
- Überblicksebene,
- detaillierte Anforderungen sowie ein
- Glossar.

Diese sind zwar je nach Einsatz unterschiedlich ausgeprägt, jedoch sollten sie beim Erstellen von Dokumentenstrukturen für Anforderungsdokumente unbedingt mit berücksichtigt werden.



TIPP

Prüfen Sie jedes konkrete Anforderungsdokument dahingehend, ob sich in seiner Struktur die hier vorgestellten Elemente identifizieren lassen. Die Abschnitte müssen dabei nicht genau gleich benannt sein. Wichtig ist nur der Inhalt.

Produkt- bzw. Projektvision

Zu Beginn eines Projekts müssen die Vorgaben und Anforderungen des Auftraggebers ermittelt, dokumentiert und abgestimmt werden. Ausgehend von der ersten Projektidee werden dabei Stück für Stück Vorgaben, Ziele und Erwartungen des Auftraggebers identifiziert und festgelegt. Die Ausformulierung der Vision als Text hilft dem Team bei der Erarbeitung eines gemeinsamen Verständnisses über die Vision und in der Kommunikation mit den Stakeholdern.



TIPP

Formulieren Sie im ersten Workshop des Projekts die Produkt- bzw. Projektvision bzw. das Ziel als Text im Umfang von einer halben DIN-A4-Seite (ca. 1.600 Zeichen) und stimmen Sie diese mit allen Beteiligten ab.

Die so erhaltene Projektvision sollte der erste Abschnitt innerhalb eines Anforderungsdokuments sein. In diesem Abschnitt ist Folgendes kurz (ca. 1.600 Zeichen) beschrieben:

1. Was ist das Ziel bzw. der Zweck des Projekts, d. h., was soll mit dem Projekt erreicht werden?
2. Welche Motivation gibt es für das Projekt, d. h., aus welchen Gründen gibt es dieses Projekt?
3. Festlegung der Ebene der in diesem Dokument beschriebenen Anforderungen, idealerweise gemeinsam mit dem Zweck und der Zielgruppe des Dokuments.

Die Produkt- bzw. Projektvision dient neben der Einleitung und Hinführung auf die Inhalte des Dokuments ebenso als Anker und Abholer für Workshops und Besprechungen mit den Stakeholdern. Darüber hinaus kann sie für die abschließende Prüfung, ob Inhalte und Form noch zu Zweck und Zielgruppe passen, eingesetzt werden.

Überblicksebene

Im Anschluss an die Projektvision folgt die Überblicksebene, die zur technischen und fachlichen Einordnung der Anwendung dient. Nach der Lektüre der Überblicksebene sollte der Leser einen allgemeinen Überblick über die Hauptfunktionen des Systems bzw. der anzupassenden Systeme, die technischen Schnittstellen, die Nutzer sowie die Einordnung in die Systemlandschaft erlangen können. Je nach Ziel und Kontext des Projekts werden nicht nur ein System, sondern mehrere Systeme verändert. In diesem Fall müssen auch systemübergreifende Anforderungen berücksichtigt und dokumentiert werden:

- Verortung der Systeme in fachlicher Prozesslandschaft, d. h. eine kurze Beschreibung, welche fachlichen Prozesse und Funktionen durch die Systeme unterstützt werden;
- Einbettung der Systeme in die IT-Landschaft der Organisation, damit eine Einordnung der Systeme im Vergleich zu anderen IT-Systemen in der Anwendungslandschaft möglich ist;
- Kurzbeschreibung der wichtigsten Systemfunktion, damit sich der Leser einen Überblick über die einzelnen Funktionen der Systeme verschaffen kann, z. B. mit Use Cases;
- technische Schnittstellen zu anderen Systemen, damit allen Beteiligten die technischen Abhängigkeiten rechtzeitig bekannt sind und frühzeitig mit der technischen Integration begonnen werden kann;
- Kurzbeschreibung der Rollen, die aktiv mit den Systemen arbeiten werden, damit klar ist, welche Stakeholder aktiv bei der Anforderungsermittlung mit einbezogen werden müssen.

Wenn im Folgenden vereinfachend von dem System gesprochen wird, sind damit alle verändernden Systeme gemeint. Bei Bedarf müssen die Dokumentationen aller zu verändernden Systeme angepasst werden.

Detaillierte Anforderungen

Jede im Überblick kurz beschriebene Systemfunktion muss angemessen dokumentiert werden. Daher ist der Bereich mit den detaillierten Anforderungen in der Regel auch der größte im Anforderungsdokument. Dabei sollte jede Systemfunktion kurz beschrieben werden, damit der Leser einen Überblick über die Funktion erhält. Angenommen, für einen Onlineshop wurden die wichtigen Systemfunktionen „Artikel in Bestand aufnehmen“, „Waren durchsuchen“, „Artikel kaufen“ und „Newsletter versenden“ identifiziert, dann muss es zu jeder dieser Funktionen im Bereich „Detaillierte Anforderungen“ einen Abschnitt geben, in dem diese ausführlich beschrieben werden.

Typischerweise setzt sich eine Systemfunktion aus verschiedenen Teilfunktionen zusammen. So kann beispielsweise die Funktion „Artikel kaufen“ eines Onlineshops aus den Teilfunktionen „Artikel in den Warenkorb legen“, „Artikel bezahlen“, „Versandabwicklung“ und „Bewertung“ zusammengesetzt sein. Diese Teilfunktionen werden innerhalb des Abschnitts der Systemfunktion dokumentiert. Zu jeder Teilfunktion werden vor dem Start der Implementierung alle Informationen zusammengestellt, die das Entwicklungsteam benötigt, um mit der Implementierung des Systems zu beginnen. Das schließt insbesondere alle relevanten funktionalen Anforderungen, Qualitätsanforderungen und Randbedingungen ein. Diese Anforderungen können in Form von Texten, als Aufzählungen, Tabellen, fachlichen Modellen, Referenzen auf externe Dokumente und Screenshots von Prototypen dokumentiert werden.

Ergänzend zu der Dokumentation der einzelnen Systemfunktionen ist es häufig sinnvoll, ein übergreifendes fachliches Objektmodell zu erstellen. Mit dem Objektmodell können die fachlichen Zusammenhänge und Eigenschaften von Geschäftsobjekten beschrieben und dokumentiert werden. Dazu kann das UML-Klassendiagramm eingesetzt werden.

Glossar

Ein weiteres wichtiges Element einer Anforderungsdokumentation ist das Glossar. Da es Fachbegriffe auf der Seite des Fachbereichs und auf der Seite der IT gibt, müssen diese im Glossar des Anforderungsdokuments gesammelt und erläutert werden. Das betrifft insbesondere fach- und organisationsspezifische sowie technische Abkürzungen. Darüber hinaus sollten ebenfalls solche Begriffe mit in das Glossar aufgenommen werden, die eine projektspezifische Bedeutung haben oder die im Rahmen des Projekts abweichend zu ihrer allgemein bekannten Bedeutung verwendet werden.

4.3 Dokumentationsformen

Ein wichtiger Aspekt bei der Dokumentation von Anforderungen an IT-Systeme ist ein möglichst kleiner Interpretationsspielraum. Dokumentierte Anforderungen sollten möglichst eindeutig formuliert werden, damit das IT-System in den folgenden Aktivitäten genau die Funktionen und Eigenschaften erhält, die bei der Erstellung der Dokumentation auch tatsächlich gemeint waren. Als mögliche Dokumentationsformen von Anforderungen kann jede Art der Darstellung verwendet werden, die die Kommunikation und Verständigung zwischen den beteiligten Stakeholdern erleichtert. Wie oben bereits genannt, sind typische und weit verbreitete Dokumentationsformen von Anforderungen Modelle, Prototypen, Skizzen, Tabellen und Text.

Text und Tabellen

Mit der Dokumentationsform Text und Tabelle können grundsätzlich alle Anforderungen beschrieben werden. Beide sind die in der Praxis für die Dokumentation von Informationssystemen am häufigsten verwendete Dokumentationsform. Die einfachsten Formen der Dokumentation von Anforderungen mit Text sind das Formulieren von geschriebenem Text oder die Auflistung einzelner Anforderungen. Tabellen helfen wiederum, die Anforderungen etwas zu strukturieren. Mit ihnen können z. B. Wertebereiche, konkrete Eigenschaften oder fachlich verschiedene, strukturell jedoch ähnliche Aspekte übersichtlich dargestellt werden.

Eine weitere Möglichkeit, geschriebene Sprache mit Struktur zu versehen, sind User Stories oder der Einsatz von Satzschablonen (siehe Pohl/Rupp 2015, S. 63ff.).

Die Vorteile der Dokumentation in Form von Text und Tabellen liegen in der einfachen Anwendung. Weder für das Dokumentieren noch für das Lesen der Dokumentation ist zusätzlicher Lernaufwand erforderlich. Darüber hinaus sind beide vielseitig einsetzbar, denn alle Arten von Anforderungen können mit ihnen beschrieben werden. Der Nachteil ist allerdings der hohe Interpretationsspielraum sowie Ungenauigkeiten in der Formulierung von Anforderungen. So wird beispielsweise häufig bei der Beschreibung von Entscheidungen in Abläufen nur der positive Fall berücksichtigt, wie in folgendem Beispiel:

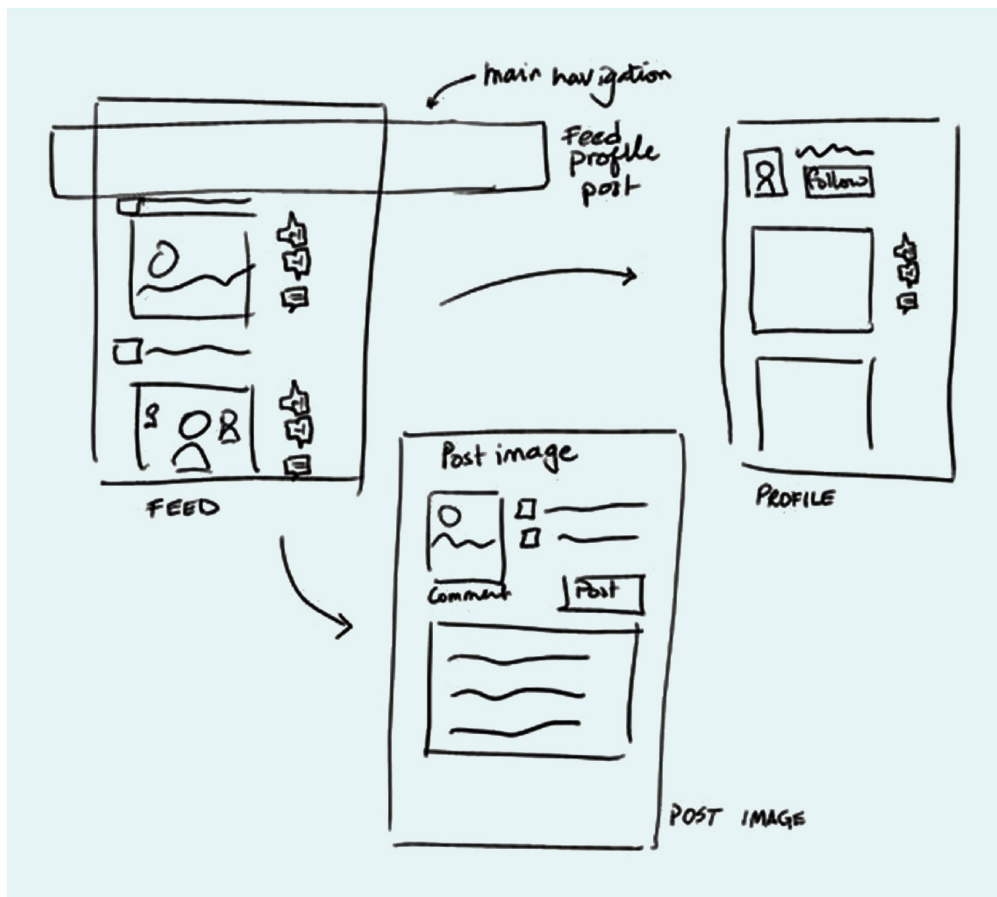
Nach der erfolgreichen Anmeldung am System werden die zur Verfügung stehenden Zahlungsoptionen angezeigt.

Oft wird vergessen, das Verhalten für den Fall zu beschreiben, bei dem die Anmeldung bzw. ein anderes Verhalten nicht funktioniert. Die Nachteile der Dokumentation ausschließlich in Textform sind in der Praxis so gravierend, dass sich neben dem Text weitere Dokumentationsformen etabliert haben.

Skizzen und einfache Grafiken

Ein einfaches Hilfsmittel, um Anforderungen etwas präziser und anschaulicher als mit Text zu dokumentieren, sind einfache Skizzen. Diese können entweder per Hand gezeichnet und fotografiert oder auch z. B. mit Programmen wie PowerPoint oder Visio erstellt und als Grafik in die Anforderungsdokumentation eingebracht werden.

Abbildung 14: Beispiel für Skizzen



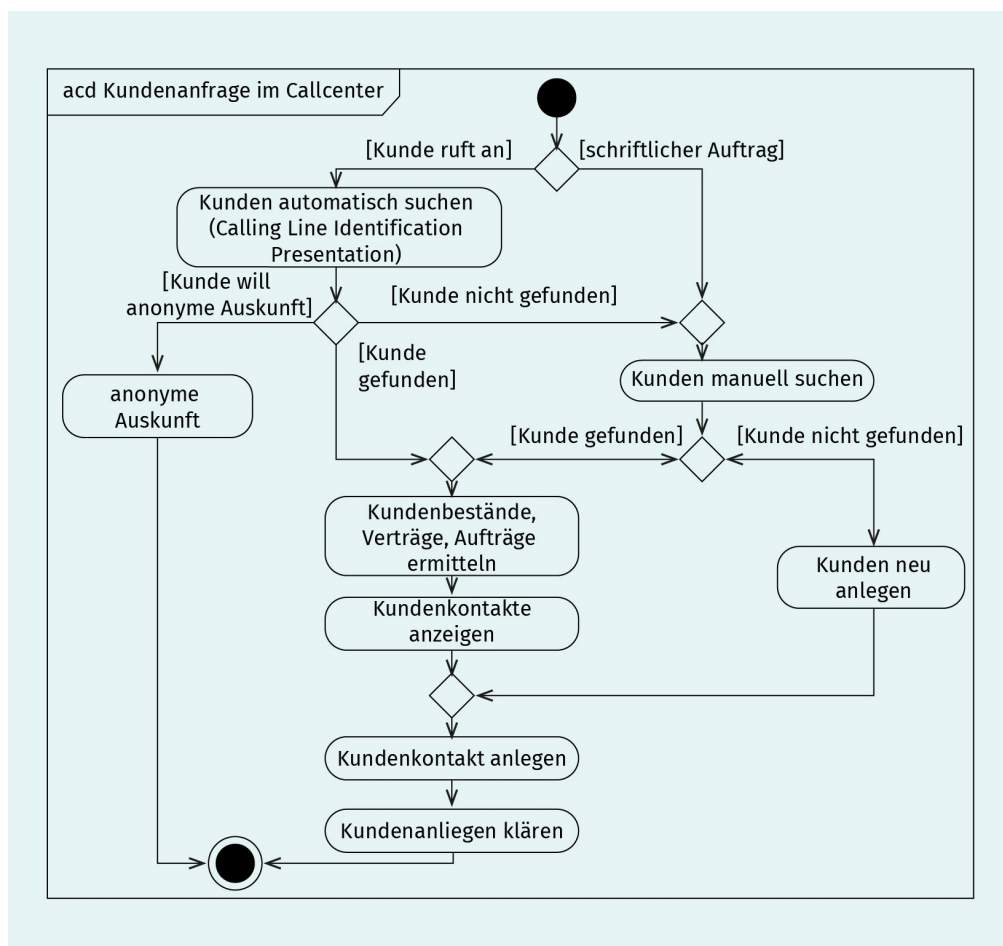
Quelle: Keynotopia 2013.

Der Vorteil von Skizzen und einfachen Grafiken, wie beispielhaft in Abbildung 14 gezeigt, liegt in der einfachen und schnellen Erstellung. Sie können beispielsweise direkt in einem Workshop mit den Stakeholdern erstellt und als Foto in das Dokument eingebracht werden. Insbesondere eignen sich Skizzen in frühen Phasen der Anforderungsermittlung, wenn noch keine technischen Details, sondern fachliche Zusammenhänge herausgestellt und dokumentiert werden müssen. Ein Nachteil von Skizzen und einfachen Grafiken liegt in ihrem großen Interpretationsspielraum. Was genau die in der Skizze oder der Grafik verwendeten Symbole, Formen und Farben bedeuten, ist in der Regel nur den direkt an der Erstellung beteiligten Personen bekannt. Daher eignen sie sich nur bedingt zur dauerhaften Dokumentation und sollten in jedem Fall durch eine Legende bzw. eine ausführliche Beschreibung ergänzt werden.

Modelle

Um den möglichen Interpretationsspielraum von Text, Skizzen und Grafiken weiter zu reduzieren, hat sich im industriellen Softwareengineering der Einsatz von Modellen etabliert. Grundsätzlich werden dabei grafische Modelle und textuelle Modelle unterschieden. Die grafischen Modelle haben eine visuelle Ausprägung, d. h., es handelt sich dabei um Grafiken, deren Notationselemente jeweils eine ganz bestimmte Bedeutung haben. Im Requirements Engineering werden sowohl grafische Prozessmodelle, z. B. BPMN (Business Process Model and Notation) oder EPK (Ereignisgesteuerte Prozesskette), als auch grafische Softwaremodelle, z. B. UML (Unified Modeling Language), eingesetzt. Mit grafischen Modellen werden Anforderungen zu Struktur und Verhalten von Systemen dokumentiert. Abbildung 15 zeigt beispielhaft einen Ablauf, der mit einem Diagramm modelliert wurde. Konkret wird dabei eine eingehende Kundenanfrage in einem Callcenter dargestellt. Mit den Prozessmodellen BPMN und EPK sowie der UML steht dem Requirements Engineer eine breite Auswahl von Diagrammtypen zur Verfügung. So kann er je nach aktueller Situation das passende Modell auswählen.

Abbildung 15: Beispiel für ein Modell, hier ein UML-Aktivitätsdiagramm



Quelle: erstellt im Auftrag der IU, 2023.

Neben den grafischen Modellen gibt es im Softwareengineering auch Modelle, die als strukturierter Text notiert werden. Ein Beispiel dafür ist XML, mit dem Datenstrukturen an technischen Systemschnittstellen dokumentiert werden können. Abbildung 16 zeigt ein Beispiel für ein Modell in Textform, in dem Kundendaten in einem System strukturiert werden.

Abbildung 16: Beispiel für ein textuelles Modell in Form einer XML-Datei

```
<?xml version="1.0" encoding="UTF-8"?>
<kunde>
  <kundennummer>9928176251</kundennummer>
  <anschrift>
    <vorname>Heinz</vorname>
    <nachname>Keller</nachname>
    <strasse>Hauptstrasse</strasse>
    <hausnummer>1b</hausnummer>
    <postleitzahl>76339</postleitzahl>
    <stadt>Freustadt</stadt>
  </anschrift>
  <aufträge>
    <auftrag>
      <auftragsnummer>a823178aj</auftragsnummer>
    </auftrag>
    <auftrag>
      <auftragsnummer>b634671kj</auftragsnummer>
    </auftrag>
  </aufträge>
</kunde>
```

Quelle: erstellt im Auftrag der IU, 2023.

Der Vorteil der Dokumentation mit Modellen ist der geringere Interpretationsspielraum. Da zu jedem Modellelement die Bedeutung bekannt ist, kann der Leser sich den in einem Modell ausgedrückten Sachverhalt selbstständig erschließen. Er muss den Ersteller des Diagramms nicht zur Bedeutung der verwendeten Elemente fragen. Darüber hinaus kann mit Modellen Information kompakter dokumentiert werden. Dabei erschließen sich grafische Modelle dem Leser meistens auch deutlich schneller als ausformulierter Text. Der Nachteil von Modellen im Vergleich zu Text oder Skizzen ist der erforderliche Schulungsaufwand. Um die Modelle richtig erstellen und lesen zu können, ist das Wissen über die Menge der verfügbaren Notationselemente und deren Verwendung erforderlich. Außerdem sind Modelle nicht universell einsetzbar. Sie konzentrieren sich in der Regel auf ganz bestimmte Aspekte, z. B. auf die Struktur oder das Verhalten von Systemen. Um ein System unter verschiedenen Gesichtspunkten zu modellieren, sind daher verschiedene Diagrammtypen erforderlich.

GUI-Prototypen

Insbesondere für die Dokumentation von Anforderungen an Benutzerschnittstellen eignen sich GUI-Prototypen als Dokumentationsform. Neben dem Einsatz dieser Prototypen zur Ermittlung von Anforderungen können GUI-Prototypen auch direkt als Dokumentationsform für Anforderungen eingesetzt werden. Mit ihnen können z. B. die genaue Größe, Position, Farbe und Beschriftung von Elementen der Nutzeroberfläche präzise festgelegt werden. Darüber hinaus kann mit ihnen die Erscheinung und der Inhalt von Fehlermeldungen genauso dokumentiert werden wie beispielsweise die Reihenfolge der einzelnen Dialogmasken in Abhängigkeit von der Nutzerinteraktion. GUI-Prototypen können auf verschiedenen Ebenen eingesetzt werden, von einfachen Skizzen, über Wireframes bis hin zu vollständigen Mock-ups.

Mischform verschiedener Dokumentationsformen

Im praktischen Einsatz werden Anforderungen an Informationssysteme in der Regel in einer Mischung von Modellen, ausformuliertem Text und GUI-Prototypen dokumentiert. Diese Kombination gleicht die individuellen Nachteile der einzelnen Dokumentationsformen aus. So sollte beispielsweise ein grafisches Modell immer durch einen Text erläutert werden. Dabei muss zwar nicht der Inhalt des Modells vollständig beschrieben werden, wichtig ist aber in jedem Fall eine kurze Beschreibung darüber, was konkret im Modell dargestellt ist und wie das dort Dargestellte in Bezug zu den anderen dokumentierten Anforderungen steht. Außerdem sollten alle wichtigen Notationselemente wie Aktionen, Funktionen, Informationsobjekte, Zustände oder Klassen kurz erläutert werden. Das unterstützt den Leser beim Verständnis und der Einordnung des modellierten Sachverhalts.

Insbesondere beim Einsatz verschiedener Dokumentationsformen muss bei der Dokumentation und bei allen späteren Überarbeitungen und Anpassungen darauf geachtet werden, dass Text und Modell den gleichen Erkenntnisstand wiedergeben und es keine Inkonsistenzen in der Dokumentation gibt.



ZUSAMMENFASSUNG

Ziel der „Kernaktivität Dokumentation von Anforderungen“ ist die Sicherung des aktuellen Erkenntnisstands und der Kommunikationsunterstützung zwischen den beteiligten Personen. Die systematische Dokumentation von Anforderungen erfolgt in vier Schritten. Zuerst wird in Abhängigkeit der aktuellen Projektsituation Zweck und Zielgruppe der Dokumentation bestimmt, bevor darauf aufbauend die Detailebene und Dokumentationsform der Anforderungen festgelegt wird. Anschließend werden die Anforderungen dokumentiert und abschließend wird geprüft, ob die erstellte Dokumentation noch zu Zweck und Zielgruppe passt.

Zwar gibt es keine allgemeingültige Dokumentationsstruktur für Anforderungsdokumente, jedoch sollten die vier Elemente Produkt- bzw. Projektvision, Überblicksebene, detaillierte Anforderungen sowie ein Glossar in der Anforderungsdokumentation unbedingt mitberücksichtigt werden.

Zur Dokumentation von Anforderungen kann jede Art der Darstellung verwendet werden, welche die Kommunikation und Verständigung zwischen den beteiligten Stakeholdern erleichtert. Typische und weit verbreitete Dokumentationsformen von Anforderungen sind Softwaremodelle, Prototypen, Skizzen, Tabellen und Text. Je nachdem, ob funktionale Anforderungen, Qualitätseigenschaften oder Randbedingungen dokumentiert werden sollen, muss eine geeignete Dokumentationsform ausgewählt und eingesetzt werden. Im praktischen Einsatz werden Anforderungen an Informationssysteme in der Regel aus einer Mischung von Modellen, ausformuliertem Text und GUI-Prototypen dokumentiert. Auf diese Weise werden Vor- und Nachteile der individuellen Dokumentationsformen ausgeglichen.

LEKTION 5

MODELLIERUNG VON PROZESSEN

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- was unter Unternehmensmodellierung, Aufbauorganisation und Ablauforganisation verstanden wird.
- was ein Geschäftsprozess ist und aus welchen Teilen er besteht.
- auf welchen Abstraktionsebenen Geschäftsprozesse modelliert werden können.
- was die Grundelemente der Business Process Model and Notation sind.
- was die Grundelemente erweiterter Ereignisgesteuerter Prozessketten sind.

5. MODELLIERUNG VON PROZESSEN

Einführung

In dieser Lektion werden zunächst Grundbegriffe von Geschäftsprozessen erläutert, und es wird dargelegt, warum Geschäftsprozesse wichtig für das Requirements Engineering sind. Danach werden mit der Business Process Model and Notation und den Ereignisgesteuerten Prozessketten zwei etablierte Modellierungsnotationen vorgestellt.

5.1 Grundlagen und Begriffe

In diesem Lernzyklus werden die Grundlagen und Begriffe zur Unternehmensmodellierung erläutert. Dazu zählen neben der Aufbauorganisation auch die Ablauforganisation und die Bestandteile von Geschäftsprozessen. Die Bestandteile eines Prozesses können verwendet werden, um die Geschäftsprozesse zu modellieren. Da Geschäftsprozesse die Grundlage des betrieblichen Handelns bilden, stellen sie eine elementare Grundlage für das Requirements Engineering für betriebliche Informationssysteme dar. In einem Geschäftsprozess werden Aufgaben durch bestimmte Organisationseinheiten erledigt. In diesen Aufgaben werden Entscheidungen getroffen und Geschäftsobjekte bearbeitet. Die Bearbeitung dieser Aufgaben kann durch IT-Systeme unterstützt werden, sodass die Angemessenheit und Korrektheit der Anforderungen an dieses System einen wesentlichen Einfluss auf den Erfolg des Unternehmens haben. Im Folgenden wird zuerst erläutert, wie Organisationen strukturell aufgebaut sein können (sogenannte Aufbauorganisation), um daraufhin die Ziele von Geschäftsprozessen in der Ablauforganisation und die Elemente von Geschäftsprozessen sowie deren Beziehungen zueinander zu erläutern.

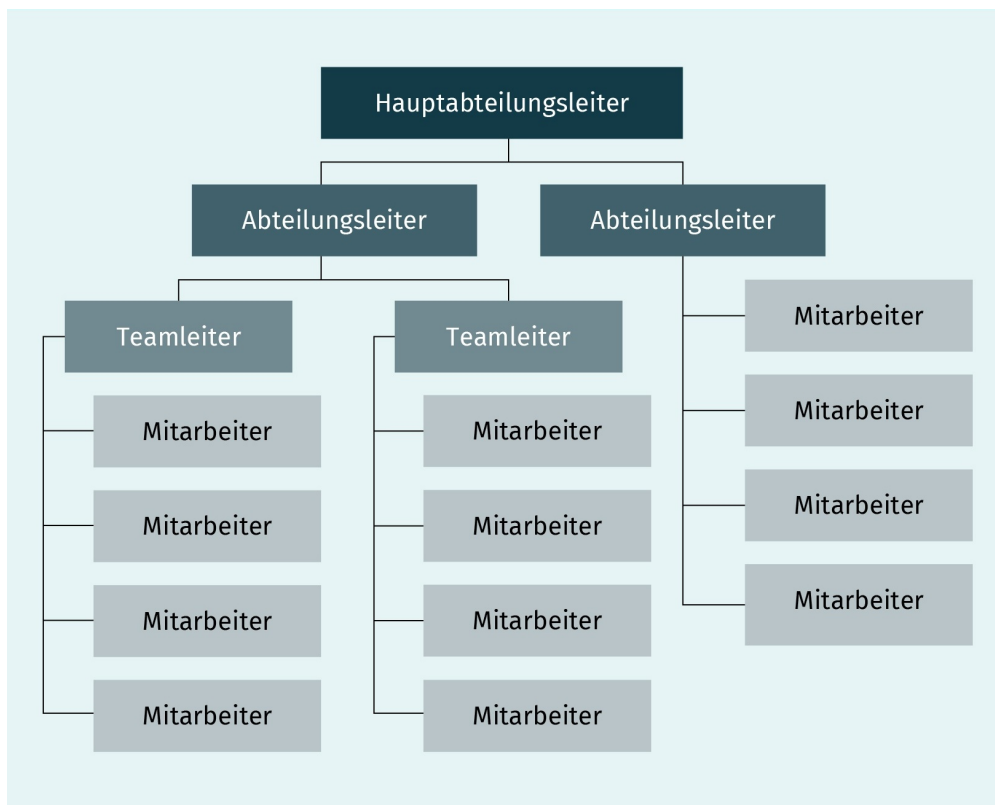
Aufbauorganisation

Die Aufbauorganisation bildet die Organisationsstruktur eines Unternehmens. Diese Struktur ist in den meisten Fällen hierarchisch aufgebaut und legt die Rahmenbedingungen für die Bearbeitung von Aufgaben in einem Unternehmen fest. Es wird also festgelegt, welche Aufgaben von welchen Menschen mit welchen Sachmitteln erledigt werden sollen. Die Ziele der Aufbauorganisation sind eine arbeitsteilige Gliederung und Ordnung der betrieblichen Handlungsprozesse durch Bildung und Verteilung von Aufgaben. In einer Hierarchie werden Führungsstrukturen und damit Weisungsbefugnisse gebildet, die eine Zuordnung von Aufgaben und Verantwortlichkeiten möglich machen. In der Aufbauorganisation gibt es mehrere Organisationsformen. Das Einlinien- und das Mehrliniensystem sowie die Stablinienorganisation werden im Folgenden erläutert.

In dem in Abbildung 17 gezeigten Einliniensystem ist die Weisungsbefugnis 1:n, was dazu führt, dass in jeder Hierarchieebene Vollkompetenz herrscht. Das bedeutet, jede Hierarchieebene, bis auf die oberste, hat genau eine übergeordnete Ebene. Die obere Ebene ist den untergeordneten Ebenen gegenüber weisungs- und entscheidungsbefugt. Die untergeordneten Ebenen haben gegenüber ihrer übergeordneten Ebene Vorschlagsrecht: Jeder

Mitarbeiter kann der nächsthöheren Ebene Vorschläge zur Veränderung und Verbesserung unterbreiten. Abbildung 17 zeigt ein Beispiel für ein Einliniensystem. Eine Linie von oben nach unten (z. B. Hauptabteilungsleiter → Abteilungsleiter → Teamleiter) heißt Dienstweg, dessen Einhaltung obligatorisch ist. Damit zwei Teamleiter miteinander arbeiten, ist die Einbeziehung des übergeordneten Abteilungsleiters verpflichtend.

Abbildung 17: Einliniensystem



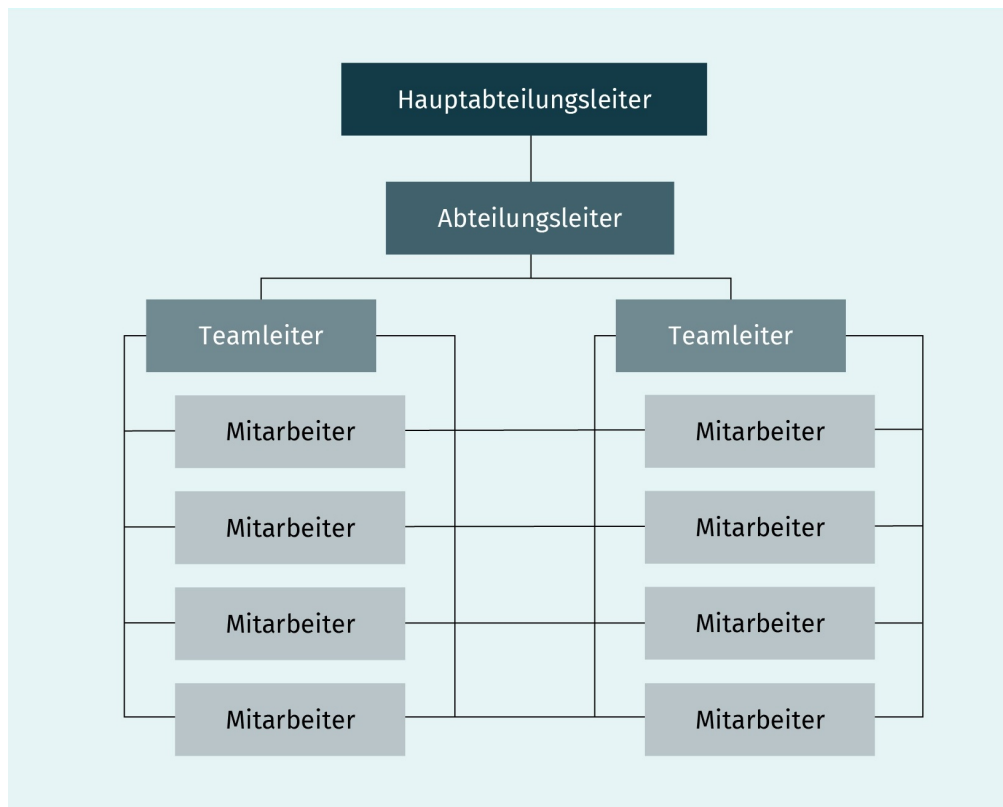
Quelle: erstellt im Auftrag derIU, 2023.

Die Vorteile des Einliniensystems liegen in der klaren Weisungsbefugnis und der eindeutigen Zuordnung von Stellen in der Hierarchie. Dies führt zu klaren Befugnissen und Verantwortungsbereichen. Demgegenüber stehen die langen Informationswege, da die Kommunikation zwischen gleichgestellten Leitungsebenen (z. B. Teamleiter) ausschließlich über die übergeordnete Ebene geschieht (den Abteilungsleiter) und so zu einer Überforderung übergeordneter Stellen führen kann. Die Überlastung Vorgesetzter ist sogar wahrscheinlich, da jede Entscheidung vom Vorgesetzten getroffen werden muss und dieser die Entscheidungskompetenz nicht delegieren kann.

Das Mehrliniensystem ist aufgebaut wie das Einliniensystem (vgl. Abbildung 18) mit dem Unterschied, dass gleichrangig Vorgesetzte auch teamübergreifende Weisungsbefugnis haben. Das Prinzip basiert darauf, dass die Vorgesetzten Fachexperten auf ihrem Gebiet sind und daher genau in ihrem Fachgebiet als Experten von Mitarbeitern konsultiert werden können. Der Teamleiter von Team A ist im Mehrliniensystem auch den Mitarbeitern

von Team B gegenüber weisungsbefugt bzw. die Mitarbeiter von Team B können sich auch an den Teamleiter von Team A wenden, ohne den Umweg über den eigenen Teamleiter und den Abteilungsleiter nehmen zu müssen.

Abbildung 18: Mehrliniensystem

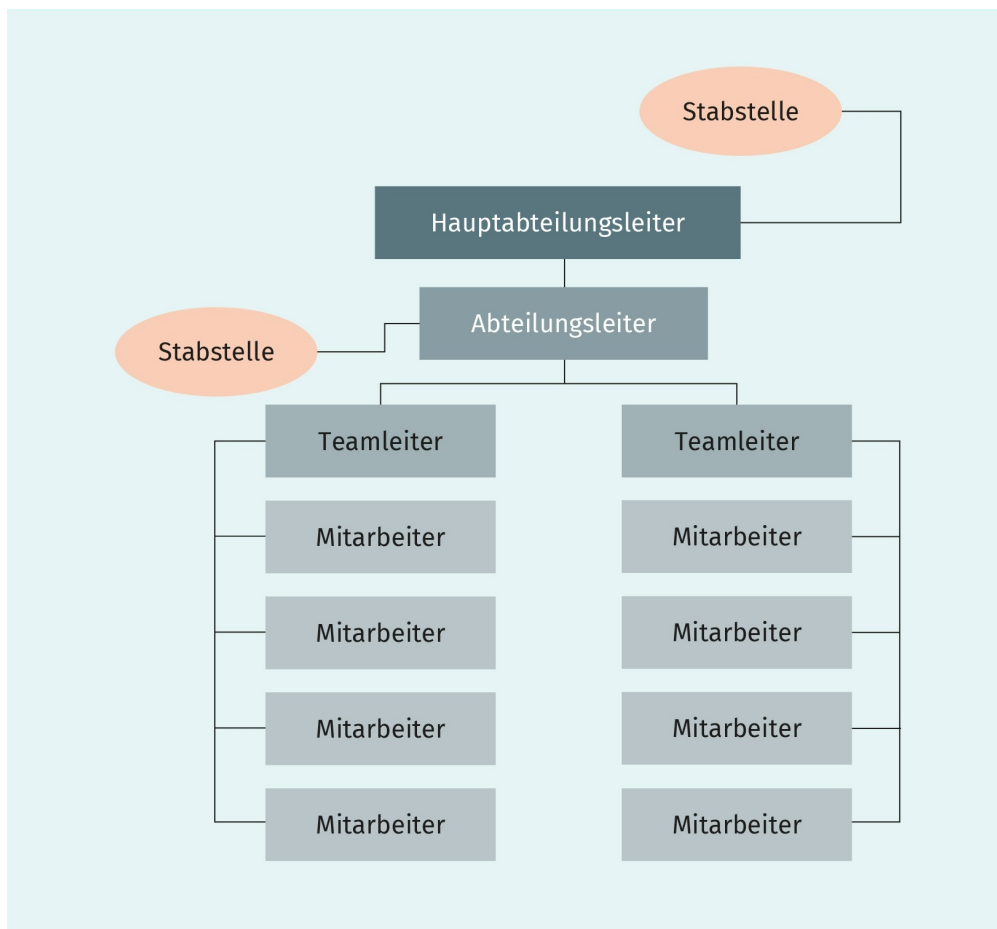


Quelle: erstellt im Auftrag der IU, 2023.

Das Mehrliniensystem führt zu kürzeren Kommunikationswegen und zu einer Spezialisierung der Leitung durch Funktionsverteilung. Durch direkte Kommunikationswege können sich Vorgesetzte mehr auf ihre Kernkompetenz konzentrieren, da Verwaltungsaufgaben, die im Einliniensystem zu erfüllen wären, wegfallen. Dies führt zu einer Betonung der Fachautorität. Es können jedoch auch Abgrenzungsprobleme der Zuständigkeiten und somit Kompetenzkonflikte entstehen.

Das Stabliniensystem ist ein um eine Stabstelle erweitertes Einliniensystem zur Entlastung der Linieninstanzen. Ein Stab ist ein Experte für bestimmte Gebiete und hat daher maximale fachliche Weisungsbefugnis. Dies bedeutet, dass der Stab beratend zur Seite steht, jedoch keine Arbeitsanweisungen vergibt (vgl. Abbildung 19).

Abbildung 19: Stabliniensystem



Quelle: erstellt im Auftrag der IU, 2023.

Die Vorteile sind die gleichen wie beim Einliniensystem, zuzüglich einer zunehmenden Entscheidungsqualität durch die Spezialisten. Die Nachteile sind jedoch eine Konzentration des spezialisierten Wissens in der Leitungsebene, was den autoritären Führungsstil verstärkt und die Gefahr einer selektiven Informationsweitergabe birgt. Darüber hinaus entstehen für Stabstellen zusätzliche Kosten. Projekte sind als temporäre Organisation häufig etwas anders organisiert. Die dauerhaft bestehenden Bereiche und Abteilungen im Unternehmen folgen in der Regel jedoch den hier gezeigten Organisationsmodellen.

Ablauforganisation

Die Ablauforganisation beschreibt den Ablauf innerhalb der Organisationsstruktur. Sie dokumentiert die Gestaltung der Arbeitsabläufe der Aufbauorganisation durch die Verkettung einzelner Arbeitsschritte unter Nutzung der Ressourcen (Stellen, Abteilungen, Rollen, Instanzen, Aufgaben) der Aufbauorganisation. Im Mittelpunkt stehen dabei die zielbezogene menschliche Handlung und die Ausstattung von Arbeitsprozessen mit Sachmitteln und Informationen (sogenannter Leistungserstellungsprozess).

Die Ziele der Ablauforganisation sind folgende:

- Die Auslastung der Leistungserstellung soll maximal sein.
- Bei maximaler Auslastung sollen Durchlaufzeiten so gering wie möglich sein. Gleiches gilt für Wartezeiten, in denen der Leistungserstellungsprozess stillsteht.
- Die Kosten der Leistungserstellung sollen so gering wie möglich sein.
- Die Qualität der Vorgangsbearbeitung und die Arbeitsbedingungen sollen verbessert werden.

Die Ablauforganisation ist durch Geschäftsprozesse definiert, deren Elemente im Folgenden erläutert werden.



HINWEIS

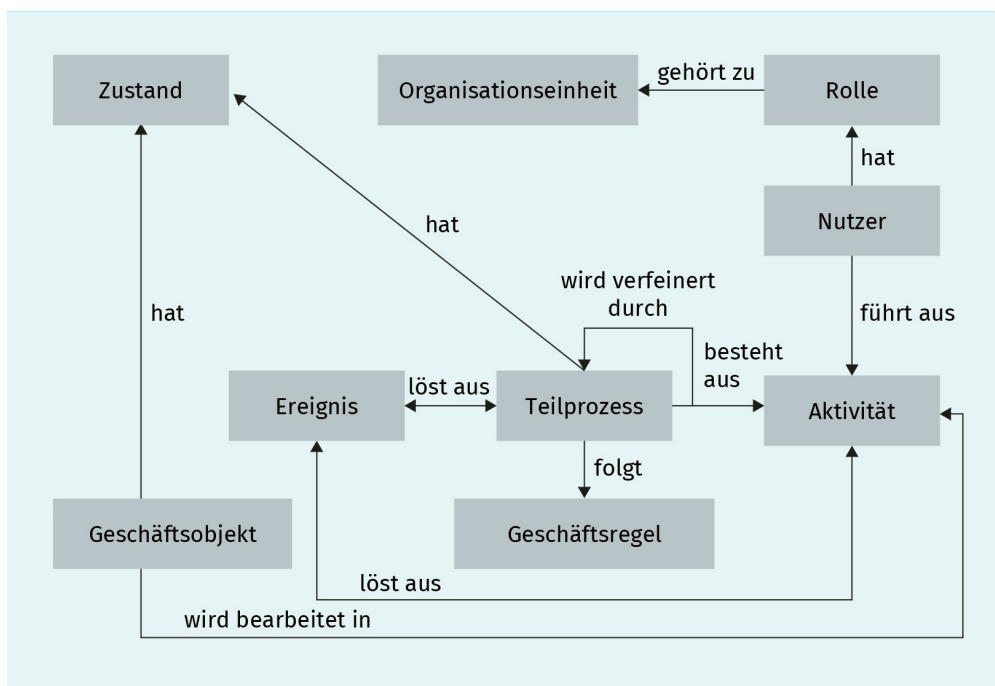
Auch die Erstellung, Veränderung und der Betrieb von IT-Systemen ist in der Regel in Form von Prozessen beschrieben.

Elemente in Geschäftsprozessen

Dem Verständnis, was ein Geschäftsprozess ist, soll zunächst eine allgemeine Definition dienen: „Ein Geschäftsprozess (GP) ist eine zielgerichtete, zeitlich-logische Abfolge von Aufgaben, die arbeitsteilig von mehreren Organisationen oder Organisationseinheiten unter Nutzung von IKT (Informations- und Kommunikationstechnologie) ausgeführt werden können. Er dient der Erstellung von Leistungen entsprechend den vorgegebenen, aus der Unternehmensstrategie abgeleiteten Prozesszielen. Der Geschäftsprozess kann formal auf unterschiedlichen Detaillierungsebenen aus mehreren Sichten beschrieben werden. Ein maximaler Detaillierungsgrad der Beschreibung ist dann erreicht, wenn die ausgewiesenen Aufgaben je in einem Zug von einem Mitarbeiter ohne Wechsel des Arbeitsplatzes ausgeführt werden können“ (Gadatsch 2022, S. 40f.).

In einem Geschäftsprozess wird also eine Reihe von Aktivitäten in einer bestimmten Reihenfolge unter Zuhilfenahme von IT durch mehrere Organisationseinheiten (Stelle, Rolle, Abteilung, Bereich, Organisation) bearbeitet. Abbildung 20 zeigt die Elemente von Geschäftsprozessen und deren Zusammenhänge, die im Folgenden erläutert werden. Ein Nutzer (z. B. Herr Klein) führt eine Aktivität (z. B. Schadenakte öffnen) in einer bestimmten Rolle (z. B. Schadensachbearbeiter) aus. Die Rolle gehört zu einer Organisationseinheit (z. B. Schaden- und Leistungsabteilung). Geschäftsprozesse bestehen aus einer Menge von Teilprozessen. Teilprozesse können durch Aktivitäten verfeinert werden, die einer oder mehreren Geschäftsregeln folgen (z. B. wenn die Schadenhöhe mehr als 100.000 € beträgt, erfolgt die Prüfung des Schadens durch einen Gutachter). Teilprozesse und Aktivitäten lösen Ereignisse (z. B. Schaden gemeldet) aus, genauso wie Ereignisse Teilprozesse auslösen können. Ein Teilprozess hat einen bestimmten Zustand (z. B. Schadenbearbeitung ausgeführt), genau wie Geschäftsobjekte (z. B. ein Schaden) einen bestimmten Zustand haben (z. B. Schaden gemeldet). Ein Geschäftsobjekt wird im Rahmen einer Aktivität bearbeitet.

Abbildung 20: Bestandteile und Zusammenhänge der Bestandteile eines Geschäftsprozesses



Quelle: erstellt im Auftrag der IU, 2023.

Tabelle 16 zeigt eine Übersicht über die Elemente von Geschäftsprozessen und gibt je Element ein Beispiel.

Tabelle 16: Elemente eines Geschäftsprozesses

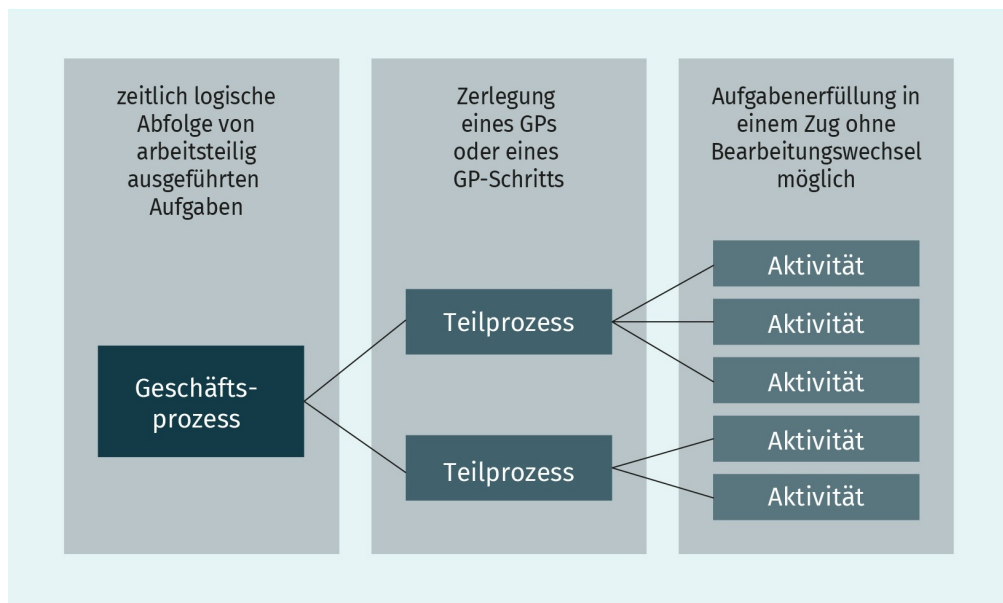
Element	Beschreibung	Beispiel
Prozess	Aktivitätsfolge mit möglichen Vorgängern und Nachfolgern	Antragsbearbeitung
Aktivität	ausführbare Einheit, die nicht sinnvoll weiter zerlegt werden kann	Adressbearbeitung
Geschäftsobjekt	materieller oder immaterieller Gegenstand	Antrag
Zustand	Status von Geschäftsobjekten, Prozessen	offener Antrag
Ereignis	Auslöser/Resultat für einen Prozess/eine Aktivität	Antragseingang
Rolle	Position mit bestimmten Aufgaben	Sachbearbeiter
Nutzer	Rolleninhaber mit bestimmtem Zweck im Prozess, Mensch oder Maschine	Karl Meier

Element	Beschreibung	Beispiel
Geschäftsregeln	Vorschriften, nach denen Prozesse ablaufen	Versicherungssumme > 250.000.000 € → separate Risikoprüfung durchführen
Organisationseinheit	Zuordnungsbereich von Kompetenz für einen oder mehrere Aufgabenträger	IT-Abteilung, IT-Sicherheitsbeauftragter

Quelle: erstellt im Auftrag der IU, 2023.

Wie bereits zuvor beschrieben, können Geschäftsprozesse durch (Teil-)Prozesse und Aktivitäten verfeinert werden (vgl. Abbildung 21). Dadurch entsteht eine Hierarchie, deren oberste Ebene Geschäftsprozess genannt wird. Ein Geschäftsprozess kann durch mehrere Teilprozesse verfeinert werden. Die Teilprozesse können in eine Menge von Aktivitäten zerlegt werden.

Abbildung 21: Strukturierung von Geschäftsprozessen



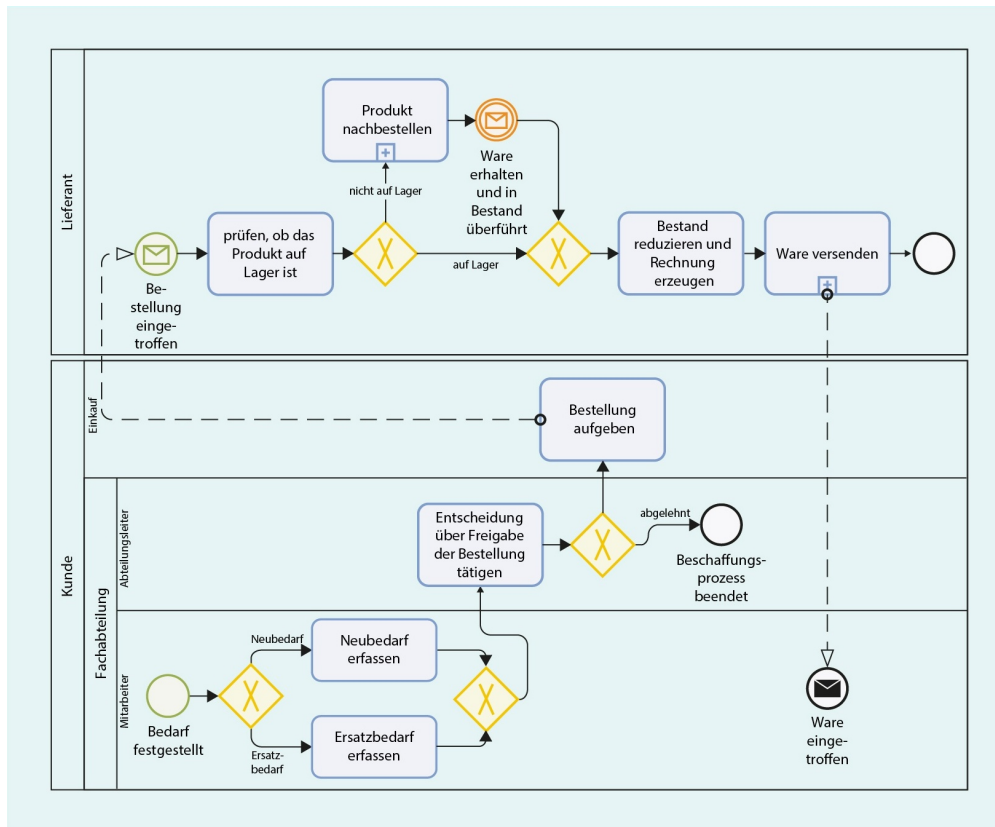
Quelle: erstellt im Auftrag der IU, 2023.

5.2 Modellierung mit der Business Process Model and Notation

Die Business Process Model and Notation (BPMN) ist eine Notation zur Modellierung von Geschäftsprozessen und wird von der Object Management Group (OMG) verwaltet und weiterentwickelt. Die OMG selber ist ein Konsortium, das aus über 800 Mitgliedsunternehmen besteht und herstellerneutrale Industriestandards erstellt und pflegt. Die OMG stellt mit der BPMN einen Modellierungsstandard bereit, der die Bedeutung grafischer Notati-

onselemente und deren Zusammenspiel definiert. Da die BPMN eine sehr ausdrucks-mächtige Sprache ist, würde eine vollständige Einführung den Umfang dieses Lernskripts sprengen.

Abbildung 22: Beispiel für einen Bestellprozess als BPMN-Modell



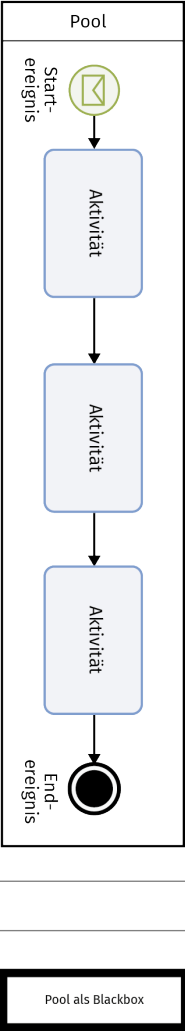
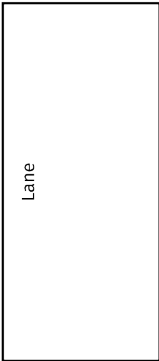
Quelle: erstellt im Auftrag der IU, 2023.

Abbildung 22 zeigt einen Bestellprozess, der in BPMN modelliert ist. Die Organisationseinheit Kunde besteht aus Fachabteilungen, die in die Rollen Mitarbeiter und Abteilungsleiter unterteilt sind. Ferner existiert beim Kunden die Rolle Einkauf. Der Geschäftsprozess beginnt damit, dass ein Bedarf beim Mitarbeiter aufgetreten ist. Ob es sich um einen Neu- oder Ersatzbedarf handelt, wird durch eine Verzweigung (sogenannter Gateway) dargestellt. Die Bestellung wird vom Einkauf nur dann aufgegeben, wenn der Abteilungsleiter die Bestellung bewilligt, andernfalls ist der Prozess beendet. Das Ereignis des Eintreffens der Bestellung beim Lieferanten löst eine Verfügbarkeitsprüfung der Lagerbestände für das bestellte Produkt aus. Bei Verfügbarkeit wird der Lagerbestand um die bestellte Anzahl reduziert, die entsprechende Rechnung erstellt und mit der Ware in den Versandprozess übergeben. Sollte die Ware nicht auf Lager sein, wird ein Wiederbeschaffungsprozess initiiert. Der Geschäftsprozess steht so lange, bis das Ereignis, dass die Ware eingetroffen und dem Bestand hinzugefügt ist, eintritt. Der Gesamtprozess ist beendet, wenn die Ware beim Mitarbeiter des Kunden angekommen ist.

Im Folgenden werden zunächst die wichtigsten Notationselemente tabellarisch vorgestellt. Die unterschiedlichen Verbindungstypen und Entscheidungsknoten, die sogenannten Gateways, werden separat erläutert.

Tabelle 17: Basisnotationselemente der BPMN ohne Gateways und Verbindungen

Name	Bedeutung	Darstellung
Aktivität	Aktivitäten repräsentieren Aufgaben, die im Prozess ausgeführt werden, und sind atomar, also nicht sinnvoll weiter zerlegbar. Aktivitäten werden aktiv formuliert und nach dem Pattern [Objekt] + [Verb] benannt (z. B. Antrag unterschreiben).	
Teilprozess	Diese Teilprozesse verfeinern Geschäftsprozesse und können durch ein weiteres BPMN-Diagramm verfeinert werden. Die Darstellung mit einem „+“ blendet den internen Ablauf im Teilprozess aus. Teilprozesse werden aktiv formuliert und nach dem Pattern [Objekt] + [Verb] benannt (z. B. Schaden bearbeiten, auf Klausur vorbereiten).	
Ereignisse	Ein Ereignis ist etwas, das im Verlauf eines Prozesses passiert und den Ablauf beeinflusst. Ereignisse haben in der Regel eine Ursache (trigger) und Auswirkungen (results). Die BPMN kennt drei Ereignistypen, die durch interne Marker weiter unterschieden werden können. Die drei Typen besitzen jeweils verschiedene Varianten (z. B. Nachrichten, Zeit, Bedingung, Signal, Fehler). Das Starterereignis kennzeichnet den Beginn eines Prozesses. Jeder (Teil-)Prozess besitzt zwingend mindestens ein Starterereignis. Das Zwischenereignis tritt im Prozessverlauf ein, das Endereignis beendet einen Prozess. Jedes Ereignis kann untypisiert sein, d. h., es besitzt keine interne Markierung. Solche Ereignisse werden auch Blanko-Ereignisse genannt. Interne Marker können Nachrichten oder Bedingungen sein. Ein Ereignis wird nach dem Muster [Objekt] und passiviertes [Verb] beschrieben (z. B. Schaden gemeldet).	 Blanko-Starterereignis  Blanko-Ereignis  Blanko-Endereignis  Getyptes Ereignis Nachricht eingetroffen
Datenobjekte	Datenobjekte werden zur Ausführung von Aktivitäten gebraucht oder erzeugt. Sie können einzelne Objekte (z. B. Schadenakte) oder ganze Sammlungen (z. B. Antragsdaten) davon repräsentieren. Datenobjekte werden mit dem sie umschließenden Prozess instanziiert und zerstört – sie besitzen keine Persistenz über den Prozess hinaus. Es gibt auch Datenspeicher, die beispielsweise Datenbanken darstellen können (siehe hierzu Allweyer 2020).	 Datenobjekt

Name	Bedeutung	Darstellung
Pools	Pools repräsentieren die Teilnehmer und Verantwortlichen eines Prozesses (z. B. der Helpdesk). Dies können Organisationen, Personen, Rollen oder Systeme sein. Ein Pool dient dazu, Prozesse zu partitionieren. Pools können auch als Black Boxes (unter Ausblendung innerer Abläufe) dargestellt werden. Jeder Pool kennzeichnet einen Teilprozess und benötigt daher ein Start- und ein Endereignis. Pools werden eingesetzt, um den Wechsel der Verantwortlichkeit in einem Geschäftsprozess zu modellieren.	 The diagram illustrates the representation of a pool in a process model. The top part shows a pool container labeled 'Pool' at the top. Inside the pool, the flow starts with a 'Start-ereignis' (start event) represented by a green circle with a lightning bolt. This is followed by three sequential 'Aktivität' (activity) boxes, each represented by a light blue rounded rectangle. The flow ends with an 'End-ereignis' (end event) represented by a thick black circle. The bottom part shows a 'Pool als Blackbox' (pool as black box) represented by a simple black rectangle.
Lanes	Eine Lane (Schwimmbahn) weist Aufgabenträgern Zuständigkeiten für Aufgaben zu und befindet sich immer innerhalb eines bestimmten Pools. Aufgabenträger können beispielsweise Personen (z. B. Heinz Müller), Rollen (z. B. Sachbearbeiter), Anwendungen (z. B. Schadenmanagementsystem) sein. Lanes können beliebig verschachtelt werden, um eine Verfeinerung der Zuständigkeiten darzustellen. Ein Flussobjekt (Aktivität, Ereignis) darf immer nur in genau einer Lane positioniert sein.	 The diagram shows a 'Lane' represented by a vertical rectangle. The word 'Lane' is written vertically inside the rectangle.

Name	Bedeutung	Darstellung
Nachrichten	Nachrichten symbolisieren den Inhalt einer Kommunikation und werden entweder an Nachrichtenflüsse assoziiert oder sie sind Bestandteil eines Ereignisses.	 gesendete Nachricht  empfangene Nachricht  Nachricht gesendet  Nachricht empfangen
Annotationen	Annotationen ermöglichen weiterführende Angaben, Kommentare oder Notizen und können an jedes Flussobjekt geheftet werden.	 Anmerkung  Kommentar  Annotation

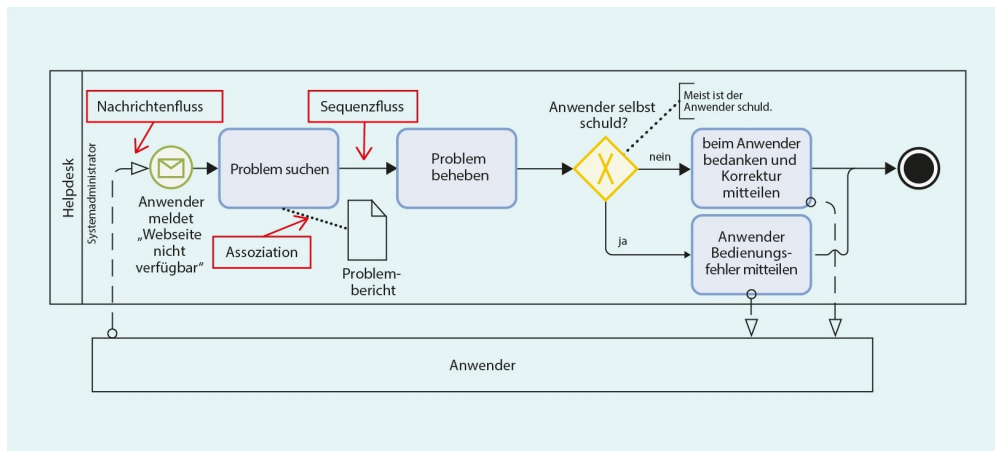
Quelle: erstellt im Auftrag der IU, 2023.

Verbindungen

Die Verbindungstypen der BPMN heißen Sequenzfluss, Nachrichtenfluss und Assoziation. Der Sequenzfluss gibt die Reihenfolge der Aktivitäten vor und kann zwar über Lane-Grenzen, jedoch nicht über Pool-Grenzen hinweg gehen. Der Nachrichtenfluss verbindet Pools miteinander, deren Kommunikation über Nachrichten läuft. Die Assoziation verbindet Datenobjekte und Annotationen mit Flussobjekten.

In Abbildung 23 wird ein Geschäftsprozess mit einer Interaktion zwischen der Rolle Anwender und der Rolle Systemadministrator als Bestandteil der Organisationseinheit Helpdesk dargestellt. Von den internen Abläufen der Rolle Anwender wird abstrahiert, deswegen ist diese als zugeklappter Pool (Black Box) modelliert. Der Prozess in der Organisationseinheit Helpdesk (modelliert als Pool) beginnt, indem bei einem Systemadministrator (modelliert als Lane) die Nachricht „Webseite nicht verfügbar“ eintrifft. Die Nachricht wird über einen Nachrichtenfluss vom Anwender zum Helpdesk übermittelt. Die Kommunikation zwischen Pools darf nur über Nachrichtenflüsse geschehen. Der Systemadministrator führt die Aktivität „Problem suchen“ aus und nutzt dazu einen „Problembericht“, der als Datenobjekt dargestellt ist. Außerdem behebt er in der folgenden Aktivität das Problem und muss entscheiden, ob der Anwender die Ursache für das Problem ist. Als Kommentar ist an die Entscheidung notiert, dass dies häufig der Fall ist. Weitere Informationen zu Entscheidungsknoten, den sogenannten Gateways, folgen noch. Ist der Anwender schuld, wird ihm der Bedienfehler über eine Nachricht mitgeteilt, sodass er daraufhin das Problem selbst lösen kann. Ist der Anwender nicht schuld, wird das Problem vom Systemadministrator behoben, der sich beim Anwender mit einer Nachricht bedankt. Nachrichten müssen nicht explizit modelliert werden. Egal ob der Anwender schuld ist oder nicht, endet danach der Prozess, was durch ein Endereignis dargestellt wird.

Abbildung 23: Beispiel für ein BPMN-Modell

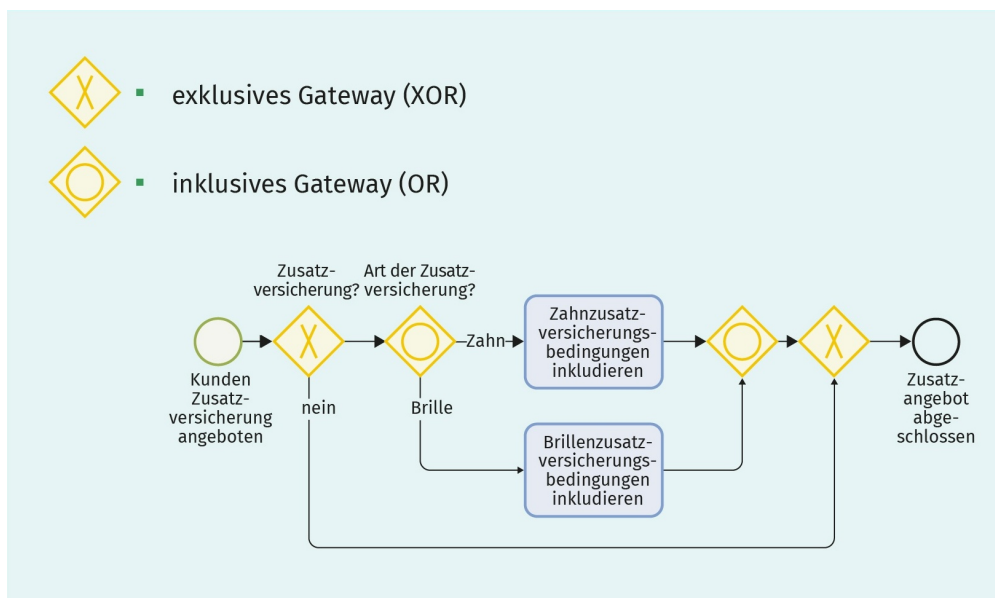


Quelle: erstellt im Auftrag der IU, 2023.

Gateways

Gateways leiten Verzweigungen im Sequenzfluss eines Diagramms ein und beenden Verzweigungen. Die interne Markierung des Gateways legt dessen Bedeutung fest. Wir unterscheiden zwischen exklusiven, inklusiven, parallelen und ereignisbasierten Gateways. Exklusive Verzweigungen teilen den Sequenzfluss in echte Alternativen (exklusives ODER); entweder wird der eine oder der andere Sequenzfluss ausgeführt, jedoch niemals mehr als einer. Inklusive Gateways teilen den Sequenzfluss ebenso in Alternativen mit dem Unterschied, dass auch mehrere Flüsse ausgeführt werden können (logisches ODER).

Abbildung 24: BPMN - XOR und OR Gateway

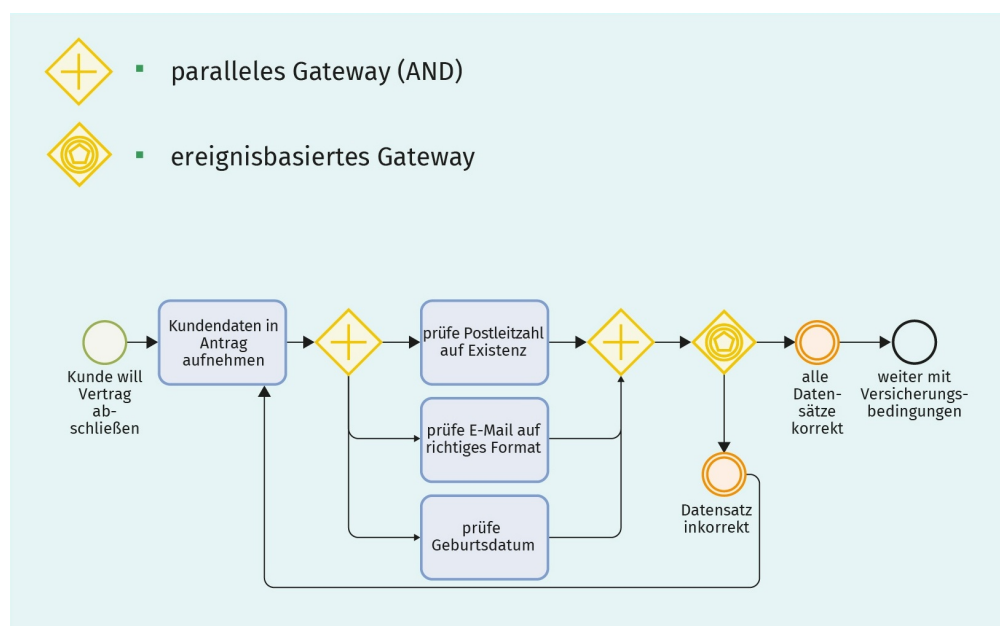


Quelle: erstellt im Auftrag der IU, 2023.

Abbildung 24 zeigt ein einfaches Beispiel für die Verwendung von exklusiven und inklusiven Entscheidungen. Die Entscheidung, ob eine Zusatzversicherung abgeschlossen wird, kann entweder mit ja oder mit nein beantwortet werden. Wenn entschieden wurde, dass eine Zusatzversicherung abgeschlossen werden soll, kann entweder eine Zahn- oder eine Brillenzusatzversicherung abgeschlossen werden oder beide.

Das parallele Gateway führt zu einer Aufspaltung des Sequenzflusses in mehrere gleichzeitig ausgeführte Sequenzflüsse. Ein ereignisbasiertes Gateway steuert den Sequenzfluss in Abhängigkeit vom Eintritt eines bestimmten Ereignisses (z. B. dem Eintreffen einer Bestellung). Abbildung 25 zeigt die Verwendung von parallelen und ereignisbasierten Gateways. Die dargestellten Prüfaktivitäten können parallel abgearbeitet werden. Sobald alle Prüfungen abgeschlossen sind, wird durch das Ereignis „Prüfergebnis fertig“ entschieden, wie der Ablauf weitergeht. Sind alle Daten korrekt, ist der Prozess abgeschlossen. Falls der Datensatz inkorrekt ist, muss die Aktivität „Kundendaten in Antrag aufnehmen“ erneut ausgeführt werden. Der Eingang zweier Sequenzflüsse in eine Aktivität ist in der BPMN erlaubt, ohne dass dies etwas Spezielles bedeutet. Entweder führt der eine Sequenzfluss zur Ausführung der Aktivität oder der andere. Dies ist nicht in allen Notationen so, wie z. B. in UML-Aktivitätsdiagrammen. Daher ist es empfehlenswert, immer zusammenführende Gateways zu modellieren, sodass beim Wechsel der Notation nicht versehentlich Fehler unterlaufen.

Abbildung 25: BPMN - AND und event-driven Gateway

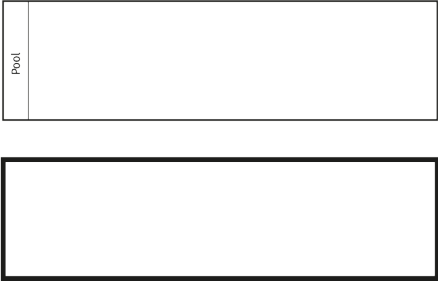


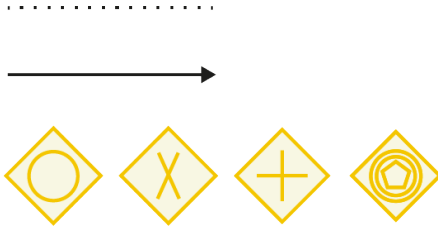
Quelle: erstellt im Auftrag der IU, 2023.

Übersicht über die BPMN-Elemente

Tabelle 18 ordnet die Elemente von Geschäftsprozessen den Notationselementen der BPMN zu.

Tabelle 18: Zuordnung BPMN zu Geschäftselementen

Element	Beispiel	Symbol(e) BPMN
Prozess	Antragsbearbeitung	
Aktivität	Adressprüfung	
Geschäftsobjekt	Antrag	
Zustand	Schaden gemeldet Schaden reguliert	
Ereignis	Antragseingang	
Rolle	Sachbearbeiter	
Nutzer	Karl Meier	

Element	Beispiel	Symbol(e) BPMN
Geschäftsregeln	Versicherungssumme > 250.000.000 € → separate Risikoprüfung durchführen	
Organisationseinheit	IT-Abteilung, IT-Sicherheitsbeauftragter	
Kontrollfluss und -steuerung	Sequenzfluss, Kontrollfluss, Parallelisierung, Entscheidung	
System	Antragssystem	
Kommentare und Notizen	Gruppierung, Annotation, Kommentar	

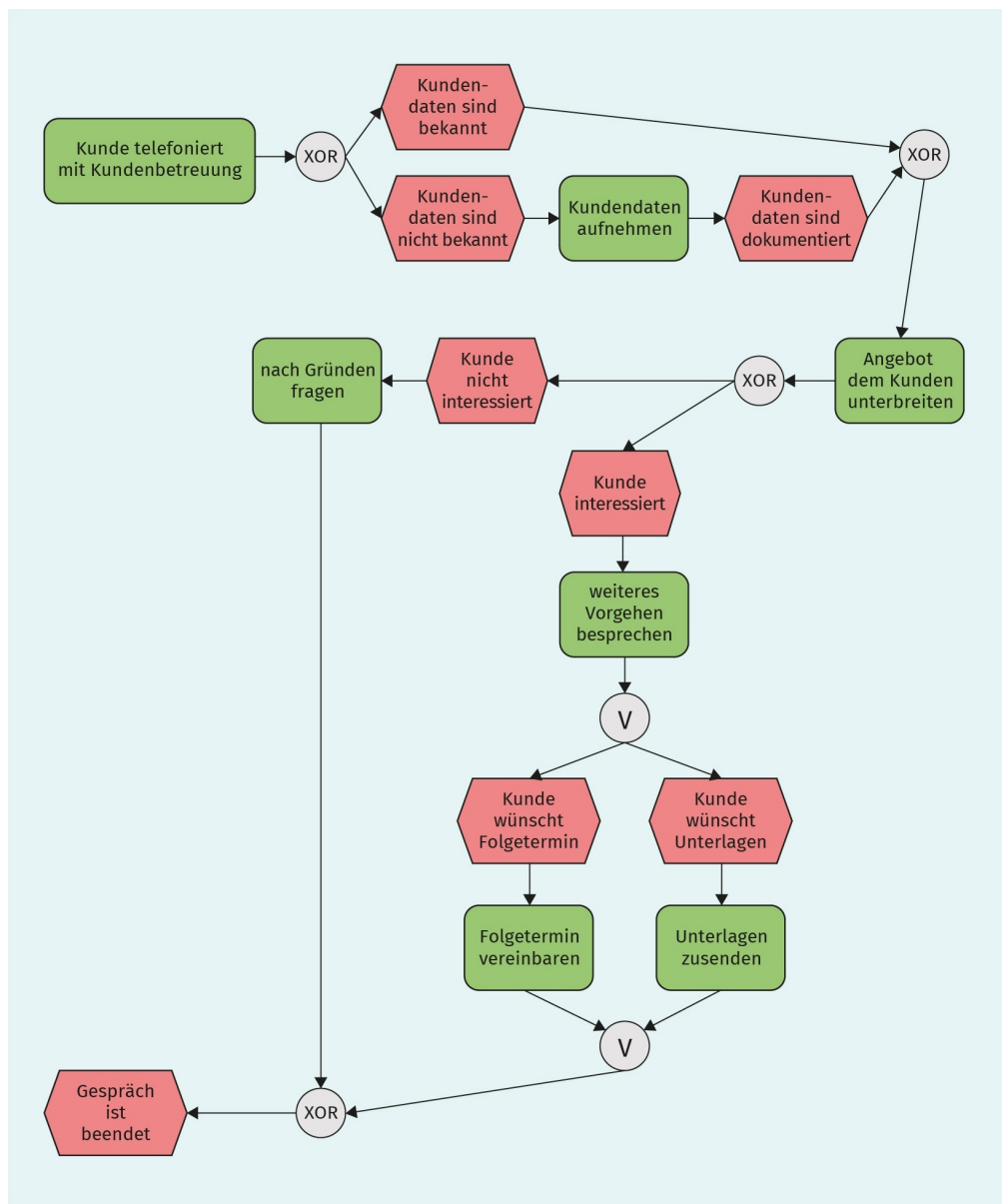
Quelle: erstellt im Auftrag der IU, 2023.

Zu Beginn des RE-Prozesses kann eine vollumfängliche Nutzung der BPMN hinderlich sein, denn die Stakeholder müssen die Anforderungen verstehen, um deren Korrektheit und Angemessenheit beurteilen zu können. Je weiter der RE-Prozess fortschreitet, desto konkreter und stabiler werden die Anforderungen und desto sinnvoller kann ein Einsatz weiterer Notationselemente der BPMN sein. Eine gute Übersicht über alle Notationselemente liefert ein kostenfrei online abrufbares Poster (BPMN 2011).

5.3 Modellierung mit Ereignisgesteuerten Prozessketten

Ereignisgesteuerte Prozessketten (EPK) sind im deutschsprachigen Raum weit verbreitet. Sie haben ihren Ursprung im Saarland. Dort wurden sie 1992 unter der Leitung von August Wilhelm Scheer von seiner Arbeitsgruppe an der Universität des Saarlandes entwickelt. Die EPK wird, genau wie die BPMN, eingesetzt, um betriebliche Abläufe zu modellieren. Erweiterte Ereignisgesteuerte Prozessketten (eEPK) erhöhen durch zusätzliche Notationselemente die Ausdrucksmächtigkeit der EPK.

Abbildung 26: EPK-Beispiel



Quelle: erstellt im Auftrag der IU, 2023.

Abbildung 26 zeigt folgenden Sachverhalt: Ein potenzieller Kunde telefoniert mit der Kundenbetreuung. Sind die Kundendaten nicht bekannt, werden diese zunächst dokumentiert. Sind die Kundendaten bekannt, wird ein Angebot unterbreitet. Ist der Kunde nicht am Angebot interessiert, werden die Gründe dazu erfragt. Dann ist das Gespräch beendet. Ist der Kunde hingegen interessiert, wird das weitere Vorgehen besprochen. Zur Auswahl stehen die Vereinbarung eines Folgetermins und das Versenden von Unterlagen. Beide Optionen können einzeln oder gemeinsam gewählt werden. Je nach Entscheidung wird

der Termin vereinbart und/oder die Unterlagen versendet. Anschließend ist das Gespräch zu Ende. An diesem Beispiel fällt eine Besonderheit der EPK auf. Funktionen und Ereignisse wechseln sich immer ab, dies ist von der Notation vorgeschrieben.

Wichtige Notationselemente

Tabelle 19: Basisnotationselemente der EPK ohne Konnektoren

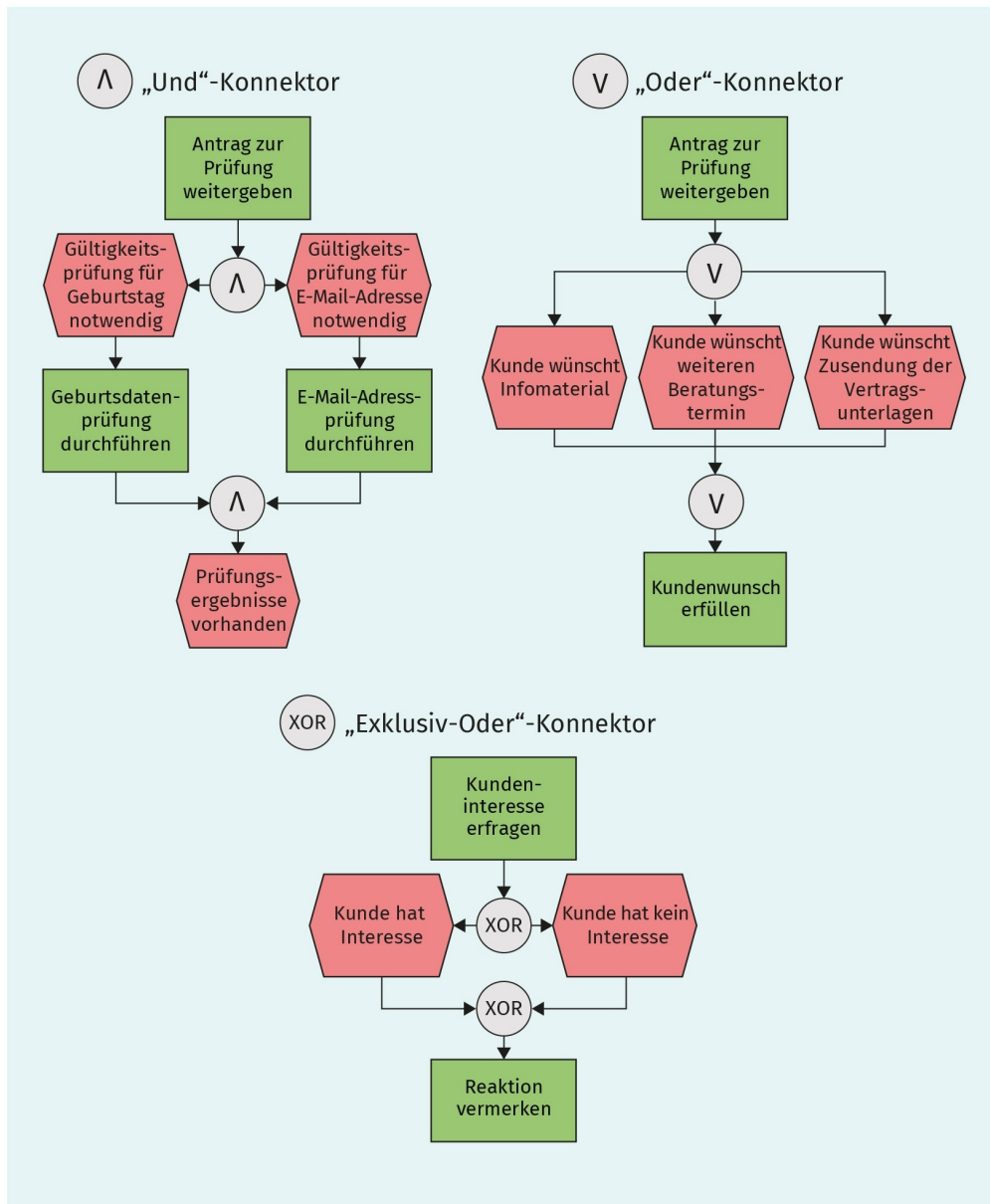
Name	Bedeutung	Darstellung
Ereignis	Ein Ereignis ist etwas, das im Verlauf eines Prozesses passiert und den Ablauf beeinflusst. Ereignisse haben in der Regel eine Ursache (trigger) und Auswirkungen (results). Es ist jedoch selber passiv und trifft keine Entscheidungen. Das heißt, Ereignisse verbrauchen weder Ressourcen noch Zeit. Jeder Geschäftsprozess hat mindestens ein Start- und ein Endereignis. Mehrere Startereignisse treten insbesondere in Teilprozessen auf. Beispiel: Ein Kundenkontakt kann die unterschiedlichsten Gründe für das Zustandekommen besitzen und genauso mit vielen differenzierten Ereignissen enden. Ein Ereignis wird nach dem Muster [Objekt] und passiviertes [Verb] beschrieben (z. B. Schaden gemeldet).	
Funktion	Eine Funktion ist eine fachliche Tätigkeit oder Aufgabe. Sie nimmt meist Zeit in Anspruch und kann Ressourcen verbrauchen. Sie kann über den weiteren Verlauf des Prozesses entscheiden, ist also aktiv und sollte auch so benannt werden. Die Funktion ist der Aktivität in BPMN gleichzusetzen. Funktionen werden aktiv formuliert und nach dem Pattern [Objekt] + [Verb] benannt (z. B. Antrag unterschreiben).	
Kontrollfluss	Kontrollflusskanten bringen Funktionen und Ereignisse in zeitliche und logische Abfolgen. Ereignisse und Funktionen folgen einander dabei streng abwechselnd, d. h., das Element nach einem Ereignis muss immer eine Funktion sein und umgekehrt. Jedes Element der Notation ist durch eine Kante verbunden. Es gibt keine losgelösten Ereignisse oder Funktionen; EPK sind immer zusammenhängende Graphen. Eine Kante verbindet immer genau zwei Elemente miteinander, d. h., ein Element hat nie zwei ausgehende Kanten.	

Quelle: erstellt im Auftrag der IU, 2023.

Konnektoren

Analog zu den Gateways der BPMN gibt es in EPK Konnektoren. Diese Konnektoren leiten Verzweigungen ein (sogenannter Split) oder beenden sie (sogenannter Join). Ein Konnektor ist entweder ein Split oder ein Join, jedoch nie beides gleichzeitig. Konnektoren sind die einzigen Symbole, die Kontrollflüsse verzweigen oder zusammenführen. Ereignisse und Funktionen haben nie mehr als einen Ein- und einen Ausgang. Abbildung 27 zeigt die drei unterschiedlichen Konnektoren. „Und“-Konnektoren umschließen Teilprozesse, die gleichzeitig bzw. unabhängig voneinander ausgeführt werden. „Oder“-Konnektoren umschließen alternative Pfade, wobei mehrere Alternativen gleichzeitig ausgeführt werden können. „Exklusiv-Oder“-Konnektoren umschließen Alternativen, von denen jeweils nur eine ausgeführt werden darf.

Abbildung 27: EPK - Konnektoren



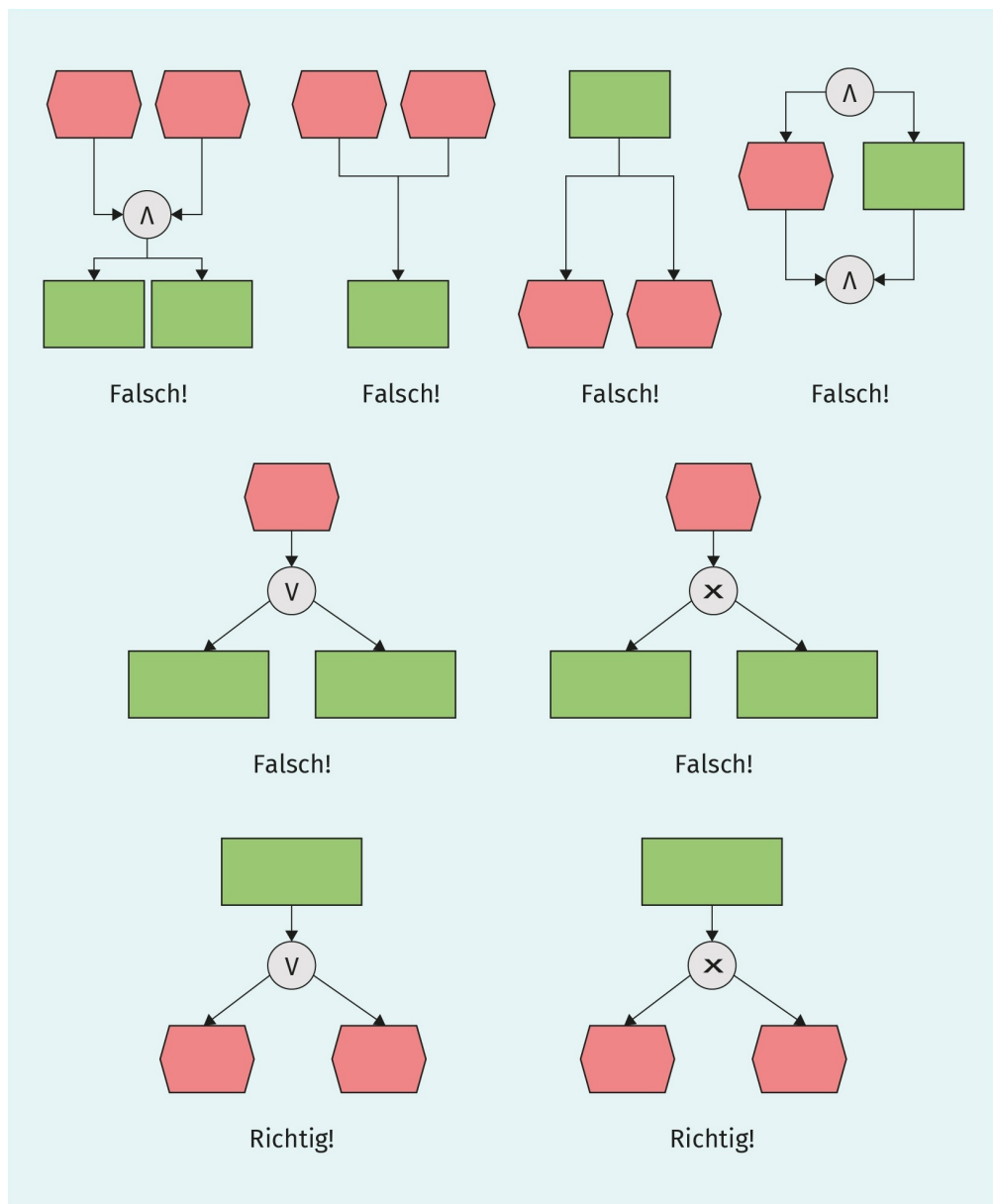
Quelle: erstellt im Auftrag der IU, 2023.

Wichtige Regeln zur Modellierung

Abbildung 28 zeigt Negativbeispiele, die im Folgenden von links nach rechts erläutert werden. In der ersten EPK werden zwei Ereignisse mit einem Und-Konnektor verbunden. Der Join wird jedoch gleichzeitig wieder als Split benutzt, was in der EPK nicht zulässig ist. Auch die beiden Kontrollflüsse von den zwei Ereignissen zu der Funktion in der zweiten EPK sind nicht zulässig, genau wie der umgekehrte Fall in der dritten EPK. Die EPK ganz rechts zeigt eine parallele Ausführung, die durch einen Und-Konnektor dargestellt ist. Der Diagrammausschnitt verstößt gegen die Regel, dass Ereignis und Funktion streng abwechseln.

selnd aufeinanderfolgen müssen, da vor dem verzweigenden Konnektor entweder ein Ereignis oder eine Funktion modelliert sein muss. Gleiches gilt für das Notationselement nach dem zusammenführenden Konnektor. Außerdem darf auf ein Ereignis keine Entscheidung folgen. Wie unten in Abbildung 28 dargestellt ist, muss einer Entscheidung eine Funktion vorangestellt sein.

Abbildung 28: Regeln zur Modellierung

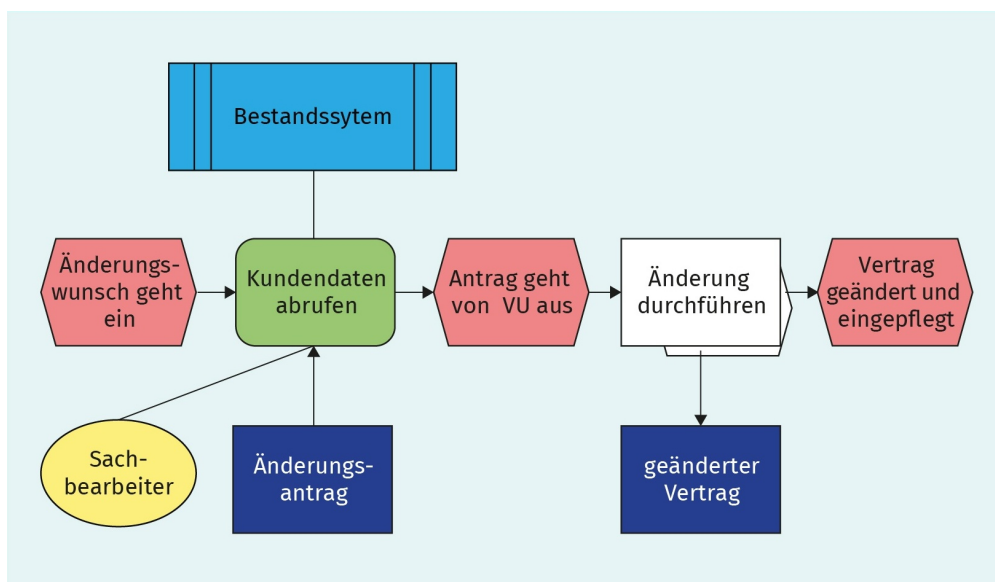


Quelle: erstellt im Auftrag der IU, 2023.

Erweiterte EPK

Abbildung 29 zeigt die Notationselemente der erweiterten Ereignisgesteuerten Prozessketten, die der Notation mehr Ausdrucksmöglichkeit verleihen. Das gelbe Oval kennzeichnet eine Organisationseinheit und ist den Pools bzw. Lanes der BPMN gleichzusetzen. Anwendungssysteme werden in einem hellblauen Rechteck mit doppelten vertikalen Strichen an den Seiten dargestellt. Informationsobjekte werden im dunkelblauen Rechteck dargestellt und können Datenobjekte bzw. eine Sammlung von Datenobjekten oder sonstigen Datenspeichern darstellen. Mit Informationsobjekten können also auch persistente Datenspeicher modelliert werden, was sie von den Datenobjekten der BPMN unterscheidet. Prozesswegweiser können als Hinweise auf andere Prozesse verstanden werden. Sie werden durch ein Rechteck symbolisiert, hinter dem sich ein Sechseck verbirgt. Mit Prozesswegweiser ist es also möglich, andere Prozesse zu referenzieren und Prozesshierarchien zu bilden.

Abbildung 29: eEPK - Beispiel

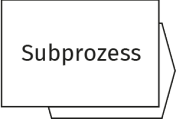
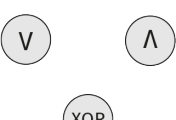


Quelle: erstellt im Auftrag der IU, 2023.

Übersicht über EPK

Tabelle 20 ordnet die Elemente von Geschäftsprozessen den Notationselementen der eEPK zu.

Tabelle 20: Zuordnung (e)EPK zu Geschäftselementen

Element, Beschreibung	Beispiel (Text)	Symbol(e) eEPK
Prozess	Antragsbearbeitung	
Aktivität	Adressprüfung	
Geschäftsobjekt	Antrag	
Zustand	Schaden gemeldet Schaden reguliert Schaden abgelehnt	
Ereignis	Antragseingang	
Rolle	Sachbearbeiter	
Nutzer	Karl Meier	
Geschäftsregeln	Versicherungssumme > 250.000.000 € → separate Risikoprüfung durchführen	
Organisationseinheit	IT-Abteilung, IT-Sicherheitsbeauftragter	

Element, Beschreibung	Beispiel (Text)	Symbol(e) eEPK
Kontrollfluss und -steuerung	Sequenzfluss, Kontrollfluss, Parallelisierung, Entscheidung	
System	Antragssystem	
Kommentare und Notizen	Gruppierung, Annotation, Kommentar	

Quelle: erstellt im Auftrag der IU, 2023.



ZUSAMMENFASSUNG

In der Unternehmensmodellierung wird eine Bestandsaufnahme durch das Erstellen einer Ist-Übersicht über die Organisation von Unternehmen durch die Darstellung von Abläufen (Geschäftsprozessen) und Strukturen (Aufbauorganisation) erzeugt. Die Aufbauorganisation bildet die Struktur einer Organisation ab. Diese Struktur ist in den meisten Fällen hierarchisch aufgebaut und legt die Rahmenbedingungen für die Bearbeitung von Aufgaben in einem Unternehmen fest. Die Ablauforganisation steht im Abhängigkeitsverhältnis zur Aufbauorganisation, denn sie behandelt die gleichen Objekte, jedoch aus einer anderen Perspektive. Die Ablauforganisation drückt aus, wie Arbeitsabläufe (Geschäftsprozesse) der Aufbauorganisation durch die Verkettung einzelner Arbeitsschritte unter Nutzung der Ressourcen (Stellen, Abteilungen, Rollen, Instanzen, Aufgaben) der Aufbauorganisation gestaltet sind.

Ein Geschäftsprozess ist eine Reihe von Aktivitäten in einer bestimmten Reihenfolge, die ggf. unter Zuhilfenahme von IT durch mehrere Organisationseinheiten (Stelle, Rolle, Abteilung, Bereich, Organisation) bearbeitet wird. Elemente eines Geschäftsprozesses sind Nutzer, Aktivitäten, Rollen und Organisationseinheiten. Ein Prozess kann durch andere (Teil-)Prozesse und Aktivitäten verfeinert werden, die einer oder mehreren Geschäftsregeln folgen. Prozesse und Aktivitäten lösen Ereignisse aus und umgekehrt. Ein Prozess hat, genau wie Geschäftsobjekte, einen bestimmten Zustand.

Um Abläufe zu modellieren, kann die Business Process Model and Notation (BPMN) verwendet werden. Außerdem kann die Ablauforganisation eines Unternehmens mit Ereignisgesteuerten Prozessketten (EPK) modelliert werden. Die EPK wird, genau wie die BPMN, eingesetzt, um

betriebliche Abläufe zu modellieren. Erweiterte Ereignisgesteuerte Prozessketten (eEPK) erhöhen durch zusätzliche Notationselemente die Ausdrucksmächtigkeit der EPK.

LEKTION 6

MODELLIERUNG VON SYSTEMEN

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

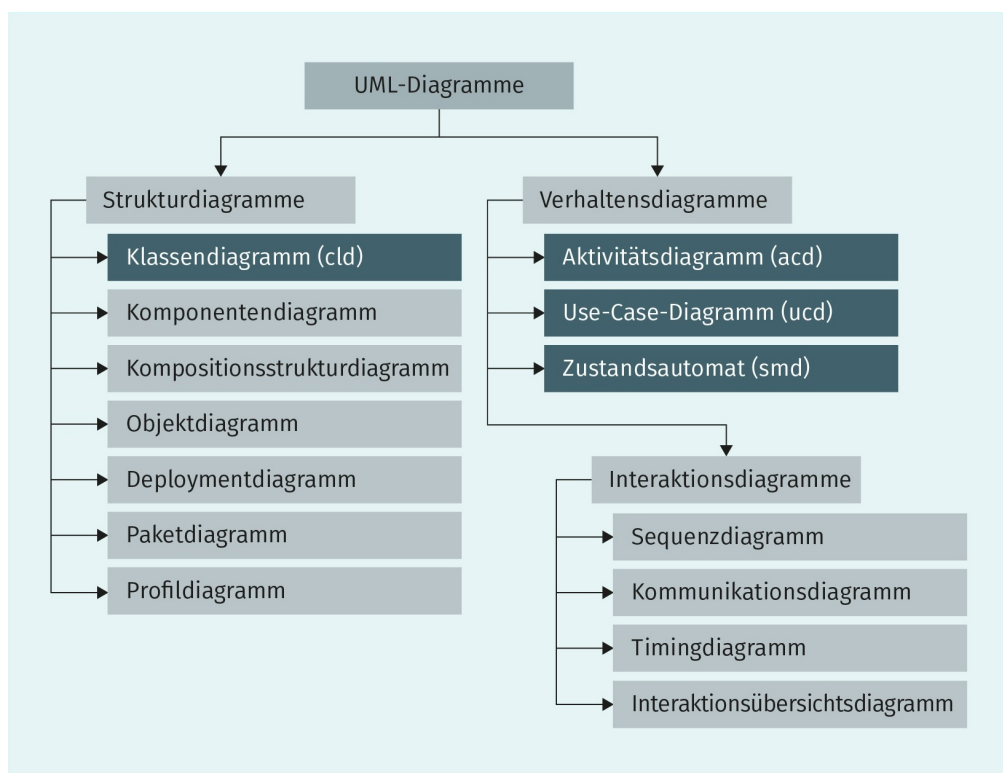
- was die Begriffe UML-Modell und UML-Diagramm bedeuten und wie diese zusammenhängen.
- wozu UML-Use-Case-Diagramme im Requirements Engineering eingesetzt werden und was deren wichtigste Notationselemente sind.
- wozu UML-Aktivitätsdiagramme im Requirements Engineering eingesetzt werden und was deren wichtigste Notationselemente sind.
- wie UML-Klassendiagramme im Requirements Engineering eingesetzt werden und was deren wichtigste Notationselemente in diesem Zusammenhang sind.
- wozu UML-Zustandsdiagramme im Requirements Engineering eingesetzt werden und was deren wichtigste Notationselemente sind.

6. MODELLIERUNG VON SYSTEMEN

6.1 Grundlagen Unified Modeling Language

Die Unified Modeling Language (kurz: UML) ist eine grafische Modellierungssprache, die etwa zeitgleich mit der Objektorientierung in den frühen 1990er-Jahren entwickelt wurde. Die UML umfasst mittlerweile 14 verschiedene Diagrammtypen, mit denen verschiedene Aspekte eines Systems modelliert werden können. Abbildung 30 gibt einen Überblick über die UML-Diagramme. Grundsätzlich lassen sich Diagramme zur Modellierung von Struktur und von Verhalten unterscheiden. Mit Strukturdiagrammen werden Aufbau, Elemente und Zusammensetzung sowie Schnittstellen von Systemen dargestellt. Vereinfacht gesagt werden Strukturdiagramme eingesetzt, um zu modellieren, woraus ein System besteht. Verhaltensdiagramme hingegen kommen zum Einsatz, wenn dargestellt werden soll, was in einem System und an seinen Schnittstellen abläuft.

Abbildung 30: Überblick Unified Modeling Language



Quelle: erstellt im Auftrag der IU, 2023.

Die UML ist weltweit verbreitet und wird sowohl in der Industrie als auch in der Forschung zur Modellierung, Dokumentation, Spezifikation und Visualisierung von komplexen Softwaresystemen eingesetzt. Grundsätzlich können die Diagramme der UML in allen Phasen des Softwareentwicklungsprozesses eingesetzt werden, von frühen Phasen der Anforderungsermittlung bis hin zur detaillierten technischen Spezifikation und der Dokumentation des implementierten Systems.

Der Einsatz der UML ist unabhängig davon, in welchem Fachgebiet oder welcher Branche das System eingesetzt werden soll. Die Unified Modeling Language ist ein von der Object Management Group (OMG) veröffentlichter und gepflegter Industriestandard. Die OMG stellt mit der UML einen Modellierungsstandard bereit, der die Bedeutung grafischer Notationselemente und deren Zusammenspiel definiert. Die UML ist jedoch weder eine Programmiersprache noch ein Vorgehensmodell. Unter dem Namen UML wurden verschiedene Modellierungskonzepte der Softwaretechnik zusammengefasst und deren Begriffe, Namen sowie Notationselemente vereinheitlicht.

Zusammenhang von UML-Modell und UML-Diagrammen

Ein UML-Diagramm stellt immer eine ganz bestimmte Sicht auf ein System oder einen Sachverhalt dar. So werden beispielsweise mit einem Klassendiagramm Geschäftsobjekte und deren Eigenschaften modelliert, während mit einem Aktivitätsdiagramm interne Abläufe des Systems modelliert werden. Beide Diagramme sind jedoch Bestandteil eines UML-Modells. Das bedeutet, wie in Abbildung 31 dargestellt, dass ein UML-Modell aus einer Menge von UML-Diagrammen besteht. Ein UML-Diagramm gehört jedoch immer zu genau einem UML-Modell.

Abbildung 31: Beziehung UML-Modell und UML-Diagramm



Quelle: erstellt im Auftrag der IU, 2023.

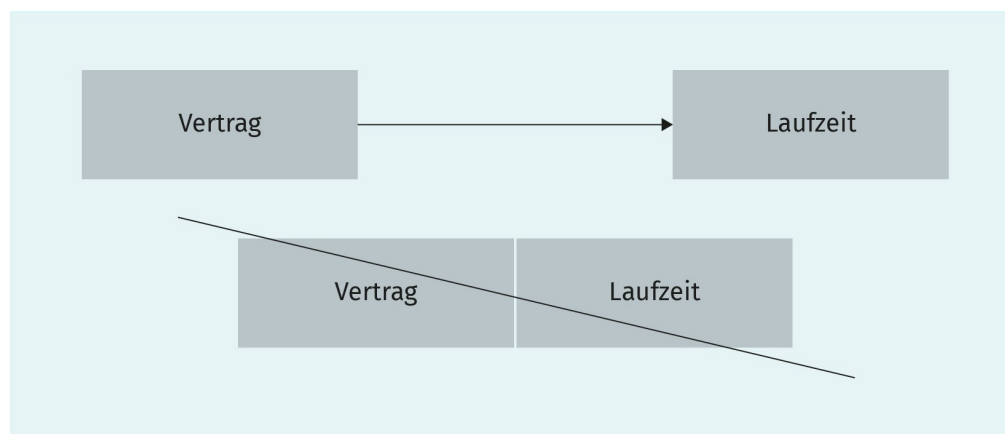
Die Menge der Informationen, die in einem UML-Modell vorhanden ist, ergibt sich aus der Menge aller Diagramme, die zu dem Modell gehören. Es gibt daher keine Information zu einem UML-Modell, die nicht in mindestens einem Diagramm enthalten ist. Modellelemente können auch in mehreren Diagrammen vorkommen, ohne dass diese dann mehrfach im Modell enthalten sind. So können beispielsweise die Eigenschaften und Elemente eines Warenkorbs in einem Onlineshop mit dem Klassendiagramm und gleichzeitig der Ablauf einer Bestellung unter Verwendung des Warenkorbs mit einem Aktivitätsdiagramm modelliert werden. Da beide Diagramme zu einem UML-Modell gehören, ist es für den Leser der Diagramme sofort klar, dass es sich dabei um das gleiche Geschäftsobjekt Warenkorb handelt.

Trennung von Bedeutung und Darstellung

Die UML ist eine Modellierungssprache zur grafischen Modellierung, in der jedes grafische Modellelement eine ganz bestimmte Bedeutung hat. Im UML-Standard ist die Menge der möglichen Begriffe festgelegt, die bei der Erstellung eines UML-Modells verwendet werden können. Weiterhin legt die UML für jeden Diagrammtyp fest, welche der Begriffe aus der Menge aller in der UML verfügbaren Begriffe im jeweiligen Diagrammtyp verwendet werden können und welche grafische Darstellung dazu verwendet werden soll. Das bedeutet zum einen, dass die Menge der erlaubten Modellelemente vom konkreten UML-Diagramm abhängig ist und zum anderen, dass sich die grafische Darstellung des gleichen Begriffs je nach Diagramm unterscheiden kann, die Bedeutung jedoch die gleiche bleibt. Damit ist die Bedeutung unabhängig von der grafischen Darstellung, was z. B. die automatische Transformation von UML-Modellen in andere Modelltypen oder direkt in Programmcode ermöglicht. Außerdem ermöglicht die konsequente Trennung von Bedeutung und Darstellung auch eine projekt- oder firmenspezifische Erweiterung oder Einschränkung der UML.

Neben der Bereitstellung einer Menge möglicher Begriffe und deren grafischer Symbole zur Modellierung eines UML-Modells legt die UML auch Regeln zur Konstruktion von Diagrammen fest. So ist zu jedem Element genau festgelegt, mit welchen anderen Elementen es auf welche Weise verbunden werden darf und was diese Verbindung konkret bedeutet. Zum Beispiel sind Klassen in einem UML-Klassendiagramm, wie in Abbildung 31 oben dargestellt, immer über Assoziationen oder Vererbung miteinander verbunden und niemals direkt miteinander, wie in Abbildung 32 unten gezeigt.

Abbildung 32: Beispiel für Konstruktionsregeln

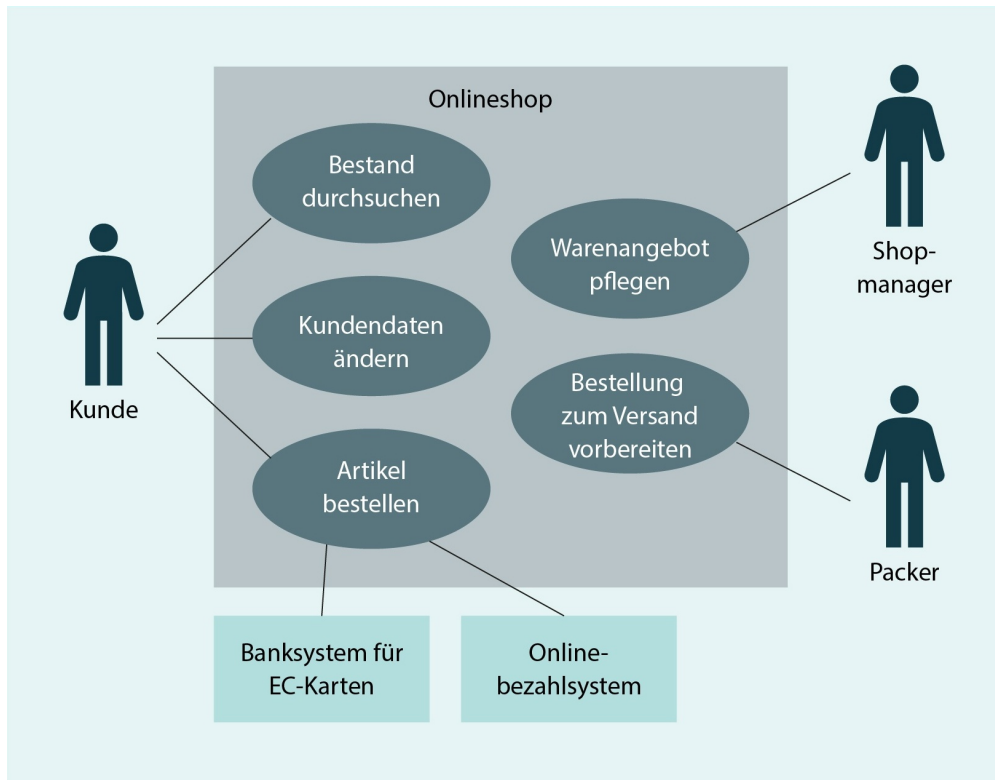


Quelle: erstellt im Auftrag der IU, 2023.

6.2 UML-Use-Case-Diagramm

Das Use-Case-Diagramm (auch: Anwendungsfalldiagramm) wird zur Darstellung der wichtigsten Funktionen eines Systems und dessen Schnittstellen in die Systemumgebung eingesetzt. Es bietet damit einen Überblick über das System und seine direkte Umgebung. Abbildung 33 zeigt beispielhaft ein einfaches Use-Case-Diagramm zu einem Onlineshop.

Abbildung 33: Beispiel für ein UML-Use-Case-Diagramm

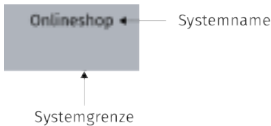
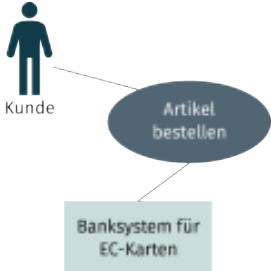


Quelle: erstellt im Auftrag der IU, 2023.

Use-Case-Diagramme werden eingesetzt, um den Systemkontext zu bestimmen und zu dokumentieren sowie um notwendige Schnittstellen des Systems zu identifizieren. Dabei wird auf einer sehr abstrakten Ebene das Verhalten des Systems aus Sicht der Nutzer bzw. aus Sicht der über technische Schnittstellen angebundenen Umsysteme dargestellt. Es werden jedoch keine Reihenfolgen, Aufrufbeziehungen oder Datenstrukturen dokumentiert.

Wichtige Notationselemente

Tabelle 21: Wichtige Notationselemente im Use-Case-Diagramm

Name	Bedeutung	Darstellung
Anwendungsfall (engl. use case)	Hauptfunktion eines Systems, die ausgeführt werden muss, um ein bestimmtes Ergebnis zu erzielen	
System, Systemgrenze	Das System stellt den Gegenstand der Betrachtung dar. Der Funktionsumfang eines Systems wird durch die Systemgrenze und die darin enthaltenen Anwendungsfälle festgelegt. Alle Elemente außerhalb der Systemgrenze befinden sich auch außerhalb des Systems.	
Akteur (engl. actor)	Ein Akteur ist eine Rolle oder ein anderes System, die/das mit dem betrachteten System interagiert. Dabei kann ein Akteur der Auslöser für Anwendungsfälle sein, der mit dem System ein bestimmtes Ergebnis erzielen kann, oder er ist zur Erreichung des Ergebnisses erforderlich, jedoch selbst nicht der Auslöser. Grundsätzlich befinden sich Akteure immer außerhalb der Systemgrenze.	
Kommunikation zwischen Akteur und System	Akteure, die bei der Ausführung eines Use Case mit dem System interagieren, werden direkt mit dem betreffenden Use Case verbunden.	

Quelle: erstellt im Auftrag derIU, 2023.

Weitere Notationselemente zu Use-Case-Diagrammen finden Sie unter anderem in Rupp/die SOPHISTEN/Queins (2012, S. 237ff.).

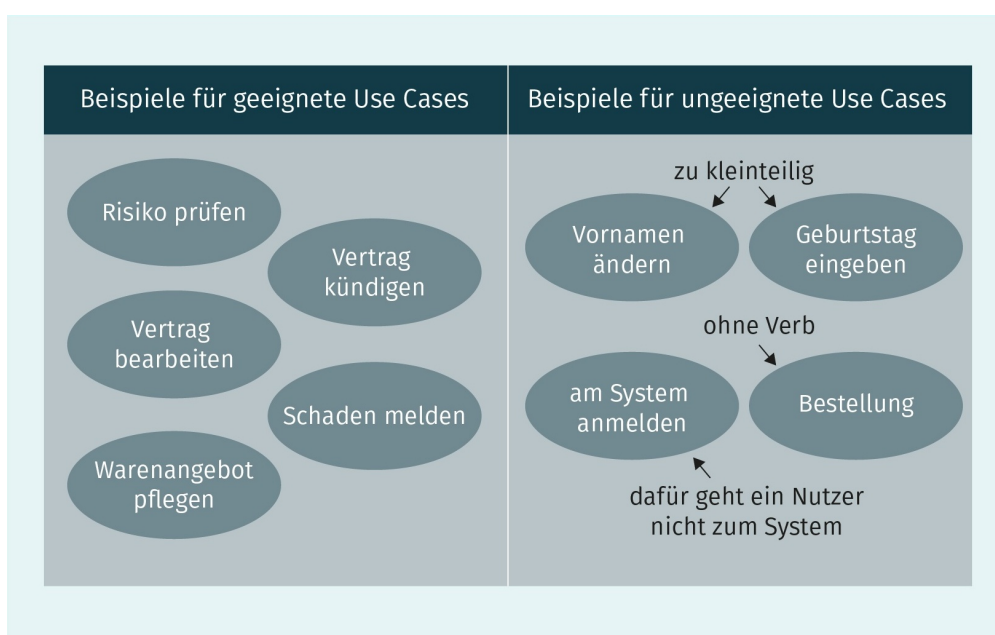
Einsatz im Requirements Engineering

Das Use-Case-Diagramm eignet sich zur einfachen Darstellung der Funktionen des Systems nach außen. Da es keine Details über das System enthält, kann es beispielsweise zur Kommunikation mit dem Anwender oder dem Management eingesetzt werden. Außerdem eignet es sich als Dokumentationsform für den Systemüberblick und zur Bestimmung des Systemkontexts.

Identifikation von Anwendungsfällen

In der Praxis fällt es häufig schwer, die richtige Ebene zu identifizieren, auf der Anwendungsfälle beschrieben werden können. So werden oft zu kleine und dadurch zu viele Use Cases dokumentiert. In der Regel ist ein Anwendungsfall eine Aufgabe, die in mehreren Schritten erledigt wird und mit welcher der Akteur ein bestimmtes Ergebnis erzielen möchte. Als Faustregel zur Identifikation von Anwendungsfällen eignet sich oft folgende Regel: Für einen Anwendungsfall meldet sich der Nutzer am System an bzw. begibt sich zum System. Abbildung 34 zeigt einige Beispiele für geeignete und ungeeignete Use Cases. Bei der Benennung der Use Cases sollte darauf geachtet werden, dass es sich immer um eine Aktivität handelt, die mit einem Verb beschrieben wird.

Abbildung 34: Beispiele für geeignete und ungeeignete Use Cases



Quelle: erstellt im Auftrag der IU, 2023.



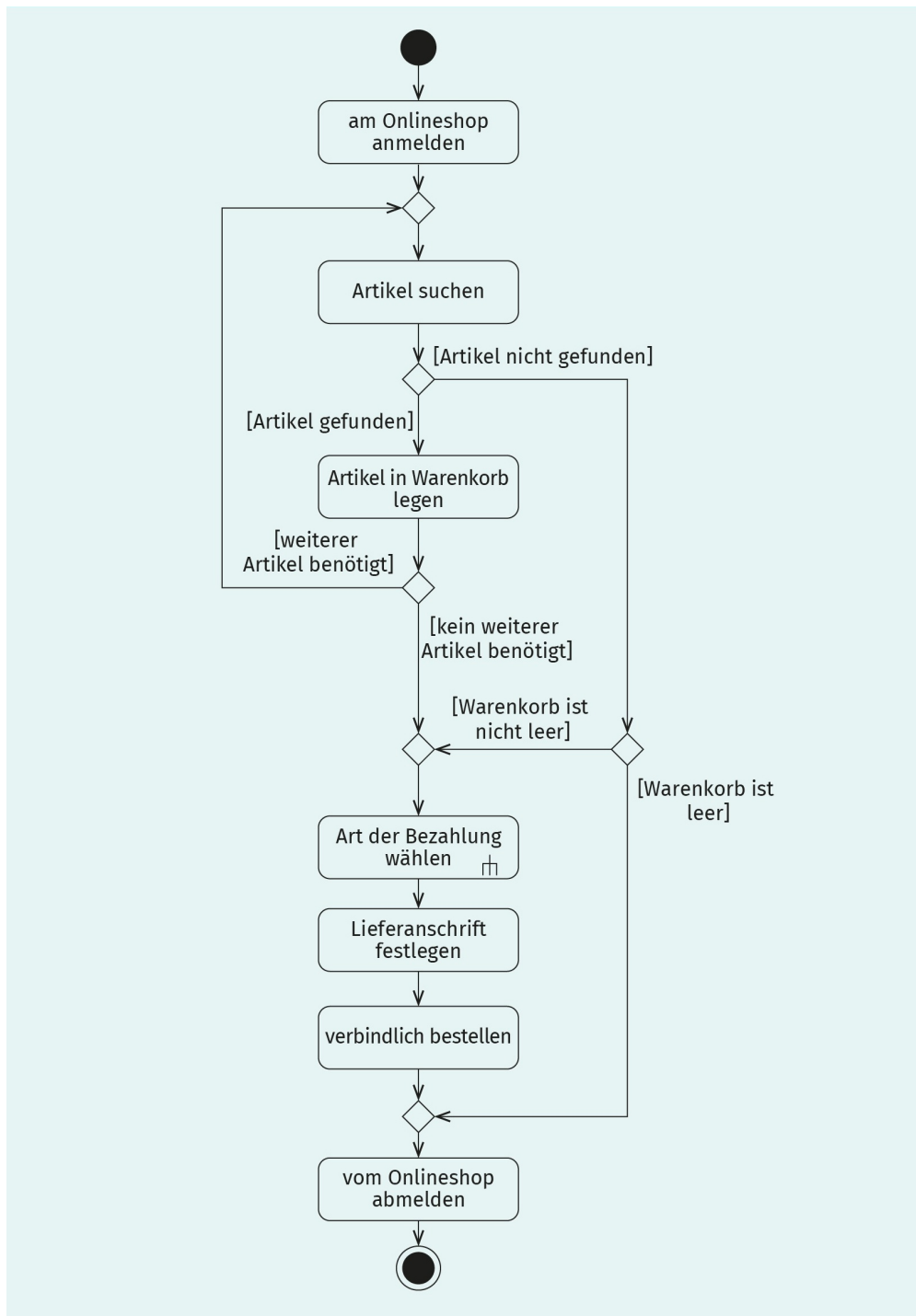
AUFGABE

Wählen Sie ein bekanntes System aus Ihrem Alltag, z. B. Kaffeevollautomat, DB-Ticketautomat oder Geldautomat, aus. Versuchen Sie, diese Systeme mit einem UML-Use-Case-Diagramm zu modellieren. Stellen Sie Ihre Lösung im Onlinetutorium vor und diskutieren Sie Ihre Lösung mit Ihren Kommilitonen und dem Tutor.

6.3 UML-Aktivitätsdiagramm

Das UML-Aktivitätsdiagramm ist ein Verhaltensdiagramm, mit dem Abläufe modelliert werden. Ein Ablauf kann dabei beispielsweise ein Geschäftsprozess sein, eine Folge von Nutzerinteraktionen mit dem System oder auch systeminterne Abläufe. Ganz allgemein wird das Aktivitätsdiagramm dann eingesetzt, wenn Aufgaben in ihre einzelnen Schritte zerlegt werden sollen. So eignet es sich beispielsweise, Anwendungsfälle eines Use-Case-Diagramms zu detaillieren. Während im Use-Case-Diagramm explizit keine Reihenfolgen oder Bedingungen modelliert werden, ist die Hauptaufgabe des Aktivitätsdiagramms die Visualisierung von detaillierten Abläufen mit Bedingungen, Schleifen und Verzweigungen. Die dabei eingesetzten Notationselemente ähneln den Notationselementen von Prozessmodellen wie EPK und BPMN. Abbildung 35 zeigt ein beispielhaftes UML-Aktivitätsdiagramm, das den Bestellvorgang in einem Onlineshop darstellt. Die Notationselemente werden im Folgenden erläutert.

Abbildung 35: Beispiel für ein UML-Aktivitätsdiagramm



Quelle: erstellt im Auftrag der IU, 2023.

Wichtige Notationselemente

In Tabelle 22 werden die Basisnotationselemente vorgestellt, mit denen einfachste Abläufe als UML-Aktivitätsdiagramm beschrieben werden können.

Tabelle 22: Wichtige Notationselemente im Aktivitätsdiagramm

Name	Bedeutung	Darstellung
Aktion (engl. action)	Eine Aktion entspricht dem Aufruf oder der Durchführung eines bestimmten Verhaltens. Die Menge aller Aktionen bestimmt das Verhalten der Aktivität.	
Kontrollfluss (engl. control flow)	gerichtete Kante mit einer Pfeilspitze, die den logischen Ablauf zwischen den Aktionen einer Aktivität festlegt	
Startknoten	Dieser legt den Startpunkt der Aktivität fest. An ihm beginnt der Kontrollfluss. Der Startknoten hat keine weitere Bedeutung.	
Endknoten für Aktivitäten	Dieser legt das Ende der Aktivität fest. Sobald ein solcher Endknoten erreicht ist, wird die Aktivität beendet.	
Aktivitäten (engl. activity)	Dies sind komplexe Aktionen, die noch weiter detailliert werden können, werden als Aktivität modelliert. Das Notationssymbol entspricht dem einer Aktion, das um ein kleines Symbol in Form einer Gabel ergänzt wird. Die Details einer Aktivität können in einem separaten Aktivitätsdiagramm modelliert werden.	

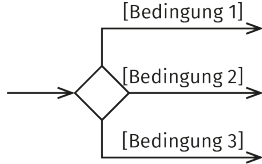
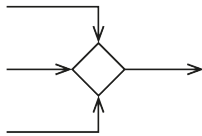
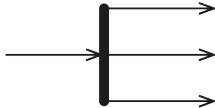
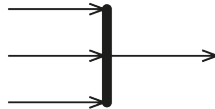
Quelle: erstellt im Auftrag der IU, 2023.

Die modellierten Aktionen und Aktivitäten in einem Aktivitätsdiagramm werden in ihrer festgelegten Reihenfolge abgearbeitet. Sobald mit der Durchführung einer Aktion gestartet wurde, können die Nachfolger dieser Aktion erst dann durchgeführt werden, wenn die Aktion beendet wurde. Es sei denn, es wurde explizit die parallele Durchführung von Aktionen modelliert.

Verzweigungen und Zusammenführungen

Neben den in Tabelle 23 vorgestellten Basisnotationselementen können auch Verzweigungen und Zusammenführungen des Kontrollflusses modelliert werden.

Tabelle 23: Verzweigungen und Zusammenführungen

Name	Bedeutung	Darstellung
Bedingung	Der Kontrollfluss wird nur dann berücksichtigt, wenn die Bedingung erfüllt ist. Ist die Bedingung nicht erfüllt, stoppt der Ablauf an dieser Stelle.	[Warenkorb ist leer]
Verzweigungs-knoten (engl. decision node)	Dieser spaltet eine Kante in mehrere Alternativen auf. Der Ablauf geht auf genau einer der ausgehenden Kanten weiter. Dieser Knoten entspricht einem logischen XOR.	
Verbindungs-knoten (engl. merge node)	Dieser führt alle eingehenden Kanten zusammen. Der Ablauf geht weiter, sobald der Fluss einer Kante den Knoten erreicht. Es findet keine Synchronisation statt, d. h., es wird nicht auf die anderen eingehenden Kanten gewartet.	
Parallelisie-rungsknoten	Dieser teilt die eingehende Kante in mehrere parallele Abläufe auf. Entspricht einem logischen AND. Dieser Knoten kann zu einem logischen OR erweitert werden, sobald die ausgehenden Kanten mit Bedingungen versehen werden.	
Synchronisati- onsknoten	Dieser führt eingehende Kanten zu einem gemeinsamen Ablauf zusammen. Dabei wird so lange gewartet, bis alle Flüsse an allen eingehenden Kanten ankommen.	

Quelle: erstellt im Auftrag der IU, 2023.

Als Faustregel für die Modellierung von Aktivitätsdiagrammen wird die paarweise Verwendung von Verzweigungs- und Verbindungsknoten sowie Parallelisierungs- und Synchronisationsknoten empfohlen. Das bedeutet, dass eine Verzweigung immer mit einer Zusammenführung wieder geschlossen wird. Ebenso sollte eine Parallelisierung immer mit einer Synchronisation wieder geschlossen werden.

Weitere Notationselemente im Aktivitätsdiagramm werden bei Rupp/die SOPHISTen/Queins (2012, S. 259ff.) erläutert.

Einsatz im Requirements Engineering

Das UML-Aktivitätsdiagramm wird im Requirements Engineering zur Verfeinerung und Detaillierung von Anwendungsfällen eingesetzt. Es eignet sich dabei zur Darstellung von Abläufen im Zusammenspiel mit fachlichen Ausführungsbedingungen. Insbesondere können im Aktivitätsdiagramm auch erforderliche Aktionen beschrieben werden, die im Fehlerfall oder in Ausnahmesituationen wichtig sind.

Grundsätzlich können Abläufe an Stelle eines Aktivitätsdiagramms auch mit BPMN oder EPK modelliert werden.



AUFGABE

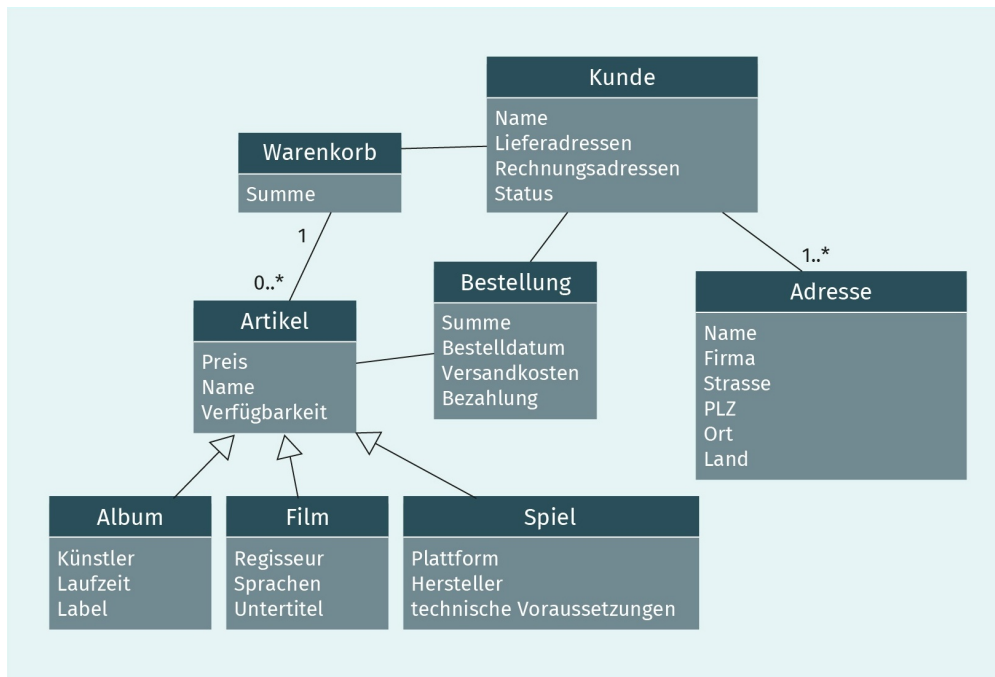
Überlegen Sie sich einen bekannten Ablauf aus Ihrem Alltag, z. B. Geldabheben am Automaten, Tätigen einer Onlineüberweisung oder Kaufen eines Online-DB-Tickets und versuchen Sie, diese Abläufe mit einem UML-Aktivitätsdiagramm zu modellieren. Stellen Sie Ihre Lösung im Onlinetutorium vor und diskutieren Sie Ihre Lösung mit Ihren Kommilitonen und dem Tutor.

6.4 UML-Klassendiagramm

Das UML-Klassendiagramm ist ein Strukturdiagramm, das zur Modellierung von Geschäftsobjekten und Systemen eingesetzt werden kann. Dieser Diagrammtyp wird sowohl auf der Überblicksebene verwendet, um allgemeine Zusammenhänge darzustellen, als auch auf einer sehr systemnahen Ebene, um detailliert die Attribute und Methoden zu Klassen in objektorientierten Systemen zu spezifizieren bzw. zu dokumentieren. Im Requirements Engineering, insbesondere mit dem Fokus auf fachliche Anforderungen, wird in der Regel nur eine sehr reduzierte Form des Klassendiagramms eingesetzt. Dabei werden implementierungsnahe Informationen wie Sichtbarkeiten oder technische Datentypen bewusst weggelassen. Insbesondere bei der Dokumentation von Geschäftsobjekten und anderen fachlichen Entitäten sowie deren Eigenschaften und Beziehungen untereinander wird auf die Modellierung von Details wie Methoden und Datentypen im Klassendiagramm bewusst verzichtet.

Das UML-Klassendiagramm ist eine der am häufigsten genutzten Dokumentationsformen bei der objektorientierten Systementwicklung. Es kann jedoch grundsätzlich zur Analyse und Dokumentation fachlicher Aspekte eingesetzt werden, ohne dass am Ende ein objektorientiertes System daraus erstellt wird. Abbildung 36 zeigt beispielhaft ein UML-Klassendiagramm, wie es für die Dokumentation fachlicher Anforderungen eingesetzt wird.

Abbildung 36: Beispiel für UML-Klassendiagramm



Quelle: erstellt im Auftrag der IU, 2023.

Notationselemente: Klassen und ihre Eigenschaften

Klassen werden immer in Form eines Rechtecks dargestellt. Der Name der Klasse wird in das Rechteck geschrieben. Zu einer Klasse können relevante Eigenschaften (auch: Attribute) identifiziert werden. Diese werden, wie in Abbildung 36 dargestellt, als Attribute unterhalb des Klassennamens in einem eigenen Rechteck notiert. Eine Klasse mit Attributen besteht also aus zwei zusammenhängenden Rechtecken: Im oberen Rechteck steht der Name der Klasse und im unteren die Liste der Attribute einer Klasse. Zu jedem Attribut ist mindestens der Name erforderlich. Weitere Eigenschaften, wie Datentyp und Defaultwerte, können bereits angegeben werden, müssen aber nicht. Die fehlenden Eigenschaften werden im Verlauf des Entwicklungsprozesses festgelegt. In Tabelle 24 wird gezeigt, wie Klassen mit und ohne Eigenschaften im UML-Klassendiagramm modelliert werden.

Tabelle 24: Klassen im UML-Klassendiagramm

Name	Bedeutung	Darstellung
Klasse	Eine Klasse entspricht einer fachlichen Entität oder einem Geschäftsobjekt.	Kunde
		Antrag
		Vertrag

Name	Bedeutung	Darstellung
Klasse mit Eigenschaften	Die zu einer Klasse relevanten Eigenschaften werden als Attribute in einem Rechteck unter dem Klassennamen modelliert.	Kunde Name Lieferadressen Rechnungsadressen Status

Quelle: erstellt im Auftrag der IU, 2023.

Notationselemente: Beziehungen zwischen Klassen

Fachliche Abhängigkeiten und Beziehungen zwischen Klassen können durch sogenannte Beziehungen (auch: Assoziationen) zwischen Klassen beschrieben werden. Im Klassendiagramm sind dabei unter anderem die Modellierung der in Tabelle 25 beschriebenen Typen von Beziehungen möglich.

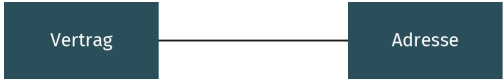
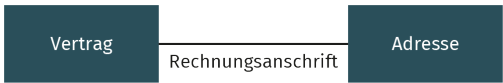
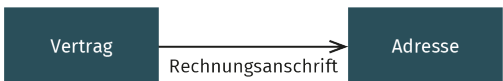
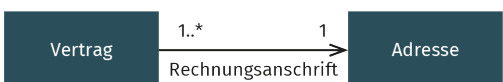
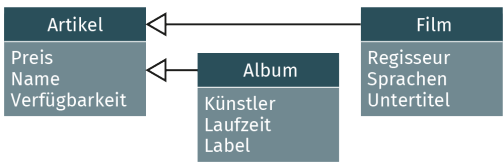
Tabelle 25: Typische Beziehungen zwischen Klassen

Beziehungstypen	Beschreibung	Beispiele
„hat/kennt“	Dieser Beziehungstyp drückt aus, dass eine Klasse eine andere Klasse „hat“ oder „kennt“.	Ein Versicherungsnehmer hat Kinder. Ein Vertrag hat Versicherungsbedingungen. Ein Kalender hat Monate. Ein Verkäufer kennt seine Kunden.
„besteht aus“	Dieser Beziehungstyp drückt aus, dass eine Klasse ein Bestandteil einer anderen Klasse ist. Er wird genutzt, wenn eine Klasse ein größeres Konstrukt ist, dessen Elemente nicht durch einfache Attribute beschrieben werden können.	Ein Auto besteht aus einem Motor, 4 Rädern, 3 Türen, 1 Getriebe und 2 Sitzen. Ein Haus besteht aus einem Dach, 23 Fenstern, 2 Türen, 6 Räumen und 1 Treppenhaus.
„ist ein“	Dieser Beziehungstyp drückt aus, dass eine Klasse A von der Art her eine Klasse B ist, aber eine spezifischere Bedeutung hat und sich ggf. um bestimmte Attribute und Methoden von Klasse B unterscheidet.	Ein Pkw ist ein Auto. Ein Lkw ist ein Auto. Ein Kunde ist eine Person. Ein Buch ist ein Artikel. Ein Igel ist ein Säugetier.

Quelle: erstellt im Auftrag der IU, 2023.

Um neben Klassen, Attributen und Methoden ebenfalls die Beziehungen grafisch darzustellen, werden Linien bzw. Pfeile zwischen den entsprechenden Klassen modelliert. Die einfachste Art Beziehungen herzustellen, ist die Verbindung von zwei Klassen durch eine durchgezogene Linie. Eine Beziehung kann, wie in Tabelle 26 gezeigt, neben der durchgezogenen Linie um weitere Informationen ergänzt werden.

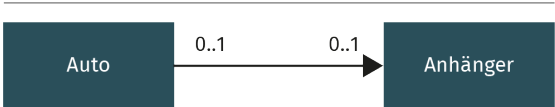
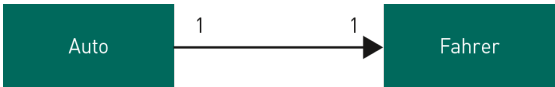
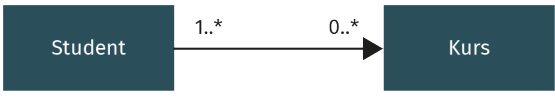
Tabelle 26: Darstellung von Beziehungen zwischen Klassen

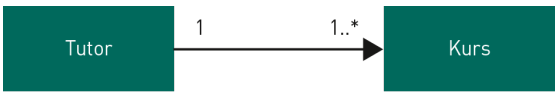
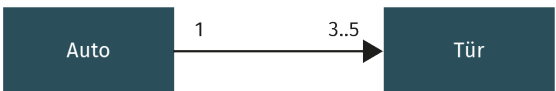
Darstellung der Beziehung	Bedeutung
 durchgezogene Linie, ohne Pfeilspitze und ohne Beschriftung	Vertrag und Adresse stehen in einer nicht näher beschriebenen Beziehung zueinander.
 durchgezogene Linie, Beschriftung der Linie mit einem Namen für die Beziehung	Vertrag und Adresse sind über eine benannte Assoziation verbunden; die beiden Klassen sind verbunden durch die Beziehung „Rechnungsanschrift“.
 durchgezogene Linie mit einer Pfeilspitze, ggf. Benennung der Beziehung	Durch die Pfeilspitze wird eine Navigationsrichtung vorgeben: vom Vertrag zur Adresse. Das bedeutet, dass der Vertrag eine Adresse kennt und man sich vom Vertrag zur Adresse durchhangeln kann, jedoch nicht von der Adresse zum Vertrag: Ein Vertrag kennt seine Adresse, aber die Adresse weiß nichts von und über die Existenz des Vertrags.
 An den Enden der Beziehung werden Multiplizitäten modelliert und damit Aussagen über die Anzahl der assoziierten Objekte gemacht.	Ein Vertrag hat genau eine Rechnungsanschrift, eine Adresse kann jedoch die Rechnungsanschrift zu mindestens einem, aber maximal beliebig vielen Verträgen sein (detaillierte Erklärung dazu siehe unten).
 Vererbungsbeziehung: durchgezogene Linie, geschlossene Pfeilspitze an einem Ende	Dies ist die Klasse, auf welche die Pfeilspitze zeigt (hier: Artikel), vererbt alle Attribute auf die Klassen am anderen Ende der Beziehung (hier: Film und Album). Die Vererbung ist eine „ist ein“-Beziehung. Die Klassen Film und Album haben ebenfalls die Attribute Preis, Name und Verfügbarkeit, ohne diese jedoch extra als eigenes Attribut zu haben.

Quelle: erstellt im Auftrag der IU, 2023.

Zu Beziehungen können mit Multiplizitäten (auch: Kardinalitäten) Mengenangaben festgelegt werden. Diese Angaben werden jeweils an das Ende und die Spitze einer Beziehung notiert: Links von „..“ steht die Untergrenze und rechts von „..“ die Obergrenze der Mengenangaben. Tabelle 27 gibt einen Überblick über mögliche Mengenangaben. Davon ausgenommen sind jedoch die Vererbungsbeziehungen. Zu ihnen können keine Mengenangaben modelliert werden.

Tabelle 27: Multiplizitäten im Klassendiagramm

Notation	Erklärung	Beispiel
0..1	optionale Assoziation	 <pre> classDiagram class Auto class Anhänger Auto "0..1" -- "0..1" Anhänger </pre> Zu einem Auto gehören 0..1 Anhänger. Zu einem Anhänger gehören 0..1 Autos.
1	obligatorische Assoziation	 <pre> classDiagram class Auto class Fahrer Auto "1" -- "1" Fahrer </pre> Zu einem Auto gehört genau 1 Fahrer. Ein Fahrer gehört zu genau 1 Auto.
0..*	optional beliebig	 <pre> classDiagram class Student class Kurs Student "1..*" -- "0..*" Kurs </pre> Ein Student kann 0..* Kurse belegen. Ein Kurs kann von 1..* Studenten belegt sein.

Notation	Erklärung	Beispiel
1..*	beliebig, aber mindestens 1	 Ein Tutor kann 1..* Kurse machen. Ein Kurs wird von genau 1 Tutor durchgeführt.
n..m	mindestens n und maximal m	 Ein Auto hat mindestens 3 und maximal 5 Türen. Eine Tür gehört zu genau 1 Auto.
	Keine Angabe entspricht 1, sollte aber vermieden werden, um Missverständnisse zu vermeiden.	 Ein Auto hat genau 1 Motor. Ein Motor gehört zu genau 1 Auto.

Quelle: erstellt im Auftrag der IU, 2023.

Weitere Notationselemente zu Klassendiagrammen finden Sie unter anderem bei Rupp/die SOPHISTen/Queins (2012, S. 101ff.).

Einsatz im Requirements Engineering

Im Umfeld des Requirements Engineering wird das Klassendiagramm genutzt, um statische Konzepte eines Anwendungsbereichs zu dokumentieren. Das umfasst Geschäftsobjekte, fachliche Entitäten (branchenspezifische Dinge der realen Welt, die mit Informationssystemen verwaltet werden sollen), Personen, Objekte, Systeme und deren relevante Eigenschaften sowie Beziehungen und Abhängigkeiten zueinander. Das vordergründige Ziel ist dabei die Dokumentation, das Verstehen und die Kommunikation des fachlichen Problems. Das bedeutet, die Klassen im Klassendiagramm werden als Konzepte der fachlichen Umgebung betrachtet und nicht als Elemente eines Systems. Eine UML-Klasse ent-

spricht dabei einem fachlichen Konzept, also einer Menge von Objekten mit den gleichen Eigenschaften, beispielsweise Kunde, Artikel oder Bestellung. Für die Darstellung von Verhalten oder Abläufen ist das Klassendiagramm nicht geeignet.



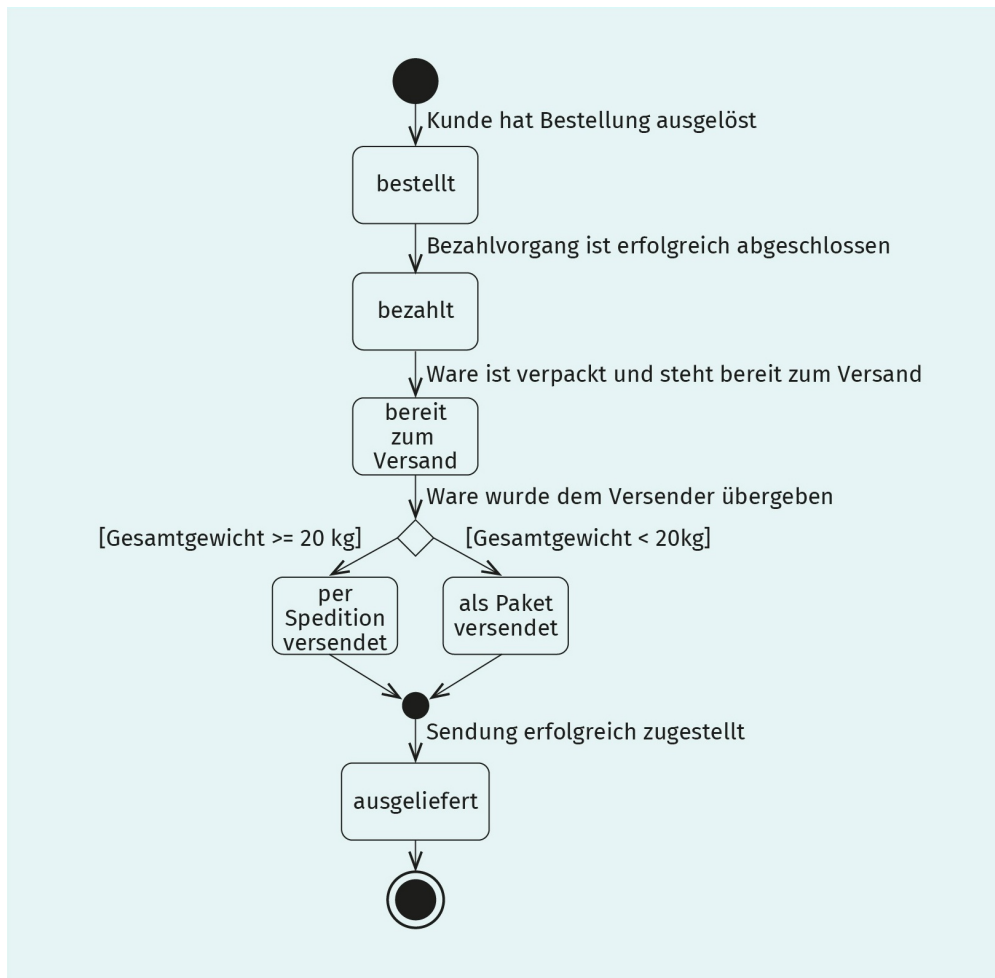
AUFGABE

Wählen Sie eine bekannte Umgebung aus Ihrem Alltag, z. B. Fernstudium, Onlineportal einer Bank oder einen Onlineshop. Versuchen Sie, die Geschäftsobjekte, Personen und andere fachliche Entitäten in einem UML-Klassendiagramm zu modellieren. Stellen Sie Ihre Lösung im Onlinetutorium vor und diskutieren Sie Ihre Lösung mit Ihren Kommilitonen und dem Tutor.

6.5 UML-Zustandsdiagramm

Das UML-Zustandsdiagramm ist ein Verhaltensdiagramm, mit dem fachliche Zustände von Objekten oder Systemen dokumentiert werden können. Im Gegensatz zu Aktionen und Abläufen, die im Aktivitätsdiagramm modelliert werden, legt das Zustandsdiagramm fest, welche Zustände ein Objekt oder ein System annehmen kann und welche Abhängigkeiten bzw. Reihenfolgen es zwischen den Zuständen gibt und wann diese erreicht werden können. In Abbildung 37 ist ein Zustandsdiagramm dargestellt, das die fachlichen Zustände einer Bestellung eines Onlineshops darstellt. Es stellt den Verlauf einer Bestellung anhand einer Abfolge von Zuständen dar.

Abbildung 37: Beispiel für UML-Zustandsdiagramm



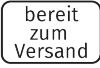
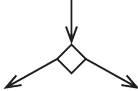
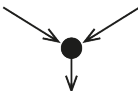
Quelle: erstellt im Auftrag der IU, 2023.

Im Gegensatz zu Abläufen beschreibt ein Zustandsdiagramm keine Aktionen, sondern die Konsequenzen bzw. Auswirkungen von Aktionen oder Aktivitäten.

Wichtige Notationselemente

In Tabelle 28 werden die wichtigsten Notationselemente vorgestellt, mit denen einfache Zustandsdiagramme beschrieben werden können.

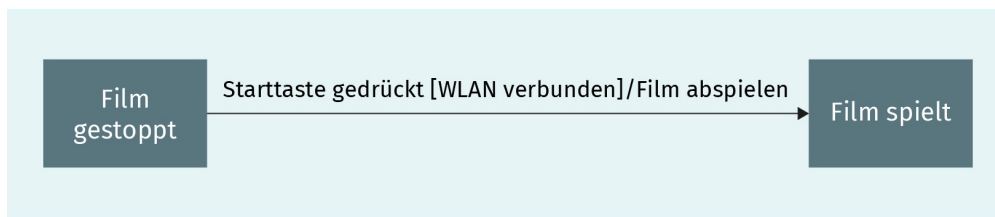
Tabelle 28: Wichtige Notationselemente im UML-Zustandsdiagramm

Name	Bedeutung	Darstellung
Zustand (engl. state)	Ein Zustand ist eine bestimmte fachliche Situation eines Objektes oder eines Systems, aus der sich fachliche Aussagen oder notwendige Aktivitäten ableiten lassen.	
Startzustand	Dieser markiert den Startpunkt im Zustandsdiagramm. Alle direkten Nachfolgezustände dieses Startzustands sind mögliche erste fachliche Zustände. Das Symbol Startzustand hat sonst keine weitere Bedeutung.	
Endzustand	Dieser markiert den Endpunkt im Zustandsdiagramm. Alle direkten Vorgängerzustände des Endzustands sind mögliche letzte Zustände des Objekts bzw. des Systems. Das Symbol Endzustand hat sonst keine weitere Bedeutung.	
Zustandsübergang, Transition (engl. transition)	Dieser bestimmt die Reihenfolge der Zustände.	
Entscheidung	Eine Entscheidung ermöglicht es, die Auswahl der nächsten Transition von einem aktuellen Ergebnis abhängig zu machen.	
Zusammenführung	Eine Zusammenführung erlaubt die Zusammenfassung mehrerer Transitionen, wenn jeweils der Nachfolgezustand identisch ist.	
Trigger [Guard]/ Aktivität	Die Elemente Trigger, Guard und Aktivität können zusätzlich an eine Transition modelliert werden: • Trigger sind Auslöser von Transitionen. • Der Guard ist eine Bedingung, die wahr sein muss, damit die Transition ausgelöst wird. • Die Aktivität ist eine konkrete Aktivität, die beim Durchlaufen der Transition ausgelöst wird.	[Gesamtgewicht < 20 kg] Sendung erfolgreich zugestellt

Quelle: erstellt im Auftrag der IU, 2023.

In Abbildung 38 ist ein Beispiel für die kombinierte Verwendung von Trigger, Guard und Aktivität im Zustandsdiagramm dargestellt. Diese zusätzlichen Eigenschaften werden häufig bei der Modellierung von eingebetteten Systemen verwendet, deren Verhalten stark auf die Reaktion von Sensoreingaben ausgerichtet ist. Im Umfeld der Informationssysteme wird diese Darstellung nicht sehr häufig verwendet.

Abbildung 38: Beispiel für Trigger, Guard und Aktivität im Zustandsdiagramm



Quelle: erstellt im Auftrag der IU, 2023.

Weitere Elemente von Zustandsdiagrammen werden beispielsweise von Rupp/die SOPHISTen/Queins (2012, S. 325ff.) erläutert.

Einsatz im Requirements Engineering

Zustandsdiagramme werden im Requirements Engineering eingesetzt, um Lebenszyklen oder bestimmte fachliche Zustände von Geschäftsobjekten und fachlichen Entitäten zu dokumentieren. Da Zustandsdiagramme insbesondere keine Aktivitäten, sondern deren Auswirkungen auf den fachlichen Zustand modellieren, kann beispielsweise anhand eines Zustandsdiagramms dokumentiert werden, welche Aktivitäten in welchen Zuständen überhaupt erlaubt sind. Da mit fachlichen Zustandsübergängen der gesamte Lebenszyklus eines Geschäftsobjekts in einer sehr kompakten Form dargestellt werden kann, eignen sie sich als Ergänzung und als Überblick zu den in der Regel recht komplexen Prozess- oder Ablaufdiagrammen.

Ein weiterer Anwendungsfall für Zustandsdiagramme ist die Modellierung von Aufrufreihenfolgen von Bildschirmmasken. Dabei entspricht jeder Zustand einer Bildschirmmaske bzw. Bildschirmseite. Die Zustandsübergänge bilden dann die Klickfolge des Nutzers ab.



AUFGABE

Wählen Sie bekannte Geschäftsobjekte aus Ihrer Umgebung. Versuchen Sie dann, den Lebenszyklus dieser Geschäftsobjekte in einem UML-Zustandsdiagramm zu modellieren. Alternativ können Sie auch das Absolvieren eines Fernstudiumskurses oder das Kaufen bzw. Ersteigern eines Gegenstands in einem Onlineauktionshaus modellieren. Stellen Sie Ihre Lösung im Onlinetutorium vor und diskutieren Sie Ihre Lösung mit Ihren Kommilitonen und dem Tutor.



ZUSAMMENFASSUNG

Die Unified Modeling Language (kurz: UML) ist eine grafische Modellierungssprache im Softwareengineering. Sie umfasst 14 verschiedene Diagrammtypen, mit denen sowohl die Struktur als auch das Verhalten von Systemen dokumentiert werden kann.

Das Use-Case-Diagramm ist ein Verhaltensdiagramm und wird zur Darstellung der wichtigsten Funktionen eines Systems und dessen Schnittstellen in die Systemumgebung eingesetzt. Es enthält keine Details über das System und eignet sich somit als Dokumentationsform für den Systemüberblick.

Das UML-Aktivitätsdiagramm ist ebenfalls ein Verhaltensdiagramm. Mit ihm werden Abläufe modelliert. Es wird eingesetzt, wenn Aufgaben in ihre einzelnen Schritte zerlegt und detaillierte Abläufe mit Bedingungen, Schleifen und Verzweigungen dokumentiert werden sollen, beispielsweise zur Detaillierung von identifizierten Use Cases aus dem Use-Case-Diagramm.

Das UML-Klassendiagramm ist ein Strukturdiagramm, das zur Modellierung von Geschäftsobjekten und Systemen eingesetzt werden kann. Es wird im Requirements Engineering zur Darstellung statischer Strukturen fachlicher Konzepte eingesetzt. Eine UML-Klasse entspricht dabei einem fachlichen Konzept, also einer Menge von Objekten mit den gleichen Eigenschaften, beispielsweise Kunde, Artikel oder Bestellung.

Das UML-Zustandsdiagramm ist ein Verhaltensdiagramm, mit dem fachliche Zustände von Objekten oder Systemen dokumentiert werden können. Es legt fest, welche Zustände ein Objekt oder ein System annehmen kann und welche Abhängigkeiten bzw. Reihenfolgen es zwischen den Zuständen gibt und wann diese erreicht werden können. Es wird eingesetzt, um Lebenszyklen oder bestimmte fachliche Zustände von Geschäftsobjekten und fachlichen Entitäten oder auch Aufrufreihenfolgen von Bildschirmmasken zu dokumentieren.

LEKTION 7

PRÜFEN UND ABSTIMMEN VON ANFORDERUNGEN

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- welche Aktivitäten zum Prüfen und Abstimmen von Anforderungen erforderlich sind.
- welche Prüfkriterien es zu Anforderungen gibt und welchen Qualitätsaspekten diese zugeordnet werden.
- mit welchen Prüfprinzipien eine hohe Qualität der Prüfergebnisse von Anforderungen erreicht werden kann.
- welche Techniken sich zum Prüfen von Anforderungen eignen.
- wie bei der Abstimmung Widersprüche in der Menge der Anforderungen aufgelöst werden können.

7. PRÜFEN UND ABSTIMMEN VON ANFORDERUNGEN

7.1 Aktivitäten zum Prüfen und Abstimmen von Anforderungen

Das Ziel der Aktivitäten zum Prüfen und Abstimmen ist die Freigabe der Anforderungen zur Umsetzung in den nachgelagerten Phasen des Softwareprojekts. Wie bereits erwähnt, muss vor der Weiterverarbeitung der ermittelten und dokumentierten Anforderungen deren Qualität gesichert werden. Das ist insbesondere notwendig, weil alle weiteren Aktivitäten im Softwareentwicklungsprozess direkt oder indirekt von den dokumentierten Anforderungen abhängig sind. Jeder Fehler in den Anforderungen, der nicht frühzeitig erkannt wird, pflanzt sich über die technische Spezifikation und die Architektur bis hin zum Programmcode fort, denn während eine fehlerhafte Anforderung in der Regel mit sehr einfachen Mitteln im Requirements Engineering durch Umformulieren behoben werden kann, entsteht bei der Fehlerbehebung in der Softwarearchitektur oder im Programmcode deutlicher Mehraufwand. Zum einen ist zu diesem Zeitpunkt bereits ein erheblicher Aufwand in die Erstellung von fehlerhaftem Code investiert worden, und zum anderen erfordert die Korrektur der Fehler weiteren Aufwand. Dazu zählen neben dem Aufwand für das Korrigieren der Fehler auch die Aufwände zum Testen des Systems und dessen Inbetriebnahme. Durch eine sorgfältige Prüfung der Anforderungen mit vergleichsweise wenig Aufwand können also bereits Fehler identifiziert und behoben werden, deren Identifikation und Behebung in nachgelagerten Phasen vergleichsweise teuer werden kann.

Darüber hinaus bergen unzureichend geprüfte Anforderungen rechtliche Risiken. Neben Vertragsstrafen aufgrund der Nichteinhaltung von Lieferzeiten besteht auch die Gefahr, gegen rechtliche Vorgaben, beispielsweise das Datenschutzgesetz, zu verstoßen, was zu strafrechtlichen Konsequenzen führen kann.

Zusätzlich zur Prüfung von Anforderungen hinsichtlich deren fachlicher und handwerklicher Qualität, ist es jedoch insbesondere für fachliche Anforderungen wichtig, dass sie mit allen relevanten Stakeholdern abgestimmt sind. Die Menge der zur Umsetzung freigegebenen Anforderungen entspricht dem Arbeitsauftrag eines Softwareprojekts, und die Umsetzung von Anforderungen in ein System verursacht immer Aufwände. Außerdem lassen sich einmal umgesetzte Anforderungen nicht ohne Weiteres wieder aus dem System entfernen. Daher hilft ein etablierter und projektbegleitender Abstimmungsprozess bei der Auswahl der wichtigsten umzusetzenden Anforderungen. In Prozessmodellen, die eine evolutive Entwicklung unterstützen, werden beispielsweise für jeden Zyklus die zu bearbeitenden Anforderungen jeweils neu festgelegt.

Eine Abstimmung zwischen den Stakeholdern ist erforderlich, sobald in der Menge der dokumentierten Anforderungen Widersprüche identifiziert wurden. Falls die sich widersprechenden Anforderungen von verschiedenen Stakeholdern stammen, die jeweils auch

noch widersprüchliche Ziele innerhalb des Projekts verfolgen, liegt eine Konfliktsituation vor. Ein weiterer Auslöser für einen Konflikt ist die Identifikation eines Widerspruchs zwischen den dokumentierten Anforderungen und der Ansicht eines Stakeholders. Gründe für diese Situation können beispielsweise im Projektverlauf geänderte Anforderungen an das System oder eine unvollständige Anforderungsermittlung sein. In der Praxis zeigt sich, dass sich widersprechende Anforderungen oft aus einem unterschiedlichen Verständnis von Begriffen, Zuständigkeiten und der Motivation des Projekts resultieren. Ziel der Abstimmung ist die Erreichung eines gemeinsamen und übereinstimmenden Verständnisses der Menge der Anforderungen unter den Stakeholdern (Pohl/Rupp 2015, S. 102). Mit den Mitteln des Konfliktmanagements muss der Konflikt gelöst und gleichzeitig die Kooperationsbereitschaft der Stakeholder aufrecht erhalten werden.

Vergleichbar mit den Kernaktivitäten „Ermittlung und Dokumentation von Anforderungen“ lässt sich auch die Kernaktivität „Prüfen und Abstimmen“ in vier Schritte unterteilen:

1. Prüfkriterien festlegen,
2. Prüfprinzipien und Prüftechniken auswählen,
3. Prüfung durchführen und Ergebnisse dokumentieren sowie
4. Abstimmen der Anforderungen/Konfliktmanagement.

Prüfkriterien festlegen

Im Vorfeld der Prüfung ist festzulegen nach welchen Kriterien genau geprüft werden soll. Im folgenden Lernzyklus werden Prüfkriterien vorgestellt, die für Anforderungen relevant sind. Es gibt allerdings so viele mögliche Prüfkriterien, dass insbesondere bei der Prüfung von umfangreichen Dokumentationen und/oder unter hohem Zeitdruck nicht alle gleichermaßen berücksichtigt werden können. Daher müssen im Vorfeld der Prüfung die relevanten Prüfkriterien bestimmt werden. Welche das im Einzelnen sind, hängt maßgeblich von der Projektsituation, Art und Wichtigkeit der Anforderungen, der verfügbaren Zeit und der dafür eingeplanten Personen ab. Der verantwortliche Requirements Engineer ist für die Erstellung verantwortlich.

Prüfprinzipien und Prüftechniken auswählen

Erst nachdem entschieden wurde, welche Prüfkriterien tatsächlich relevant sind, und wenn feststeht, wie viele Ressourcen für die Prüfungen zur Verfügung stehen, werden die zu berücksichtigenden Prüfprinzipien und Prüftechniken ausgewählt.

Prüfung durchführen und Ergebnisse dokumentieren

Nach Abschluss der fachlichen Vorbereitung muss die Prüfung organisatorisch geplant werden. Dazu zählen die Organisation der benötigten Infrastruktur, die zeitliche Planung aller Beteiligten und ggf. die Koordination von externen Gutachtern. Wichtig bei der Durchführung ist dabei die Konzentration auf die Dokumentation der Prüfungsergebnisse. Die Fehlerbehebung und Überarbeitung der dokumentierten Anforderungen erfolgt im Anschluss und ist nicht Teil der Prüfung.

Abstimmen der Anforderungen/Konfliktmanagement

Wurden bei der Prüfung in der Menge der dokumentierten Anforderungen widersprüchliche Aussagen identifiziert oder gibt es Widersprüche zwischen Stakeholdern und dokumentierten Anforderungen, dann müssen diese Widersprüche abgestimmt und ggf. mit geeigneten Maßnahmen zum Konfliktmanagement die Konflikte aufgelöst werden.

7.2 Prüfkriterien

Wie oben bereits erwähnt, ist eine vollumfängliche Prüfung von Anforderungen nach allen erdenklichen Kriterien in der Praxis nicht möglich, da die Aufwände hierzu in keinem wirtschaftlich vernünftigen Verhältnis zum Nutzen stehen würden. Daher gibt es in der Literatur zum Requirements Engineering verschiedene Prüfkriterien, die jeweils gezielt einen Aspekt fokussieren. Möglich ist z. B. die Unterscheidung der drei Qualitätsaspekte Inhalt, Form und Abgestimmtheit (Pohl/Rupp 2015, S. 103), die sich wiederum an den Zielen des Requirements Engineering orientieren.

Mithilfe der Qualitätsaspekte wird die Menge der Prüfkriterien so strukturiert, dass man bei der Auswahl der Prüfkriterien zuerst die relevanten Qualitätsaspekte identifizieren und anschließend aus der Menge der den Qualitätsaspekten zugeordneten Prüfkriterien die jeweils relevanten auswählen kann. Im Folgenden werden mögliche Prüfkriterien für Anforderungen vorgestellt, von denen die meisten bei Pohl/Rupp (2015, S. 105ff.) beschrieben werden.

Prüfkriterien zum Inhalt

Die zentrale Frage, die mit den Kriterien zum Qualitätsaspekt Inhalt beantwortet werden soll, lautet sinngemäß: „Sind alle relevanten Anforderungen im erforderlichen Detailgrad so erfasst, dass sie verstanden werden können?“. Folgende Prüfkriterien können für eine inhaltliche Überprüfung herangezogen werden:

- **Vollständigkeit aller Anforderungen:** Hier wird die Gesamtheit der dokumentierten Anforderungen betrachtet. Mögliche Fragen sind: Umfasst die Menge der dokumentierten Anforderungen auch wirklich alle relevanten Anforderungen an das System? Fehlen Anforderungen? Wurden Anforderungen dokumentiert, die nicht relevant sind?
- **Vollständigkeit einzelner Anforderungen:** Hierzu wird jede einzelne Anforderung dahingehend untersucht, ob alle relevanten Informationen zur Beschreibung dieser Anforderung vorliegen: Sind alle Informationen zum Verständnis dieser Anforderungen dokumentiert?
- **Verfolgbarkeit:** Zu diesem Kriterium wird überprüft, ob zu jeder Anforderung die notwendigen Informationen dokumentiert sind, um eine angemessene Nachverfolgbarkeit zu gewährleisten: Falls es Querverweise auf zusätzliche Dokumente gibt, sind diese eindeutig? Falls es Abhängigkeiten zwischen Anforderungen gibt, sind diese klar erkennbar? Ist klar, wer der Ersteller bzw. die Quelle der Anforderung ist?

- Adäquatheit bezüglich der Wünsche der Stakeholder (auch: Korrektheit): Hier wird geprüft, ob die dokumentierten Anforderungen auch tatsächlich das wiedergeben, was die Stakeholder gemeint bzw. sich gewünscht haben: Hat das der Stakeholder wirklich so gesagt? Entsprechen die Anforderungen der Intention der Quelle?
- Widerspruchsfreiheit (auch: Konsistenz): Hierzu wird geprüft, ob es in der Menge der dokumentierten Anforderungen Widersprüche oder Inkonsistenzen gibt: Ist die gegebene Menge an Anforderungen erfüllbar? Schließen sich Anforderungen gegenseitig aus? Widersprechen sich Anforderungen?
- Keine vorzeitigen Entwurfsentscheidungen: Hier wird geprüft, ob in Anforderungen bereits festgelegt wird, wie die Anforderungen zu erfüllen sind, ohne dass es dazu verbindliche organisatorische oder rechtliche Randbedingungen gibt: Werden bereits Vorschriften zur konkreten Gestaltung des Systems gemacht? Werden zu verwendende Technologien vorgeschrieben? Werden detaillierte Angaben zu internen technischen Abläufen gemacht? Werden technische Einschränkungen oder Bedingungen beschrieben?



TIPP

Häufig werden in Anforderungsdokumenten fachlich unnötige Beschränkungen von Eingabefeldern gefordert, z. B. „Der Text darf nicht länger als 256 Zeichen sein“. Solche Anforderungen sollten ganz besonders kritisch hinterfragt werden, denn hinter diesen Einschränkungen verbergen sich oft Querbezüge zu ggf. noch nicht identifizierten Abhängigkeiten.

- Dokumentation des fachlichen Problems: Hierzu wird geprüft, ob sich zu jeder dokumentierten Anforderung das fachliche Problem identifizieren lässt: Ist zu jeder Anforderung klar, welches fachliche Problem sie löst bzw. welcher Zweck damit erfüllt wird?
- Überprüfbarkeit: Hier wird geprüft, ob sich aus der Dokumentation zu der Anforderung die korrespondierenden Abnahmekriterien und Testfälle für die Akzeptanztests erstellen lassen. Das erstellte System wird mit Testfällen getestet, die auf Grundlage der dokumentierten Anforderungen erstellt werden. Daher müssen die Anforderungen so formuliert werden, dass nach Abschluss der Implementierung deren Umsetzung überprüft werden kann: Lassen sich konkrete Testfälle ableiten? Ist klar, wie die Anforderungen nach der Umsetzung auf Richtigkeit überprüft werden können? Sind bereits im Abnahmetest zu prüfende Kriterien festgelegt?
- Notwendigkeit bezüglich des Zielbeitrags: Hierzu wird geprüft, wie relevant bzw. notwendig die Anforderung tatsächlich zur Erreichung des formulierten Ziels ist. Oft werden alle Anforderungen dokumentiert, welche die Stakeholder in dem Moment der Ermittlung wichtig finden. Dabei tragen nicht alle Anforderungen gleichermaßen zur Erreichung des Projektziels bei. Daher wird mit diesem Kriterium gezielt jede Anforderung reflektiert, inwieweit sie tatsächlich einen Beitrag zum Projektziel leistet: Lässt sich der Beitrag der Anforderung zum Projektziel beschreiben? Erfüllt die Anforderung ein wichtiges Bedürfnis der Stakeholder? Wird mit ihr eine Funktion oder Eigenschaft beschrieben, die vorerst nicht so wichtig ist?

Prüfkriterien zur Dokumentation

Wurden mit den Kriterien zur Prüfung des Inhalts insbesondere die fachlichen Aspekte geprüft, zielt die Prüfung der Dokumentation auf eine geeignete Kommunikationsunterstützung. Mit den Kriterien zur Prüfung der Dokumentation wird die zentrale Frage „Wurden die Anforderungen verständlich und unter Einhaltung der Vorschriften zur Dokumentation dokumentiert?“ beantwortet. Zur Überprüfung der Dokumentation können folgende Prüfkriterien eingesetzt werden:

- **Konformität zur Dokumentenstruktur:** Hierzu wird geprüft, ob zum einen die Anforderungen an der richtigen Stelle dokumentiert wurden und ob alle geforderten Elemente der Dokumentenstruktur ausgefüllt wurden: Entspricht die Dokumentenstruktur dem vorgegebenen Template? Sind alle Pflichtkapitel ausgefüllt? Werden insbesondere Qualitätseigenschaften und Randbedingungen an den richtigen Stellen im Dokument beschrieben?
- **Verständlichkeit:** Hierzu wird geprüft, ob der formulierte Text vom Leser verstanden werden kann und ob fach- oder projektspezifische Begriffe und Abkürzungen im Glossar erläutert werden: Ist jede Abkürzung im Glossar beschrieben? Ist der Text verständlich formuliert? Sind alle benötigten Kontextinformationen gegeben, damit die Anforderungen richtig verstanden werden können?
- **Eindeutigkeit:** Hierzu wird überprüft, ob die Anforderungen genauso verstanden werden können, wie sie tatsächlich gemeint waren oder ob die Dokumentationsform einen unzulässig hohen Interpretationsspielraum zulässt: Ist der Text eindeutig formuliert? Auf welche Weise kann der Text noch verstanden werden? Gibt es Doppeldeutigkeiten in der Verwendung von Begriffen? Sind die richtigen Fachbegriffe verwendet worden? Wurde bei der Dokumentation eine angemessen präzise Ausdrucksweise gewählt?
- **Konformität zu Dokumentationsregeln:** Hierbei wird geprüft, ob die Anforderungen entsprechend den aktuellen Modellierungskonventionen oder sonstigen Vorschriften dokumentiert wurden. So gibt es häufig organisationsspezifische Vorgaben, z. B. nach welchem Schema oder in welcher Reihenfolge Anforderungen zu dokumentieren sind oder welche Modellierungssprachen für welchen Zweck eingesetzt werden: Entspricht die Dokumentation den aktuellen Vorschriften?
- **Konformität zu Dokumentationsformat:** Hierzu wird überprüft, ob die Anforderungen im vorgeschriebenen Format und unter Einsatz der vorgeschriebenen Anwendungen dokumentiert werden. Die Vorschriften dazu werden häufig organisationsspezifisch festgelegt: Wurden die Anforderungen in dem dazu vorgesehenen Format dokumentiert?

Prüfkriterien zur Abgestimmtheit

Mit den Kriterien zur Prüfung der Abgestimmtheit der Dokumentation soll die Frage „Stimmen alle relevanten Stakeholder den dokumentierten Anforderungen zu und sind die bekannten Konflikte gelöst?“ beantwortet werden. Insbesondere durch das Erlangen neuer Erkenntnisse im Verlauf des Softwareprozesses können neue Anforderungen identifiziert werden, die sich mit den bereits dokumentierten Anforderungen widersprechen.

- **Abgestimmtheit jeder Anforderung:** Zu diesem Kriterium wird geprüft, inwieweit die Menge der Anforderungen nach der Dokumentation bereits mit den Stakeholdern abgestimmt wurde: Wurde die Anforderung bereits mit den relevanten Stakeholdern abgestimmt? Gibt es Anforderungen, die nach der Dokumentation noch nicht mit den Stakeholdern besprochen wurden?
- **Abgestimmtheit nach Änderung von Anforderungen:** Hierbei wird geprüft, ob nach Änderungen der Dokumentation bereits abgestimmter Anforderungen erneut abgestimmt wurde. Das wird insbesondere dann erforderlich, wenn durch Abhängigkeiten zwischen Anforderungen Änderungen erfolgen, die ursprünglich nicht vorherzusehen waren.
- **Konflikte aufgelöst:** Hier wird geprüft, ob alle identifizierten Konflikte zwischen den Stakeholdern beseitigt und die Widersprüche in der Menge der dokumentierten Anforderungen aufgelöst wurden: Sind alle bekannten Konflikte gelöst? Sind alle bekannten Widersprüche behoben?
- **Freigabe erteilt:** Vor der Umsetzung der Anforderungen müssen diese explizit freigegeben werden. Im Unterschied zur fachlichen Abgestimmtheit bedeutet die Erteilung der Freigabe den Beschluss, dass auf Basis des aktuellen Stands mit der Umsetzung begonnen werden darf. Unter Umständen kann dies auch der Fall sein, wenn noch nicht alle Qualitätsaspekte erfüllt sind, mit der Umsetzung jedoch trotzdem schon begonnen werden sollte. Andererseits kann die Umsetzung von bereits fertig abgestimmten Anforderungen auch zurückgestellt werden, weil Anforderungen neu priorisiert wurden: Ist die Freigabe zur Umsetzung erteilt?

7.3 Prüfprinzipien

Im Rahmen der Prüfung von Anforderungen können verschiedene Prüfprinzipien unterschieden werden, die zwar selber noch keine Prüftechnik darstellen, bei der Auswahl der Prüftechniken jedoch helfen. Prüftechniken hingegen sind konkrete Verfahren oder Vorgehensweisen, die zur Durchführung der Prüfung eingesetzt bzw. genutzt werden. Eine Prüftechnik kann ein oder mehrere Prüfprinzipien unterstützen. Daher werden in diesem Lernzyklus zuerst die Prüfprinzipien vorgestellt, bevor konkrete Prüftechniken erläutert werden.

Für das Requirements Engineering eignen sich die im Folgenden beschriebenen sechs Prüfprinzipien zur Verbesserung der Qualität der Prüfergebnisse (Pohl/Rupp 2015, S. 106ff.):

Prinzip 1: Beteiligung der richtigen Stakeholder

Für die Überprüfung der Anforderungen sollten Prüfer und Ersteller am besten verschiedene Personen sein. Idealerweise werden die Anforderungen sogar von unabhängigen Personen geprüft, die als Prüfer im Rahmen ihrer Prüftätigkeit keine Interessenkonflikte haben. Die Auswahl der konkreten Prüfer soll daher in Abhängigkeit des Ziels der Prüfung und der zu überprüfenden Anforderungen erfolgen. Tabelle 29 stellt beispielhaft die Vor- und Nachteile eines externen Prüfers dar. Insbesondere zur Prüfung von speziellen fachli-

chen oder technischen Aspekten, bei denen das Projektteam nicht sehr erfahren ist, die jedoch als kritisch für das Projekt eingeschätzt werden, ist die Beteiligung eines externen Reviewers zu empfehlen.

Tabelle 29: Gegenüberstellung interner und externer Prüfer

Interner Prüfer	Externer Prüfer
+ leichter zu koordinieren	– hoher Aufwand
+ leichter zu organisieren	– kostenpflichtig
+ günstig	+ unabhängig
– ggf. beeinflusst	
→ frühe Prüfungsphasen	→ qualitativ hochwertige Anforderungen

Quelle: erstellt im Auftrag der IU, 2023.

Folgendes Anwendungsbeispiel illustriert die Auswahl der Reviewer in Abhängigkeit der aktuellen Projektsituation: Zur Überprüfung der Testbarkeit von Anforderungen können zunächst interne Tester prüfen. Wenn ein angemessener Qualitätsgrad hergestellt ist, können zur finalen Prüfung externe Tester hinzugezogen werden.

Prinzip 2: Trennung von Fehlersuche und Fehlerkorrektur

Die Grundidee hinter diesem Prinzip ist die Konzentration ausschließlich auf die Dokumentation der Mängel während der Prüfung. Erst nach der Prüfung erfolgt die Bewertung der vermeintlichen Fehler und deren Konsolidierung. Die Korrektur der Fehler erfolgt erst im Anschluss an die Konsolidierung. Damit soll sichergestellt werden, dass die eigentlich zur Prüfung eingeplante Zeit auch tatsächlich zum Prüfen und nicht zum Beheben genutzt wird.

Beispiel:

„In den Anforderungen F-R-1 und F-R-3 ist die Authentifizierungsmethode an einem System inkonsistent beschrieben.“ Dieser Mangel wird nur dokumentiert und nicht behoben, da es sein kann, dass es wirklich zwei Authentifizierungsmethoden gibt, dies jedoch erst später im Dokument erläutert wird.

Prinzip 3: Prüfung aus unterschiedlichen Sichten

Mit dem Prinzip der Prüfung aus unterschiedlichen Sichten soll erreicht werden, dass die dokumentierten Anforderungen aus möglichst verschiedenen Gesichtspunkten betrachtet und analysiert werden. Das Konzept der unterschiedlichen Sichten wird im Requirements Engineering sowohl als Ermittlungstechnik als auch beim perspektivenbasierten Lesen in der Prüfung eingesetzt.

Beispiel:

Beim Prüfen konzentriert sich der Anwender ausschließlich auf die Funktionalität, der Tester ausschließlich auf die Testbarkeit der Anforderungen und ein Vertreter der Anwender nur auf die Aspekte der Benutzbarkeit.

Prinzip 4: Geeigneter Wechsel der Dokumentationsform

Die Grundidee dieses Prüfprinzips ist die Übertragung von dokumentierten Anforderungen in eine andere Dokumentationsform. Während der Übertragung wird die ursprüngliche Dokumentation intensiv gelesen, analysiert und mögliche Fehler oder Lücken identifiziert. Ein typischer Anwendungsfall für den Wechsel der Dokumentationsform ist die Prüfung von als Text beschriebenen Abläufen durch die Übertragung in ein Prozessmodell oder ein UML-Aktivitätsdiagramm. In der Praxis hat sich gezeigt, dass bei der Formulierung von Abläufen oft das Verhalten im Fehlerfall, beispielsweise eine fehlgeschlagene Prüfung, nicht beschrieben wird.

Beispiel:

Ein in Textform beschriebener Ablauf zum Datenaustausch zwischen zwei Anwendungen wird als UML-Diagramm modelliert. Dann wird anhand des Diagramms geprüft, ob der Text die Anforderungen richtig wiedergibt.

Prinzip 5: Konstruktion von Entwicklungsartefakten

Mit der Konstruktion von Entwicklungsartefakten wird direkt die Verwendbarkeit der dokumentierten Anforderungen in den nachgelagerten Phasen des Softwareprozesses geprüft. Typische Anwendungsfälle sind die beispielhafte Konstruktion von Entwurfsartefakten oder die Erstellung konkreter Testfälle. Dabei werden Elemente der Anforderungsdokumentation auf die Weise eingesetzt, wie es tatsächlich in späteren Phasen geplant ist.

Beispiel:

Aus den fachlichen Anforderungen werden die Testfälle für den Akzeptanztest erstellt. Die Konstruktion von Entwicklungsartefakten ist sehr aufwendig und eignet sich daher insbesondere nur bei sehr risikoreichen Anforderungen.

Prinzip 6: Wiederholte Prüfung

Wie bereits beschrieben, erfolgt das Requirements Engineering zum einen iterativ, zum anderen lässt sich keine sinnvolle Reihenfolge der Kernaktivitäten zu Beginn eines Projekts planen. Daher ändert sich die Menge der dokumentierten Anforderungen in Abhängigkeit vom Erkenntnisgewinn der Stakeholder. Außerdem werden Abhängigkeiten zwischen einzelnen Anforderungen erst im Verlauf des Projekts erkannt, was auch zu Änderungen in der Dokumentation führt. Daher ist die einmalige erfolgreiche Prüfung von Anforderungen keine Garantie dafür, dass die Anforderungen zu einem späteren Zeitpunkt im Projekt immer noch einer Prüfung standhalten. Die wiederholte Prüfung von Anforderungen ist daher ein Prüfprinzip, das insbesondere dann relevant ist, wenn über die Lauf-

zeit des Projekts eine größere Menge an Anpassungen berücksichtigt werden muss. Außerdem sollten Anforderungen wiederholt geprüft werden, falls sie in ein anderes als das ursprünglich gedachte System übernommen werden sollen.

Beispiel:

Die Anforderungen des Antragssystems an den Datenaustausch mit dem Bestandssystem sollen in die Anforderungen des Inkassosystems übernommen werden. Hierzu ist eine erneute Prüfung der Anforderungen notwendig.

7.4 Prüftechniken

Da die dokumentierten Anforderungen Ausgangspunkt aller Aktivitäten in einem Softwareprojekt sind und das System selbst erst in späteren Phasen konstruiert wird, handelt es sich bei den Prüftechniken für Anforderungen grundsätzlich um Techniken zur statischen Analyse des Prüflings: In den meisten Fällen werden die Anforderungen von Menschen durchgelesen und bewertet. Die am häufigsten eingesetzten Prüftechniken sind Review-techniken, bei denen die Beteiligten bestimmte Rollen einnehmen und je nach Technik verschiedene Aufgaben haben. Je nach Projektsituation wird der Review mehr oder weniger stark durchorganisiert. Vergleichbar mit der Situation zur Qualitätssicherung von fertigen Softwaresystemen, ist es auch bei der Qualitätssicherung von Anforderungen in der Regel wirtschaftlich nicht sinnvoll, eine hundertprozentige Fehlerfreiheit erzielen zu wollen. Daher müssen auch bei der Qualitätssicherung von Anforderungen die zur Verfügung stehenden Ressourcen so eingesetzt werden, dass mit ihnen der größtmögliche Nutzen erzielt wird.

Aus diesen Gründen kann der verantwortliche Requirements Engineer auch bei der Bestimmung der Prüftechniken aus einer Vielzahl erprobter und bekannter Techniken die für die jeweilige Situation am besten Geeignetste auswählen.

Stellungnahme

Die einfachste Technik des Reviews ist eine informelle Stellungnahme. Dabei wird die Anforderungsdokumentation durch eine an der Dokumentation der Anforderungen unbeteiligten und unabhängigen Person gelesen und hinsichtlich der relevanten Prüfkriterien bewertet. Mängel werden dabei im Dokument markiert und kurz begründet. Die Stellungnahme kann beispielsweise durch einen Kollegen oder einen externen Dienstleister durchgeführt werden. Der Ablauf folgt dabei keinem bestimmten Schema oder Vorgaben.

Folgendes Szenario skizziert ein Beispiel für eine Stellungnahme: Das Anforderungsdokument wird an einen Testingenieur übergeben. Dieser prüft das Dokument auf Testbarkeit der Anforderungen, markiert die Mängel und gibt dem Anforderungsingenieur Feedback zur Verbesserung.

Walkthrough

Im Unterschied zur Stellungnahme wird das Dokument bei einem Walkthrough von mehreren Personen gelesen und die identifizierten Mängel in der Gruppe diskutiert. Bei einem Walkthrough müssen die Rollen Autor, Reviewer und Protokollant vergeben werden. Diese Prüftechnik läuft folgendermaßen ab:

1. Die dokumentierten Anforderungen werden an die Reviewer verteilt und die relevanten Prüfkriterien benannt.
2. Die Reviewer erhalten ausreichend Zeit, um das Dokument hinsichtlich der genannten Kriterien individuell zu prüfen und ggf. Mängel zu notieren.
3. Der Autor, alle Reviewer und ein Protokollant treffen sich zur Walkthrough-Sitzung. Dort stellt der Autor schrittweise die zu überprüfenden Anforderungen vor und erläutert diese. Je nach Größe der Gruppe und Disziplin der Teilnehmer kann auch ein Moderator eingebunden werden, der die Sitzung leitet.
4. Sobald der Autor an den betreffenden Anforderungen angekommen ist, stellen die Reviewer die gefundenen Mängel vor und erläutern und diskutieren diese untereinander. Neben den im Vorfeld identifizierten Mängeln können auch noch während der Sitzung Qualitätsmängel identifiziert werden.
5. Der Protokollant ist verantwortlich für das Notieren der Mängel, sodass die Autoren im Anschluss an die Sitzung das Dokument bei Bedarf überarbeiten können.

Ein formale Version des Walkthroughs ist eine Inspektion. Details zu dieser Prüftechnik werden in Pohl/Rupp (2015, S. 110ff.) beschrieben.

Perspektivenbasiertes Lesen

Das perspektivenbasierte Lesen ist eine Technik, die sich in Kombination mit einem Review (z. B. Stellungnahme oder Walkthrough) einsetzen lässt. Die Reviewer sollen das Dokument nicht aus einer generellen Perspektive lesen, sondern ihnen werden gezielte Perspektiven zugeteilt, aus der sie die Anforderungen bewerten sollen. Welche Perspektiven dabei gewählt werden, ist abhängig von der aktuellen Projektsituation. Beispielsweise können gezielte Rollenperspektiven vergeben werden: Einer liest aus der Sicht des Kunden, einer aus der Sicht des Testers und einer aus der Sicht des Softwarearchitekten.

Eine andere Möglichkeit ist die Orientierung der Perspektiven anhand der relevanten Prüfkriterien. Falls beispielsweise neun relevante Kriterien identifiziert wurden und drei Reviewer zur Verfügung stehen, können die Kriterien unter den Reviewern aufgeteilt werden. So muss nicht jeder Reviewer alle Aspekte im Auge behalten, sondern kann sich auf die ihm zugeteilten Kriterien konzentrieren. Die Konsolidierung der Fehler erfolgt dann im Anschluss gemeinsam, sodass sich die Reviewer auch untereinander zu den gefundenen Fehlern austauschen können.

Mit der Verteilung von solchen konkreten Reviewaufträgen wird eine Parallelisierung erreicht, mit der auch große Anforderungsdokumente effizient unter verschiedenen Gesichtspunkten überprüft werden können.

**TIPP**

Falls ein Anforderungsdokument von mehreren Personen gelesen werden soll, ist es hilfreich, sich vorher zu überlegen, ob sich sinnvolle Perspektiven identifizieren bzw. die Reviewaufträge aufteilen lassen.

Einsatz von Checklisten

Checklisten als Hilfsmittel zur Sicherstellung, dass im Rahmen einer Prüfung oder Abarbeitung keine Aufgabe vergessen wird, sind in vielen verschiedenen Bereichen des privaten Alltags, aber auch bei industriellen Abläufen ein wichtiges Hilfsmittel. So helfen Checklisten sowohl bei der Zusammenstellung des privaten Reisegepäcks bei langen Reisen oder Bergexpeditionen als auch Flugzeugpiloten bei der Durchführung des Systemchecks vor dem Start des Flugzeugs. Auch bei der Prüfung von Anforderungen können Checklisten eine wertvolle Unterstützung bieten, um bei komplexen Sachverhalten keine Aspekte zu vergessen.

Bei der Erstellung von Checklisten zur Qualitätssicherung von Anforderungen sind organisationsinterne Richtlinien und Vorgaben, relevante Prüfkriterien, Prüfprinzipien und die Erfahrung aus vergangenen Projekten von Nutzen. Je nach Verfügbarkeit von Checklisten können bestehende Checklisten wiederverwendet oder auch projektspezifische Checklisten erstellt werden. Insbesondere bei der wiederholten Prüfung von großen Dokumenten lohnt sich die projektspezifische Erstellung von Checklisten. Diese dient dem Reviewer beim Durcharbeiten der Anforderungen als Leitfaden, um den Fokus nicht zu verlieren und sicherzustellen, dass alle relevanten Fragen bzw. Prüfkriterien berücksichtigt wurden. Checklisten lassen sich gut mit dem perspektivenbasierten Lesen kombinieren. So können gezielt für jede Perspektive die relevanten Fragen formuliert und dokumentiert werden.

**TIPP**

Formulieren Sie auf einer Checkliste konkrete Fragen, die zusammen nicht länger als eine Seite sind. So hat der Reviewer alle Aspekte in einer für ihn verständlichen und eindeutigen Form immer im Blick.

Prüfung durch Prototypen

Eine andere Prüftechnik, die sich nicht nur auf das Lesen und Auswerten von dokumentierten Anforderungen beschränkt, ist die Prüfung unter Verwendung von Prototypen. Diese eignen sich nicht nur als Ermittlungstechnik, sondern auch explizit als Dokumentations-technik. Dabei lassen sich insbesondere beim Einsatz von Prototypen die Aktivitäten Ermittlung und Prüfung nicht immer klar voneinander abgrenzen. Die Grundidee des Einsatzes von Prototypen bei der Prüfung von Anforderungen besteht darin, durch die Kon-

struktion von Systemartefakten dem Stakeholder das eigene Verständnis der ermittelten Anforderungen zurückzuspiegeln. Anhand des Prototypen können die Stakeholder dann bewerten, ob ihre gemeinten Anforderungen auch so verstanden wurden, ohne bereits ein vollständiges System konstruieren zu müssen. Durch die relativ einfache Änderbarkeit von Prototypen können Prüfung, Abstimmung und Überarbeitung in mehreren Zyklen durchgeführt werden.

7.5 Abstimmen von Anforderungen

Falls sich widersprechende Anforderungen identifiziert wurden und diese nicht einfach behoben werden können, liegt eine Konfliktsituation vor. Mit den Aktivitäten zur Abstimmung von Anforderungen soll zum einen die Erreichung eines gemeinsamen und übereinstimmenden Verständnisses der Menge der Anforderungen unter den Stakeholdern erreicht werden (Pohl/Rupp 2015, S. 102). Zum anderen müssen aber auch Entscheidungen getroffen werden, welche Anforderungen tatsächlich umgesetzt werden sollen. Dabei soll mit den Mitteln des Konfliktmanagements der Konflikt aufgelöst und gleichzeitig die Kooperationsbereitschaft der Stakeholder aufrecht erhalten werden. Daher werden im Folgenden die typischen Aufgaben im Konfliktmanagement mit Bezug zum Requirements Engineering vorgestellt:

1. Konfliktidentifikation,
2. Bestimmung des Konflikttyps,
3. Konfliktauflösung sowie
4. Dokumentation der Konfliktauflösung.

Konfliktidentifikation

Bevor die Auflösung des Konflikts möglich ist, muss er zuerst identifiziert werden können. Konflikte im Requirements Engineering können grundsätzlich zu jeder Zeit erkannt werden: während der Ermittlung, der Dokumentation und der Prüfung von Anforderungen. Die Identifikation von Konflikten bei der Ermittlung erfordert eine hohe Aufmerksamkeit und Sensibilität des verantwortlichen Requirements Engineer. Widersprüchliche fachliche Anforderungen sind häufig nicht offensichtlich als solche zu erkennen. Die technischen Zusammenhänge und Abhängigkeiten kennt unter Umständen nur der Softwarearchitekt. Daher ist es empfehlenswert, bei der Analyse von Anforderungen auf Widersprüche auch den Softwarearchitekten miteinzubeziehen.

Bestimmung des Konflikttyps

Wurde das Vorliegen eines Konflikts erkannt, sollte zunächst analysiert werden, um was für einen Konflikt es sich handelt. Dazu können Konflikte verschiedenen Konflikttypen zugeordnet werden. Diese Zuordnung hilft dann im Anschluss bei der Auswahl der Lösungsstrategie. Dabei ist jedoch zu beachten, dass in der Praxis häufig ein Konflikt die Merkmale mehrerer Konflikttypen trägt. Daher lassen sich die Typen nicht immer scharf voneinander abgrenzen. Pohl/Rupp (2015, S. 118ff.) identifizieren unter anderem folgende Konflikttypen:

- **Sachkonflikt:** Typisch für einen Sachkonflikt ist das Vorliegen von falscher oder mangelnder Information bzw. eine unterschiedliche Interpretation durch die Stakeholder.
Beispiel: Ein Stakeholder stuft eine automatische Vervollständigung der Adresse als überflüssig ein, weil ihm Hintergrundinformationen zum konkreten Anwendungsfall fehlen.
- **Interessenkonflikt:** Haben Stakeholder unterschiedliche Interessen oder unterschiedliche Ziele bezüglich des Systems oder des Softwareprojekts, liegt ein Interessenkonflikt vor.
Beispiel: Während einige Stakeholder befürchten, dass mit dem zu bauenden System der eigene Arbeitsplatz wegfällt, wünschen sich andere Stakeholder eine schnellere Bearbeitung durch die Unterstützung von IT-Systemen.
- **Wertekonflikt:** Ein Wertekonflikt ist durch individuelle Ideale und kulturelle Unterschiede gekennzeichnet.
Beispiel: Ein Stakeholder beharrt auf den Einsatz eines ganz bestimmten Betriebssystems, weil er es gut kennt und für das Beste hält.
- **Beziehungskonflikt:** Ein Beziehungskonflikt besteht, wenn Stakeholder untereinander persönliche Konflikte haben, die nicht unmittelbar mit dem System zusammenhängen, sich jedoch in der Art der Kommunikation äußern. Typische Merkmale sind eine schlechte Kommunikation, Missachtung, Beleidigungen und starke Emotionen.
Beispiel: Zwei Stakeholder, die privat zerstritten sind, tragen ihren Konflikt durch ihr Verhalten mit in das Projekt hinein.
- **Strukturkonflikt:** Ein Strukturkonflikt liegt vor, wenn die Konfliktparteien unterschiedliche Macht bzw. Entscheidungsbefugnisse haben.
Beispiel: Der Vorgesetzte entscheidet über Anforderungen, ohne Rücksicht auf die Bedürfnisse und Wünsche der Mitarbeiter zu nehmen.

Konfliktauflösung

Nachdem ein identifizierter Konflikt auf seine Eigenschaften hin analysiert wurde, muss er aufgelöst werden. Eine wesentliche Herausforderung bei der Konfliktauflösung ist die Erhaltung der Kooperationsbereitschaft der vom Konflikt betroffenen Stakeholder. Daher ist es in jedem Fall wichtig, alle relevanten Stakeholder bei der Lösung miteinzubeziehen, nicht zuletzt, um eine Akzeptanz der erreichten Lösung bei allen Beteiligten zu erreichen.

Im Folgenden wird eine Auswahl möglicher Lösungsstrategien vorgestellt, die zum größten Teil in Pohl/Rupp (2015, S. 120ff.) vorgestellt werden:

- **Einigung:** Der Konflikt wird durch den Austausch von Informationen, Argumenten und deren Diskussion gelöst. Die Konfliktparteien einigen sich auf eine Lösung und beschreiben diese.
- **Abstimmung:** Alle relevanten Stakeholder stimmen ab, und es wird die Lösung umgesetzt, welche die vorher festgelegte erforderliche Stimmenanzahl erhält.
- **Ober-sticht-Unter:** Die Konfliktpartei mit der stärkeren Macht bzw. der höheren Autorität entscheidet.
- **Pro-Contra-Liste:** Zu den widersprüchlichen Anforderungen werden jeweils deren Vor- und Nachteile untersucht und gegenübergestellt.

- **Werteorientierte Analyse:** Zu jeder der widersprüchlichen Positionen wird ermittelt, welcher Wert durch deren Umsetzung geschaffen wird. Anschließend wird diejenige verwirklicht, deren Umsetzung den größten Wert für das Unternehmen bringt.
- **Zurückstellen der Lösung:** Diese Strategie löst den Konflikt nicht, sondern verschiebt die Lösung auf einen späteren Zeitpunkt. Der Ansatz ist sinnvoll, wenn abzusehen ist, dass zukünftig noch ein deutlicher Erkenntnisgewinn erfolgt, der eine einfache Lösung des Konflikts ermöglicht. Außerdem kann mit der Zurückstufung von Konflikten zu unwichtigen Anforderungen verhindert werden, dass sich das Projektteam zu lange an unbedeutenden Kleinigkeiten aufhält.

Dokumentation der Konfliktauflösung

Der verantwortliche Requirements Engineer muss im Anschluss an die Konfliktauflösung darauf achten, dass der Konflikt, dessen Lösung und das Vorgehen zur Lösungsfindung geeignet dokumentiert werden. Nur so kann verhindert werden, dass der gleiche Konflikt später im Projektverlauf wieder auftaucht und dann noch einmal gelöst werden muss. Falls der Konflikt aber doch wieder entsteht, kann er mit dem Verweis auf die bereits gefundene Lösung schnell wieder beendet werden. Zum anderen kann sich zu einem späteren Zeitpunkt herausstellen, dass die gefundene Lösung nicht geeignet ist. Anhand der Dokumentation kann in diesem Fall die Entscheidungsfindung nachvollzogen und bei der Erarbeitung einer anderen Lösung berücksichtigt werden.



ZUSAMMENFASSUNG

Das Ziel der Aktivitäten zum Prüfen und Abstimmen ist die Freigabe der Anforderungen zur Umsetzung in den nachgelagerten Phasen des Softwareprojekts. Vor der Weiterverarbeitung der ermittelten und dokumentierten Anforderungen muss jedoch deren Qualität gesichert werden, da sich Fehler aus Anforderungsdokumenten über alle nachfolgenden Aktivitäten fortpflanzen.

Im Vorfeld der Prüfung ist festzulegen, nach welchen Kriterien genau geprüft werden soll. Mithilfe der Qualitätsaspekte Inhalt, Dokumentation und Abgestimmtheit wird die Menge der Prüfkriterien strukturiert, sodass man bei der Auswahl der Prüfkriterien zuerst die relevanten Qualitätsaspekte identifiziert. Anschließend werden aus der Menge der Prüfkriterien eines Qualitätsaspekts die relevanten Kriterien ausgewählt. Danach wählt der verantwortliche Requirements Engineer aus der Menge der Prüftechniken die aus, die für die jeweilige Situation am besten geeignet sind.

Die am häufigsten eingesetzten Prüftechniken sind Reviewtechniken, bei denen die Anforderungen von Menschen durchgelesen und bewertet werden. Die Techniken perspektivenbasiertes Lesen und Checklisten unterstützen bei der Durchführung des Reviews. Falls sich widersprechende Anforderungen identifiziert wurden, müssen diese mit allen rele-

vanten Stakeholdern abgestimmt werden. Dazu wird mit den Mitteln des Konfliktmanagements der Konflikt aufgelöst und gleichzeitig darauf geachtet, dass die Kooperationsbereitschaft der Stakeholder aufrecht erhalten wird.

LEKTION 8

MANAGEMENT VON ANFORDERUNGEN UND TECHNIKEN ZUR PRIORISIERUNG

LERNZIELE

Nach der Bearbeitung dieser Lektion werden Sie wissen, ...

- welche Eigenschaften Anforderungen haben.
- wie Sichten auf Anforderungen erzeugt werden können.
- was Versionierung von Anforderungen bedeutet.
- welche Techniken zur Anforderungspriorisierung genutzt werden können und wie diese funktionieren.
- was unter dem Lebenszyklus von Anforderungen verstanden wird.

8. MANAGEMENT VON ANFORDERUNGEN UND TECHNIKEN ZUR PRIORISIERUNG

8.1 Verwalten von Anforderungen

Software Engineering ist ein erkenntnisgetriebener Prozess. Das Wissen der Stakeholder über das System und die Erfahrung reift und verändert sich im Verlauf der Entwicklung, insbesondere immer dann, wenn neue Systemversionen bereitgestellt werden. Daher entwickeln sich Anforderungen im Laufe eines Projekts weiter. Aus diesem Grund haben Anforderungen unterschiedliche Status, die je nach Stabilität der Anforderung variieren. Um zu überprüfen, ob die Funktionalität und die Qualität korrekt umgesetzt wurden, muss nachvollziehbar sein, welche Anforderungen welche Features und welche Qualitätsaspekte definieren. Ein weiterer Aspekt, warum Anforderungen nachvollziehbar sein müssen, ist, dass sie die Grundlage eines Vertrags bilden können. Durch die Anforderungen wird festgelegt, was im Rahmen der Systementwicklung geliefert wird. Dabei besitzen Anforderungen unterschiedliche Relevanz für den Erfolg des Systems. Manche gehören zur grundlegenden Funktionalität, und ihre Abwesenheit kann zur Unzufriedenheit der Nutzer mit dem System führen. Auch explizit geforderte Anforderungen, die nicht oder falsch umgesetzt wurden, können zu mangelnder Akzeptanz des Systems führen. Das Management von Anforderungen wird auch Verwalten von Anforderungen genannt und von einigen Autoren als vierte Kernaktivität beschrieben (vgl. z. B. Pohl 2008). Das Management von Anforderungen ist wichtig, da die Anforderungen im ersten Schritt des Softwareprozesses ermittelt werden und so die Grundlage für die weiteren Aktivitäten bilden. Die Herausforderung bei der Verwaltung von Anforderungen ist, dass in der Praxis oft mehrere hundert bis mehrere tausend Anforderungen in einem Projekt ermittelt, dokumentiert, geprüft, abgestimmt und verwaltet werden. Zudem werden Softwaresysteme über Jahre hinweg durch verschiedene Personen immer wieder angepasst und weiterentwickelt. Daher müssen Anforderungen nicht nur zum Zeitpunkt der Erstellung eines Systems, sondern über dessen gesamten Lebenszyklus verwaltet werden.

Eigenschaften von Anforderungen

Um Anforderungen über den kompletten Softwareprozess nachvollziehbar zu machen, müssen Anforderungen gewisse Eigenschaften besitzen. Diese Eigenschaften werden als Attribute zu einer Anforderung notiert. Attribute können je nach Menge der Anforderungen händisch oder rechnergestützt verwaltet und als Metadaten von Anforderungen bezeichnet werden. Typische Attribute sind ...

- ... ein eindeutiger „Identifizierer (ID)“,
- ein „Name“ und
- eine „Beschreibung“ einer Anforderung.
- Anforderungen werden von einem „Autor“ erstellt und
- stammen aus einer „Quelle“.

- Anforderungen entwickeln sich bis zu einem gewissen Grad weiter, daher ist eine „Versionsnummer“ eine Möglichkeit, diese Entwicklung zu dokumentieren.
- Kombiniert mit dem Attribut „Stabilität“ besteht die Möglichkeit, den Status einer Anforderung zu dokumentieren und nachvollziehbar zu machen. Typische Zustände sind z. B.
 - aufgenommen: Die Anforderung ist ermittelt und dokumentiert, aber nicht abgestimmt.
 - in Abstimmung: Die Anforderung befindet sich im Abstimmungsprozess, ist jedoch noch nicht abgestimmt, da z. B. noch Konflikte aufgelöst werden müssen.
 - umgesetzt: Die Anforderung ist bereits implementiert.
- Die „Kritikalität“ und die „Priorität“ geben Auskunft über die Wichtigkeit einer Anforderung.

Sichten auf Anforderungen

Da Anforderungen in großer Zahl für ein System definiert werden und die Informationen zu einer Anforderung sehr umfangreich sein können, werden Sichten auf Anforderungen erzeugt. Eine Sicht auf eine Menge von Anforderungen dient immer einem bestimmten Darstellungszweck (z. B. das Zeigen aller abgestimmten Anforderungen) und kann speziell für einen bestimmten Adressaten erstellt werden (z. B. das Management). Werkzeuge zur Verwaltung von Anforderungen erzeugen diese Sichten automatisch durch sogenannte Filter. Diese Filter können jedoch auch manuell angewendet werden. Im Folgenden werden die selektive und die verdichtende Sicht erläutert.

Eine selektive Sicht auf Anforderungen stellt nur einen Teil der gesamten zu einer Anforderung verfügbaren Informationen dar. Es können beispielsweise bestimmte Anforderungen ausgewählt oder einzelne Attribute ausgeblendet werden. Tabelle 30 zeigt eine Ausgangstabelle, in der die Anforderungen durch ihre Attribute beschrieben wurden. Tabelle 31 zeigt dann eine solche selektive Sicht. Dazu wurden aus der oberen Tabelle alle Anforderungen, die durch den Autor „Meier“ und mit der Stabilität „fest“ definiert sind, ausgewählt.

Tabelle 30: Beispiel für eine tabellarische Darstellung von Anforderungsattributen (1)

ID	Anf. 1	Anf. 2	Anf. 3
Name	„Tastatureingabe“ ...	„Spracheingabe“ ...	„Empfang von“ ...
Kurze Beschreibung	„Das System“ ...	„Das System“ ...	„Das System“ ...
Autor	Meier	Müller	Müller
Quelle	PM	Altsystem	Altsystem
Verantwortlicher	Schulze	Schulze	Schulze
Stabilität	fest	gefestigt	gefestigt
Status Inhalt	Konzept	Idee	Idee
Status Überprüfung	ungeprüft	geprüft	In Prüfung

ID	Anf. 1	Anf. 2	Anf. 3
Querbezüge	Anf. 2; Anf. 5	Anf. 1; Anf. 5	Anf. 21; Anf. 7

Quelle: erstellt im Auftrag der IU, 2023.

Tabelle 31: Beispiel für eine tabellarische Darstellung von Anforderungsattributen (2)

ID	Anf. 4	Anf. 5	...
Name	„Ferndiagnose“	„Eingabe von“ ...	...
Kurze Beschreibung	„Das System“ ...	„Das System“ ...	...
Autor	Müller	Wagner	...
Quelle	Kunde	Kunde	...
Verantwortlicher	Schulze	Schulze	...
Stabilität	volatil	volatil	...
Status Inhalt	detail. Inhalt	Konzept	...
Status Überprüfung	geprüft	ungeprüft	...
Querbezüge	Anf. 6; Anf. 7	Anf. 1; Anf. 2	...

Quelle: erstellt im Auftrag der IU, 2023.

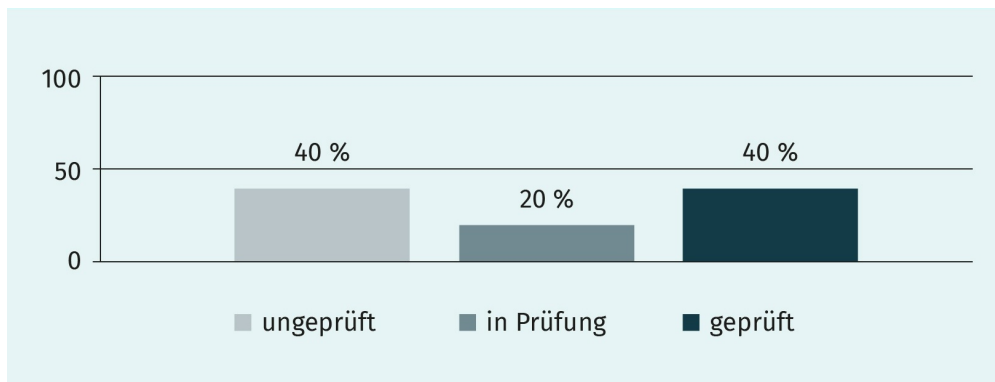
Tabelle 32: Beispiel für eine selektive Sicht auf Anforderungen

ID	Anf. 1
Name	„Tastatureingabe“ ...
Kurzbeschreibung	„Das System“ ...
Autor	Meier
Quelle	PM
Verantwortlicher	Schulze
Stabilität	fest
Status Inhalt	Konzept
Status Überprüfung	ungeprüft
Querbezüge	Anf. 2; Anf. 5

Quelle: erstellt im Auftrag der IU, 2023.

Eine weitere Sicht auf Anforderungen ist die verdichtende Sicht. Anders als bei der selektiven Sicht werden hier die Daten über die Anforderungen aggregiert. Dadurch können Aussagen über alle Anforderungen getroffen werden (z. B.: Wie viele Anforderungen wurden bereits umgesetzt?). Es ist jedoch auch möglich, durch eine Kombination von selektiver und verdichtender Sicht Aussagen über eine Teilmenge aller Anforderungen zu treffen. Abbildung 39 zeigt eine solche verdichtende Sicht, in ihr ist die Verteilung des Validierungsstatus aller Anforderungen aus Tabelle 30 dargestellt.

Abbildung 39: Beispiel einer verdichtenden Sicht auf Anforderungen

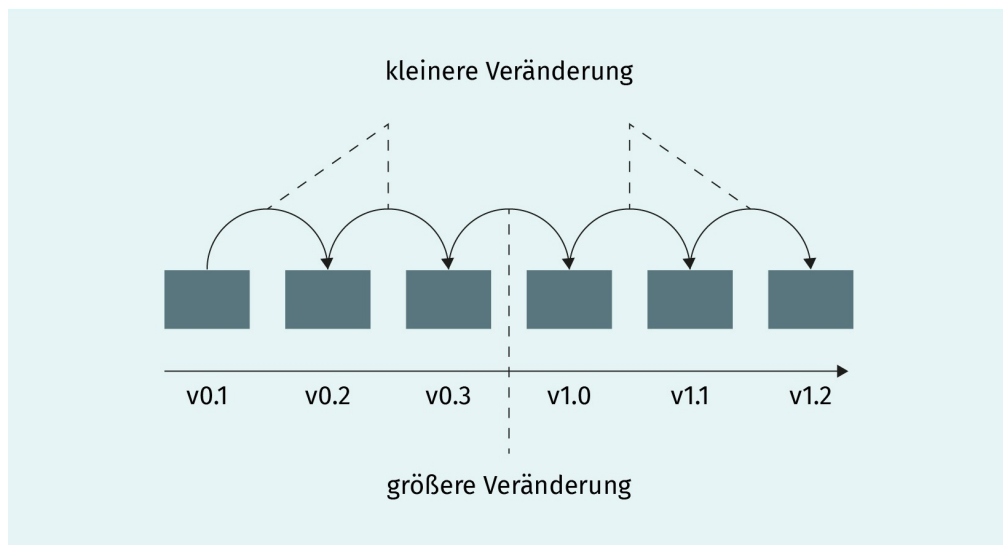


Quelle: erstellt im Auftrag der IU, 2023.

Versionierung von Anforderungen

Die Versionierung von Anforderungen hilft bei der Nachvollziehbarkeit von Änderungen und ermöglicht es dem Requirements Engineer, auf spezifische Änderungszustände zuzugreifen. Die Identifikation geschieht über eine eindeutige Versionsnummer, welche bei kleineren Änderungen im Dezimalbereich inkrementiert wird. Bei größeren Veränderungen wird eine Stelle vor dem Komma inkrementiert (vgl. Abbildung 40).

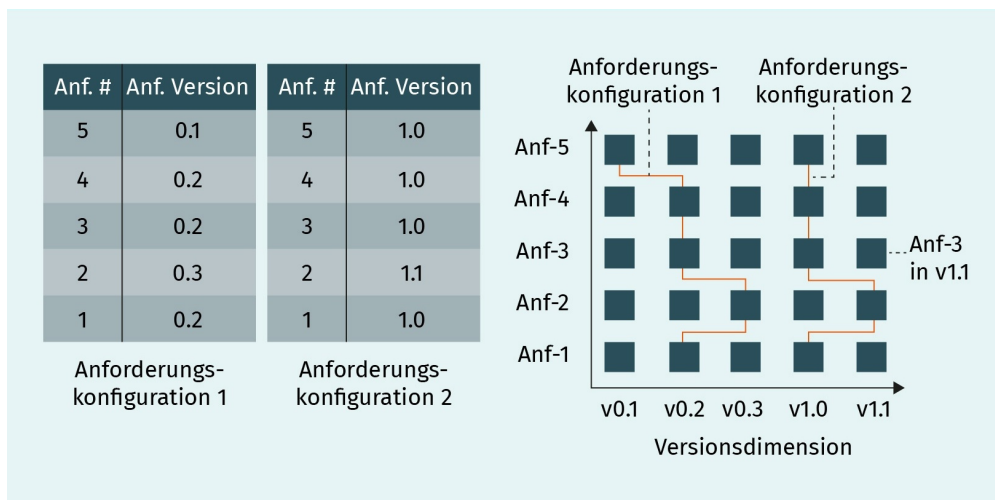
Abbildung 40: Versionierung von Anforderungen



Quelle: erstellt im Auftrag der IU, 2023.

Zu einem Zeitpunkt im Entwicklungsprozess hat jede Anforderung eine eindeutige Versionsnummer. Diese Zusammensetzung von Anforderungen mit eindeutiger Versionsnummer wird Anforderungskonfiguration genannt. Die Beschreibung einer Anforderungskonfiguration besteht aus zwei Dimensionen. Die Produktdimension zeigt einzelne Anforderungen der Anforderungsbasis (vgl. vertikale Achse in Abbildung 41), und die Versionsdimension zeigt unterschiedliche Änderungsstände der Anforderungen (vgl. horizontale Achse in Abbildung 41). Eine bestimmte Anforderungskonfiguration setzt sich aus einer Kombination von Anforderungen in bestimmten Versionen zusammen, dabei muss nicht jede Anforderung in einer Konfiguration vorhanden sein. Eine Anforderungskonfiguration ist immer widerspruchsfrei, lässt sich eindeutig identifizieren und ist nicht veränderbar. Wenn eine Konfiguration verändert wird, entsteht zwangsläufig eine neue Konfiguration.

Abbildung 41: Beispiel für Anforderungskonfigurationen

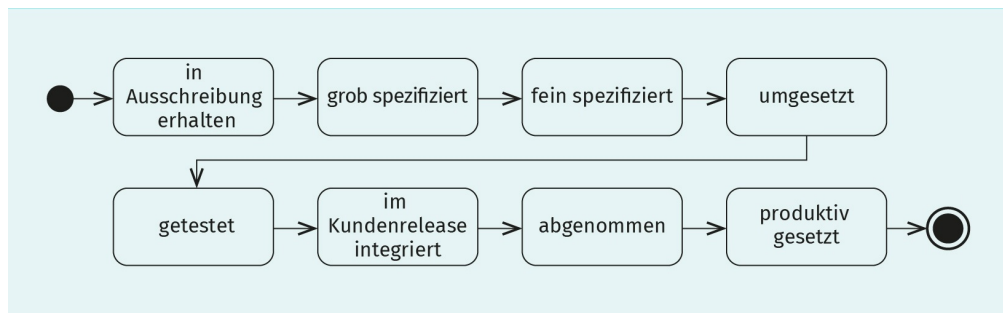


Quelle: erstellt im Auftrag der IU, 2023.

Lebenszyklus von Anforderungen im Softwareprozess

Anforderungen bestimmen den funktionalen Umfang eines Systems und definieren Qualitätsaspekte und Randbedingungen eines jeden Softwareentwicklungsprojekts. Alle nachgelagerten Phasen im Softwareprozess basieren auf den Anforderungen. Daher ist es wichtig, die Entwicklung von Anforderungen im Softwareprozess nachvollziehen zu können. Auf Basis der Anforderungen können Aufwände geschätzt werden oder kann überprüft werden, ob die geforderte Funktionalität umgesetzt wurde. Anforderungen haben also einen Lebenszyklus, in dem sie unterschiedliche Zustände annehmen. Zunächst sind Anforderungen meist in einer Ausschreibung enthalten, d. h., sie sind als Projektvision abstrakt formuliert. In der Regel werden die Anforderungen zunächst grob spezifiziert, also erhoben und dokumentiert. Anschließend werden die Anforderungen mit der höchsten Priorität verfeinert und im Detail spezifiziert. Erforderlich ist dabei der Detailgrad, den das Entwicklungsteam benötigt, um die Anforderungen zu verstehen und umzusetzen. Die dokumentierten Anforderungen bilden die Grundlage des zu liefernden Ergebnisses. Auf Basis der Spezifikation können Testfälle erstellt werden, die zur Überprüfung der Umsetzung der Anforderungen dienen. Sind die Anforderungen implementiert, d. h. umgesetzt, werden die Testfälle ausgeführt und das Ergebnis dokumentiert. Bei nicht ausreichender Testabdeckung müssen die gefundenen Fehler korrigiert und die Testfälle erneut durchgeführt werden. Nach erfolgreicher Testabnahme werden die Features und damit die Anforderungen in das System integriert. Sobald die Software ausgeliefert, also eine Systemversion beim Kunden installiert wird, sind die Anforderungen Bestandteil des Kundenreleases. Dieser wird zunächst in einer Testumgebung installiert und erst nach erfolgreicher Abnahme durch den Kunden produktiv gesetzt. Abbildung 42 zeigt eine vereinfachte Darstellung der beschriebenen Zustände von Anforderungen. Auf Entscheidungen und alternative Zustände wurde dabei bewusst verzichtet.

Abbildung 42: Vereinfachte Darstellung der Zustände im Lebenszyklus von Anforderungen



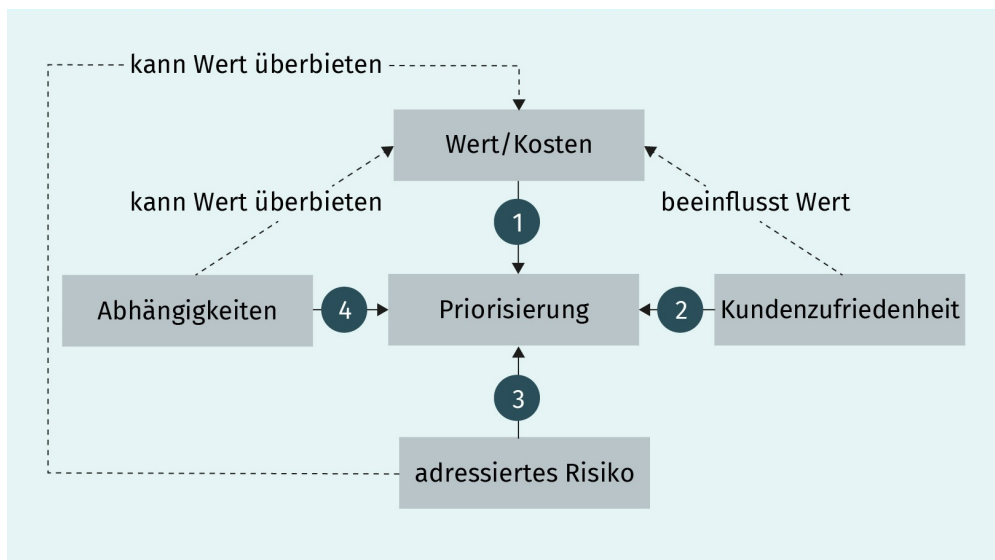
Quelle: erstellt im Auftrag der IU, 2023.

8.2 Techniken zur Priorisierung von Anforderungen

Eine typische und projektbegleitende Aufgabe ist die Priorisierung von Anforderungen und das Festlegen, welche Aktivitäten im Projekt als nächstes erfolgen sollen. Das ist bei Softwareprojekten nicht nur anfangs relevant, wenn eine erste Priorisierung der zu Beginn des Projekts bekannten Anforderungen durchgeführt wird. Im Verlauf eines Projektes ändern sich in der Regel noch die Menge der fachlichen Anforderungen und manchmal auch die technischen oder organisatorischen Rahmenbedingungen. Bei allen agilen Prozessen werden die Anforderungen beispielsweise in der Vorbereitung und Durchführung der Sprintplanung neu priorisiert. In den letzten Projektphasen müssen außerdem oft aus der Liste der bekannten Fehler die wichtigsten identifiziert werden, die mit den noch zur Verfügung stehenden Ressourcen behoben werden müssen. In jeder dieser Situationen müssen Anforderungen priorisiert werden. Es ist also jedes Mal neu zu bestimmen, in welcher Reihenfolge welche der Aufgaben umgesetzt werden.

Um die Priorität einer Anforderung zu bestimmen, müssen verschiedene Faktoren in Betracht gezogen werden. Der Graph in Abbildung 43 zeigt Faktoren zur Festlegung der Priorität einer Anforderung und wie sich diese gegenseitig beeinflussen. Der finanzielle Wert einer Anforderung drückt den Beitrag zum Geldverdienst bzw. zur Kostenreduktion aus. Die Kosten (z. B. für Entwicklungsaufwand, Hardware, kostenpflichtige Dienste, Lizenzen oder Support) beeinflussen die Priorität genauso wie das entstehende Risiko, wenn eine Anforderung gar nicht oder nicht ausreichend umgesetzt und getestet wird. Darüber hinaus können Abhängigkeiten zwischen Anforderungen die Reihenfolge der Bearbeitung beeinflussen. Die priorisierenden Faktoren abzuwägen, ist die Aufgabe des Requirements Engineer. Wichtiger Faktor zur Bestimmung der Priorität einer Anforderung ist die Kundenzufriedenheit, welche beispielsweise in Kano-Typen ausgedrückt werden kann.

Abbildung 43: Einflussfaktoren auf die Priorisierung von Anforderungen



Quelle: erstellt im Auftrag der IU, 2023 in Anlehnung an Wirdemann 2011, S. 110.

Alle vier genannten Faktoren sollten gleichen Einfluss auf die Priorität haben, jedoch gibt es spezifische Konstellationen der Faktoren, die dazu führen können, dass der Wert und die Kosten beeinflusst werden bzw. eine untergeordnete Rolle spielen. Wenn das Risiko einer Anforderung höher ist als deren Wert, hat das Risiko Priorität. Gesetze bergen ein Risiko für das Projekt, wenn diese nicht oder falsch in einem Informationssystem umgesetzt werden. Das Risiko ist in diesem Fall stärker zu gewichten als der Wert einer anderen Anforderung. Die Kundenzufriedenheit hat wesentlichen Einfluss auf den Wert einer Anforderung. Akzeptiert ein Stakeholder das System nicht, weil ihm grundlegende Funktionalität fehlt, sinkt der Wert des Systems. Abhängigkeiten können die Priorität so beeinflussen, dass statt der Umsetzung von Anforderungen mit dem höchsten Wert die Anforderungen in einer durch die Abhängigkeit festgelegten Reihenfolge umgesetzt werden. Beispielsweise kann das Speichern eines Vertrags nicht sinnvoll implementiert werden, bevor das System nicht in der Lage ist, Verträge zu erzeugen.

MoSCoW-Methode

Beim Einsatz der sogenannten MoSCoW-Methode werden Anforderungen in eine von vier Prioritätskategorien eingeordnet:

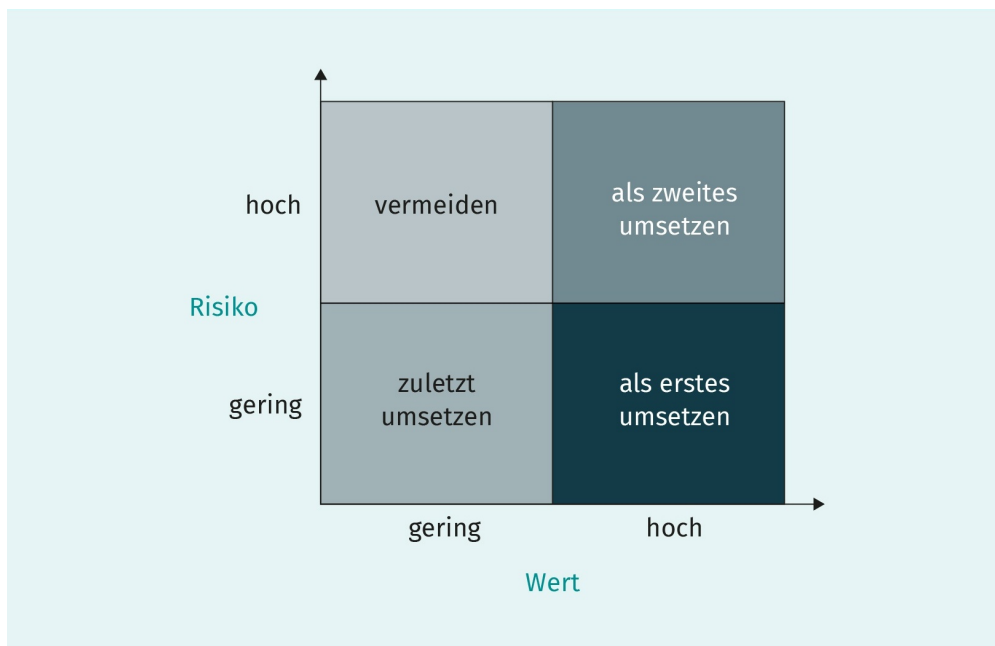
- **Muss** (engl. must): Anforderungen in dieser Kategorie sind essenziell und nicht verhandelbar. Für einen erfolgreichen Projektabschluss müssen sie umgesetzt werden.
- **Soll** (engl. should): Anforderungen dieser Kategorie sind zwar nicht kritisch, jedoch spielen sie für den Erfolg des Projekts eine sehr große Rolle.
- **Kann** (engl. could): Anforderungen in dieser Kategorie sind für den Projekterfolg nicht relevant und werden dann berücksichtigt, falls alle Soll-Anforderungen bearbeitet sind.
- **Nicht umsetzen** (engl. won't): Anforderungen dieser Kategorie werden momentan nicht umgesetzt, jedoch vielleicht im späteren Verlauf des Projekts.

Mithilfe dieser vier Kategorien lässt sich eine große Menge von Anforderungen mit wenigen Personen relativ schnell kategorisieren. Jedoch zeigt die Erfahrung insbesondere bei der Einbeziehung von Anwendern und Auftraggebern bei der Priorisierung, dass am Ende fast alle Anforderungen als Muss- und Soll-Anforderungen kategorisiert werden. Falls es mehr als eine Muss-Anforderung gibt, müssen alle Muss-Anforderungen priorisiert werden, damit klar ist, mit welcher denn nun das Projekt gestartet wird. Zu einer detaillierten Priorisierung eignet sich z. B. die im Folgenden vorgestellte Technik der Wert-Risiko-Matrix.

Wert-Risiko-Matrix

Eine einfache Technik zur Priorisierung von Anforderung ist die Wert-Risiko-Matrix. Bei dieser werden die Kundenzufriedenheit und Abhängigkeiten zwischen Anforderungen nicht explizit mit berücksichtigt. Nur der Wert, der durch die Umsetzung der Anforderungen geschaffen wird und das Risiko, dass es bei der Umsetzung zu großen Problemen kommt, führen zu ihrer Priorität. Der Wert wird dem Risiko gegenübergestellt. Die Granularität, in der Wert und Risiko beziffert werden, kann variabel gestaltet werden. Je mehr Abstufungen in der Bewertung erfolgen, desto genauer lässt sich die Priorität einer Anforderung bestimmen, desto höher ist jedoch auch der Aufwand, die exakte Kategorie zu bestimmen und desto größer ist die Gefahr einer Fehleinschätzung. Zur Erläuterung der Wert-Risiko-Matrix soll im Folgenden die Abstufung geringer und hoher Wert sowie geringes und hohes Risiko dienen (vgl. Abbildung 44). Zur Bestimmung der Priorität einer Anforderung muss jede Anforderung genau in ein Feld eingeordnet werden. Die Priorisierung ergibt sich dann wie folgt: Anforderungen mit geringem Wert und hohem Umsetzungsrisiko sind zu vermeiden; ist auch das Risiko gering, so sind diese Anforderungen als letztes umzusetzen. Die Anforderungen mit hohem Wert und geringem Umsetzungsrisiko erzeugen einen schnellen Wertzuwachs und sind daher als erstes zu bearbeiten. Anforderungen, die hohen Wert und hohes Umsetzungsrisiko bergen, sind unmittelbar als zweites umzusetzen. Der Vorteil der Wert-Risiko-Matrix ist die Einfachheit. Demgegenüber steht der Nachteil, dass Kundenzufriedenheit und Abhängigkeiten nicht explizit mit berücksichtigt werden. Darüber hinaus besteht auch hier die Möglichkeit, dass viele Anforderungen einen hohen Wert haben und ein hohes Risiko bergen, sodass nicht entschieden werden kann, mit welchen Anforderungen begonnen werden soll. Um die wichtigsten Anforderungen detaillierter zu priorisieren, können diese nach Einfluss auf die Kundenzufriedenheit anhand der Kano-Typen klassifiziert werden.

Abbildung 44: Wert-Risiko-Matrix



Quelle: erstellt im Auftrag der IU, 2023.

Kundenzufriedenheit nach Kano

Das Kano-Modell klassifiziert Anforderungen hinsichtlich der Wichtigkeit für Stakeholder. Dabei unterscheidet es nach Basis-, Leistungs- und Begeisterungsfaktoren. Basisfaktoren werden von Stakeholdern als selbstverständlich vorausgesetzt und führen zu extremer Unzufriedenheit, wenn diese nicht umgesetzt werden. Leistungsfaktoren sind grundlegende, aber explizit geforderte Anforderungen, deren Umsetzung zur Zufriedenheit der Stakeholder führt. Begeisterungsfaktoren sind unerwartete Anforderungen, die der Stakeholder als angenehme Überraschung wahrnimmt und ihn begeistern. Tabelle 31 gibt einen Überblick über die Kano-Typen und jeweils ein Beispiel.

Tabelle 33: Kano-Typen

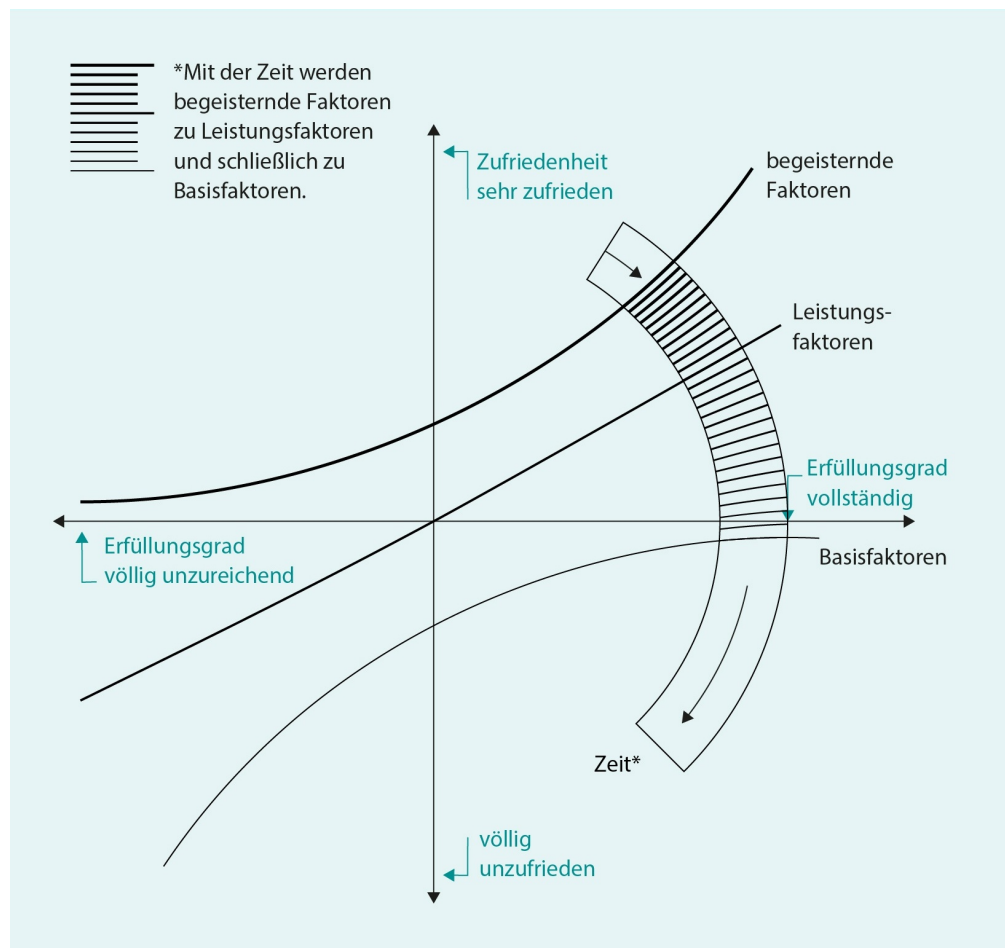
Bezeichnung	Beschreibung	Beispiel
Basisfaktor	selbstständig vorausgesetzte Systemmerkmale, deren Nichterfüllung zu extremer Unzufriedenheit führt	Undo-Funktion
Leistungsfaktor	explizit geforderte Systemmerkmale, die zu Zufriedenheit führen	Drag-and-Drop
Begeisterungsfaktor	Systemmerkmale, die der Nutzer nicht kennt und als angenehm nützliche Überraschung wahrnimmt	zuverlässige Sprachsteuerung

Quelle: erstellt im Auftrag der IU, 2023.

Die Definition, was Basis-, Leistungs- und Begeisterungsfaktoren sind, verändert sich im Laufe der Zeit, denn Stakeholder gewöhnen sich an Funktionalität und setzen diese nach gewisser Zeit als selbstverständlich voraus.

Als Beispiel soll die Undo-Funktion dienen. Diese wurde 1977 erstmal von Lance A. Miller in dem Paper „Behavioral Issues in the Use of Interactive Systems“ (Blaser/Hackl 1977) vorgeschlagen. Kurz nachdem die Undo-Funktion Teil von Programmen war, wurde diese als echte Innovation wahrgenommen, da sie die Arbeit merklich erleichtert. Kurze Zeit nachdem sich die Undo-Funktion als fester Bestandteil von Systemen etabliert hat, wurde diese explizit gefordert (Leistungsfaktor). Heutzutage wird kein Stakeholder die Undo-Funktion explizit fordern, sondern die Implementierung voraussetzen, sie ist also ein Basisfaktor geworden. Die Evolution von Anforderung und deren Einfluss auf die Kundenzufriedenheit bildet Abbildung 45 ab.

Abbildung 45: Evolution von Anforderungen nach Kano



Quelle: Rupp 2014, S. 82.

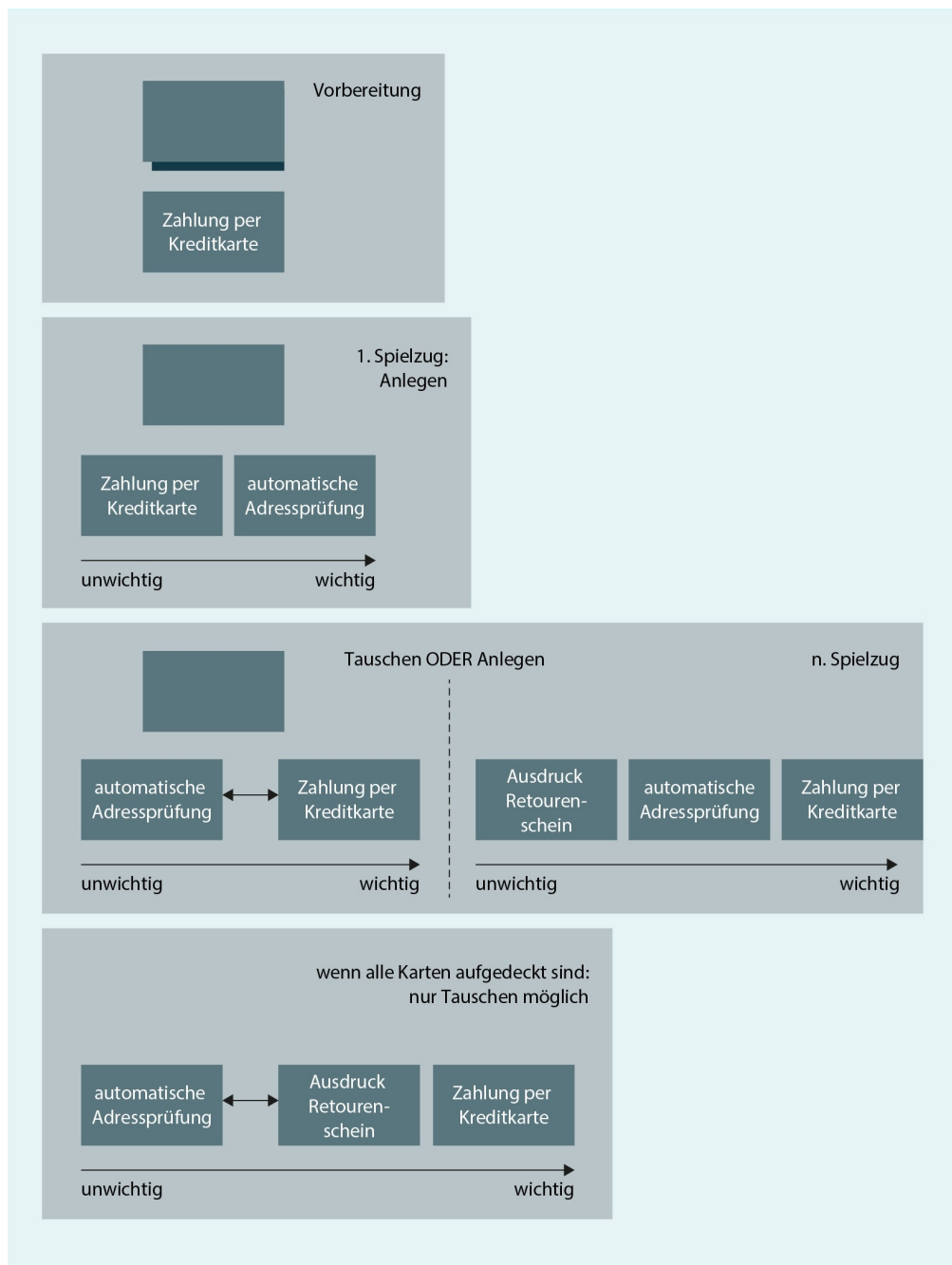
Team Estimation Game

Bei der konkreten Planung von Anforderungen hilft die Priorisierung in Form einer linearen totalen Ordnung. Umgangssprachlich ausgedrückt bedeutet das, dass alle Anforderungen vollständig durchnummeriert werden können und keine Nummer mehr als einmal vergeben wird. Die Anforderung mit der kleinsten Nummer hat die höchste Priorität, alle folgenden Prioritäten ergeben sich aus der aufsteigenden Reihenfolge. Mit einer auf diese Weise geordneten Liste von Aktivitäten oder Anforderungen ist klar festgelegt, in welcher Reihenfolge die Abarbeitung erfolgen muss.

Zur Erreichung einer möglichst schnellen Einigung über die Reihenfolge unter Einbeziehung von mehr als einer Person, eignet sich das im Folgenden beschriebene „Team Estimation Game“.

Ergebnis des Team Estimation Game ist eine sortierte Liste, wobei das Sortierkriterium frei festgelegt werden kann. Beispiele für mögliche Kriterien sind Relevanz, Aufwand, Größe oder Komplexität. Das Team Estimation Game kann also in jeder Situation angewendet werden, in der eine Menge von Dingen in eine Reihenfolge gebracht werden muss. Abbildung 46 illustriert den im Folgenden beschriebenen Ablauf eines Priorisierungsworkshops. In der Vorbereitung auf den Workshop müssen die zu priorisierenden Aktivitäten oder Anforderungen jeweils auf eine Karteikarte geschrieben oder gedruckt werden. Diese Karten werden dann als verdeckter Stapel auf den Tisch gelegt. Anschließend wird die erste Karte aufgedeckt und offen auf den Tisch gelegt (Vorbereitung). Nun deckt der erste Spieler eine zweite Karte auf und legt sie entweder links oder rechts neben die bereits liegende Karte auf den Tisch, je nachdem ob er die zweite Karte für wichtiger oder weniger wichtig hält (erster Spielzug). Der zweite Mitspieler hat nun folgende Optionen: Entweder er zieht eine neue Karte vom Stapel und sortiert diese in die seiner Meinung nach richtige Stelle zwischen die bereits aufgedeckten Karten ein, oder er tauscht zwei bereits offenliegende Karten, von denen er glaubt, dass sie nicht richtig liegen. Anschließend ist der nächste Spieler an der Reihe (n-ter Spielzug). Sobald die letzte Karte vom Stapel der verdeckten Karten eingereicht wurde, haben die Spieler nur noch die Möglichkeit, jeweils Karten zu tauschen. Das geschieht so lange, bis keiner der Spieler mehr tauschen möchte.

Abbildung 46: Ablauf Team Estimation Game



Quelle: erstellt im Auftrag der IU, 2023.

Zusammenspiel der Techniken zur Priorisierung von Anforderungen

Falls sehr viele Anforderungen in eine Reihenfolge gebracht werden müssen, können die vorgestellten Techniken kombiniert werden. In einem ersten Schritt kann die Wichtigkeit der Anforderungen mit der MoSCoW-Methode vorkategorisiert werden. Anschließend werden die Muss-Anforderungen in eine Wert-Risiko-Matrix eingetragen. Die wertvollsten und

risikoreichsten Anforderungen können nach Kano klassifiziert werden. Kommt bei der Klassifikation nach Kano und der Einordnung in die Wert-Risiko-Matrix keine ausreichend priorisierte Liste von Anforderungen heraus, können die wichtigsten Anforderungen noch mit Team Estimation in eine Ordnung gebracht werden. Auf Basis der Ordnung kann festgelegt werden, welche Anforderungen als nächstes umgesetzt werden.



ZUSAMMENFASSUNG

Softwareengineering ist ein erkenntnisgetriebener Prozess. Das Wissen der Stakeholder über das System und die Erfahrung reift und verändert sich mit jeder ausgelieferten Systemversion. Daher entwickeln sich Anforderungen im Laufe eines Projekts weiter und haben unterschiedliche Status, die je nach Stabilität der Anforderung variieren.

Darüber hinaus haben Anforderungen Attribute, die zu Metadaten aggregiert werden. Auf Basis dieser Metadaten können Sichten auf Anforderungen erzeugt werden. Die selektive Sicht zeigt gewisse Anforderungen, die nach bestimmten Attributen gefiltert werden. Die verdichtende Sicht liefert eine aggregierte Sichtweise auf die Anforderungen, und mit ihrer Hilfe können Aussagen über die Gesamtheit aller Anforderungen getroffen werden. Die Sichten lassen sich kombinieren.

Die Entwicklung von Anforderungen im Softwareprozess kann durch eine Versionierung ausgedrückt werden. Ein Abbild einer Kombination von Anforderungen, jede in einer bestimmten Version, wird Anforderungskonfiguration genannt. Anforderungen werden zunächst spezifiziert. Die folgende Umsetzung und der anschließende Test transformieren Anforderungen zu Software, welche nach erfolgreicher Abnahme durch den Kunden in seiner Umgebung produktiv gesetzt wird. Es wird von einem Lebenszyklus von Anforderungen gesprochen.

Eine typische und projektbegleitende Aufgabe ist die Priorisierung von Anforderungen und das Festlegen, welche Aktivitäten im Projekt als nächstes erfolgen sollen. Das ist bei Softwareprojekten nicht nur am Anfang relevant, wenn eine erste Priorisierung der zu Beginn des Projekts bekannten Anforderungen durchgeführt wird. Im Verlauf eines Projekts ändert sich in der Regel noch die Menge der fachlichen Anforderungen und manchmal auch die technischen oder organisatorischen Rahmenbedingungen. Durch die Kombination verschiedener Techniken kann eine große Menge von Anforderungen in vergleichsweise kurzer Zeit priorisiert werden.

ANHANG

LITERATURVERZEICHNIS

- Allweyer, T. (2020): *BPMN 2.0 – Business Process Modeling Notation. Einführung in den Standard für die Geschäftsprozessmodellierung*. 4. Aufl., Books on Demand, Norderstedt.
- Backerra, H./Malorny, C./Schwarz, W. (2019): *Kreativitätstechniken. Kreative Prozesse anstoßen, Innovationen fördern*. 4. Aufl., Hanser, München.
- De Bono, E. (2016): *Six thinking hats*. 4. Aufl., Penguin Books, London.
- Blaser, A./Hackl, C. (Hrsg.) (1977): *Interactive Systems*. Springer, Berlin.
- BPMN (2011): *BPMN 2.0 – Business Process Model and Notation*. (URL: http://www.bpmb.de/images/BPMN2_0_Poster_DE.pdf) [letzter Zugriff: 28.11.2022].
- Eclipse Tips – Prakash G. R. (2021): *WireframeSketcher – Review and free licenses*. (URL: <http://blog.eclipse-tips.com/2009/02/wireframesketcher-review-and-free.html>) [letzter Zugriff: 15.02.2021].
- Gadatsch, A. (2022): *Grundkurs Geschäftsprozess-Management. Methoden und Werkzeuge für die IT-Praxis: eine Einführung für Studenten und Praktiker*. 10. Aufl., Springer Vieweg, Wiesbaden.
- Keynotopia (2013): (URL: <http://keynotopia.com/guides-ppt>) [letzter Zugriff: 28.11.2022].
- Lauenroth, K. (2011): *Eine kleine praktische Philosophie über das Requirements Engineering*. (URL: <http://www.slideshare.net/adessoAG/vortrag-kim-lauenroth>) [letzter Zugriff: 28.11.2022].
- Marczak, S./Kwan, I./Damian, D. (2009): *Investigating Collaboration Driven by Requirements in Cross-Functional Software Teams*. In: *Proceedings of the 2009 Collaboration and Intercultural Issues on Requirements: Communication, Understanding and Softskills*. Washington, DC, USA.
- Miller, L. A./Thomas, J. C. Jr. (1977): *Behavioral issues in the use of interactive systems*. In: Blaser, A./Hackl, C. (Hrsg.): *Interactive Systems*. Bd. 49, Springer, Berlin.
- Nambisan, S./Wilemon, D. (2000): *Software development and new product development: potentials for cross-domain knowledge sharing*. In: *IEEE Transactions on Engineering Management*, Jg. 47, Heft 2, S. 211–220.
- Object Management Group (OMG) (2020): *Unified Modeling Language UML*. (URL: <http://www.uml.org>) [letzter Zugriff: 28.11.2022].
- Pohl, K. (2008): *Requirements Engineering: Grundlagen, Prinzipien, Techniken*. 2. Aufl., dpunkt.verlag, Heidelberg.

- Pohl, K./Rupp, C. (2015): *Basiswissen Requirements Engineering – Aus- und Weiterbildung zum „Certified Professional for Requirements Engineering“*. 4. Aufl., dpunkt.verlag, Heidelberg.
- Pohl, K./Rupp, C. (2021): *Basiswissen Requirements Engineering – Aus- und Weiterbildung zum „Certified Professional for Requirements Engineering“*. 5. Aufl., dpunkt.verlag, Heidelberg.
- Rupp, C./die SOPHISTen (2014): *Requirements-Engineering und -Management. Aus der Praxis von klassisch bis agil*. 6. Aufl., Hanser, München.
- Rupp, C./die SOPHISTen (2021): *Requirements-Engineering und -Management. Aus der Praxis von klassisch bis agil*. 7. Aufl., Hanser, München.
- Rupp, C./dieSOPHISTen/Queins, S. (2012): *UML 2 glasklar: Praxiswissen für die UML-Modellierung*. 4. Aufl., Hanser, München.
- Schwaber, K./Sutherland, J. (2017): *Der Scrum Guide - Der gültige Leitfaden für Scrum: Die Spielregeln*. (URL: <https://www.scrumguides.org/docs/scrumguide/v2017/2017-Scrum-Guide-German.pdf> [letzter Zugriff: 28.11.2022]).
- Trac.assembla (2014): *UISkizze*. (URL: <https://trac.assembla.com/sepm2009/wiki/UISkizze> [letzter Zugriff: 01.03.2014]).
- Wirdemann, R. (2011): *Scrum mit User Stories*. 2. Aufl., Hanser, München.
- Wirdemann, R. (2022): *Scrum mit User Stories*. 4. Aufl., Hanser, München.

ABBILDUNGSVERZEICHNIS

Abbildung 1: Kernaktivitäten im Requirements Engineering	14
Abbildung 2: Zusammenhang zwischen Problem, Anforderung und System	17
Abbildung 3: Zusammenhang von System, Systemkontext und irrelevanter Umgebung	23
Abbildung 4: Stakeholder-Priorisierungsmatrix	25
Tabelle 1: Stakeholder-Tabelle	26
Abbildung 5: Überblick über Techniken der Anforderungsermittlung	29
Abbildung 6: Experiment zur menschlichen Wahrnehmung	32
Abbildung 7: Natürlich-sprachliche Kommunikation zwischen Sender und Empfänger	33
Tabelle 2: Eignung konkreter Techniken zum praktischen Einsatz (1)	36
Tabelle 3: Eignung konkreter Techniken zum praktischen Einsatz (2)	37
Tabelle 4: Steckbrief „Perspektivwechsel“	40
Tabelle 5: Steckbrief „Verbales Brainstorming“	40
Abbildung 8: Beispiel für eine User Story	41
Tabelle 6: Steckbrief „Schriftliches Brainstorming mit User Stories“	42
Tabelle 7: Steckbrief „Workshop“	43
Abbildung 9: Schritte bei der Durchführung eines Interviews	45
Tabelle 8: Steckbrief „Interview“	46
Tabelle 9: Steckbrief „Fragebogen“	46
Tabelle 10: Steckbrief „Feldbeobachtung“	47
Tabelle 11: Steckbrief „Apprenticing“	48
Abbildung 10: Vertikale und horizontale Prototypen	49

Abbildung 11: Horizontaler Oberflächenprototyp als Handskizze	50
Tabelle 12: Steckbrief „Handskizze“	50
Abbildung 12: Horizontaler Oberflächenprototyp als Wireframe	51
Tabelle 13: Steckbrief „Wireframes“	51
Abbildung 13: Horizontaler Oberflächenprototyp als Mock-up	52
Tabelle 14: Steckbrief „Mock-up“	52
Tabelle 15: Steckbrief „Vertikaler Prototyp“	53
Abbildung 14: Beispiel für Skizzen	63
Abbildung 15: Beispiel für ein Modell, hier ein UML-Aktivitätsdiagramm	64
Abbildung 16: Beispiel für ein textuelles Modell in Form einer XML-Datei	65
Abbildung 17: Einliniensystem	71
Abbildung 18: Mehrliniensystem	72
Abbildung 19: Stabliniensystem	73
Abbildung 20: Bestandteile und Zusammenhänge der Bestandteile eines Geschäftsprozesses	75
Tabelle 16: Elemente eines Geschäftsprozesses	75
Abbildung 21: Strukturierung von Geschäftsprozessen	76
Abbildung 22: Beispiel für einen Bestellprozess als BPMN-Modell	77
Tabelle 17: Basisnotationselemente der BPMN ohne Gateways und Verbindungen	78
Abbildung 23: Beispiel für ein BPMN-Modell	81
Abbildung 24: BPMN - XOR und OR Gateway	81
Abbildung 25: BPMN - AND und event-driven Gateway	82
Tabelle 18: Zuordnung BPMN zu Geschäftsprozesselementen	83
Abbildung 26: EPK-Beispiel	86

Tabelle 19: Basisnotationselemente der EPK ohne Konnektoren	87
Abbildung 27: EPK - Konnektoren	89
Abbildung 28: Regeln zur Modellierung	90
Abbildung 29: eEPK - Beispiel	91
Tabelle 20: Zuordnung (e)EPK zu Geschäftselementen	92
Abbildung 30: Überblick Unified Modeling Language	96
Abbildung 31: Beziehung UML-Modell und UML-Diagramm	97
Abbildung 32: Beispiel für Konstruktionsregeln	98
Abbildung 33: Beispiel für ein UML-Use-Case-Diagramm	99
Tabelle 21: Wichtige Notationselemente im Use-Case-Diagramm	100
Abbildung 34: Beispiele für geeignete und ungeeignete Use Cases	101
Abbildung 35: Beispiel für ein UML-Aktivitätsdiagramm	103
Tabelle 22: Wichtige Notationselemente im Aktivitätsdiagramm	104
Tabelle 23: Verzweigungen und Zusammenführungen	105
Abbildung 36: Beispiel für UML-Klassendiagramm	107
Tabelle 24: Klassen im UML-Klassendiagramm	107
Tabelle 25: Typische Beziehungen zwischen Klassen	108
Tabelle 26: Darstellung von Beziehungen zwischen Klassen	109
Tabelle 27: Multiplizitäten im Klassendiagramm	110
Abbildung 37: Beispiel für UML-Zustandsdiagramm	113
Tabelle 28: Wichtige Notationselemente im UML-Zustandsdiagramm	114
Abbildung 38: Beispiel für Trigger, Guard und Aktivität im Zustandsdiagramm	115
Tabelle 29: Gegenüberstellung interner und externer Prüfer	124

Tabelle 30: Beispiel für eine tabellarische Darstellung von Anforderungsattributen (1) .	135
Tabelle 31: Beispiel für eine tabellarische Darstellung von Anforderungsattributen (2) .	136
Tabelle 32: Beispiel für eine selektive Sicht auf Anforderungen	136
Abbildung 39: Beispiel einer verdichtenden Sicht auf Anforderungen	137
Abbildung 40: Versionierung von Anforderungen	138
Abbildung 41: Beispiel für Anforderungskonfigurationen	139
Abbildung 42: Vereinfachte Darstellung der Zustände im Lebenszyklus von Anforderungen	140
Abbildung 43: Einflussfaktoren auf die Priorisierung von Anforderungen	141
Abbildung 44: Wert-Risiko-Matrix	143
Tabelle 33: Kano-Typen	143
Abbildung 45: Evolution von Anforderungen nach Kano	144
Abbildung 46: Ablauf Team Estimation Game	146



IU Internationale Hochschule GmbH
IU International University of Applied Sciences
Juri-Gagarin-Ring 152
D-99084 Erfurt



Postanschrift
Albert-Proeller-Straße 15-19
D-86675 Buchdorf



media@iu.org
www.iu.org



Hilfe & Kontakt (FAQ)
Antworten rund um Dein Studium findest
Du jederzeit auf myCampus.